UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

JULIANA RODRIGUES DE VARGAS

Hardware Accelerator for Matrix Inversion in Decentralized Massive MIMO LMMSE Equalization Under Colored Noise

Work presented in partial fulfillment
of the requirements for the degree of
Bachelor in Computer Engineering

Advisor: Prof. Dr. Leslie Lamport
Coadvisor: Prof. Dr. Donald Ervin Knuth

Porto Alegre
November 2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Reitor: Prof. Rui Vicente Oppermann
Vice-Reitora: Prof[a]. Jane Fraga Tutikian
Pró-Reitor de Graduação: Prof. Vladimir Pinheiro do Nascimento
Diretora do Instituto de Informática: Prof[a]. Carla Maria Dal Sasso Freitas
Coordenador do Curso de Engenharia de Computação: Prof. André Inácio Reis
Bibliotecária-chefe do Instituto de Informática: Beatriz Regina Bastos Haro

ABSTRACT

Este documento é um exemplo de como formatar documentos para o Instituto de Informática da UFRGS usando as classes LaTeX disponibilizadas pelo UTUG. Ao mesmo tempo, pode servir de consulta para comandos mais genéricos. *O texto do resumo não deve conter mais do que 500 palavras.*

Keywords: Formatação eletrônica de documentos. LaTeX. ABNT. UFRGS.

Using LaTeX to Prepare Documents at II/UFRGS

RESUMO

This document is an example on how to prepare documents at II/UFRGS using the LaTeX classes provided by the UTUG. At the same time, it may serve as a guide for general-purpose commands. *The text in the abstract should not contain more than 500 words.*

Palavras-chave: Electronic document preparation. LaTeX. ABNT. UFRGS.

LIST OF FIGURES

Figure 3.1 Resource elements of one PRB in one slot, with the DM-RS symbols. 29
Figure 3.2 Intervals of duration T_c that contain a data symbol, and their DM-RS REs per PRB, in the TDLB100-400 condition. 30
Figure 3.3 Lower triangular systolic array that computes a Gram matrix. 35

Figure 4.1 Simulation flow: QuaDRiGa channels, Python simulator and fixed-point inverse. 37
Figure 4.2 Detailed flow of the Python simulator: Monte Carlo loop, equalizers and SER. 39
Figure 4.3 Steps of the inversion module. Each block is one C++ function of the HLS code. 41
Figure 4.4 Loop pipelining in the LU decomposition. The code is simplified. 51
Figure 4.5 Dataflow between the stages of the inversion. The code is simplified. 52
Figure 4.6 Batch of 12 matrices with 6 copies, for the 606-cycle limit. The code is simplified. 53
Figure 4.7 Systolic array of the forward substitution and minimum step of the LU decomposition. The code is simplified. 54

Figure 5.1 Maior aumento de SNR, em dB, no grupo 1, configurações 1A a 1H ($M = 128$, $C = 8$, $M_c = 16$). 59
Figure 5.2 Aumento de SNR do BCD-MMSE sobre o LMMSE no grupo 1 ($M = 128$, $C = 8$, $M_c = 16$), no pior caso de cada barra. 60
Figure 5.3 Maior aumento de SNR, em dB, no grupo 2, configurações 2A a 2H ($M = 256$, $C = 8$, $M_c = 32$). 61
Figure 5.4 Aumento de SNR do BCD-MMSE sobre o LMMSE no grupo 2 ($M = 256$, $C = 8$, $M_c = 32$), no pior caso de cada barra. 62
Figure 5.5 Maior aumento de SNR, em dB, no grupo 3, configurações 3A a 3H ($M = 256$, $C = 16$, $M_c = 16$). 63
Figure 5.6 Aumento de SNR do BCD-MMSE sobre o LMMSE no grupo 3 ($M = 256$, $C = 16$, $M_c = 16$), no pior caso de cada barra. 64
Figure 5.7 Largest SNR increase, in dB, of each format in each mobility profile, in the three groups. 66
Figure 5.8 Curvas de SER dos três grupos na TDLC300-100, nos formatos escolhidos. 68
Figure 5.9 Uso de cada recurso do XCU50 por arquitetura, do menor ao maior entre os formatos. 70
Figure 5.10 Energia por inversão por formato e condição de teste. 75
Figure 5.11 Ocupação do recurso mais usado do XCU50 (esquerda) e potência dinâmica (direita), por formato e condição de teste. 76
Figure 5.12 Aumento de SNR e potência dinâmica de cada formato nas condições de teste, grupo 1. 78
Figure 5.13 Aumento de SNR e potência dinâmica de cada formato nas condições de teste, grupo 3. 79

Figure C.1 Curvas de SER dos três grupos na TDLB100-400, nos formatos escolhidos. 89
Figure C.2 Curvas de SER dos três grupos na TDLA30-5 e na TDLA30-10, nos formatos escolhidos. 90

Figure D.1 Largest SNR increase, in dB, of each configuration in each mobility profile, in the three groups, with the selected formats. 91

LIST OF TABLES

Table 2.1 TL-BD-INSA full detector pipeline hardware results (128×8, 64-QAM). FPGA: Virtex-7 XC7VX980T. ASIC: TSMC 45 nm((WANG et al., 2021)). 14

Table 3.1 Cenários do modelo 3GPP 38.901 no QuaDRiGa. 22
Table 3.2 Number of DM-RS REs per PRB available to every data symbol of the slot, in each test condition. 29
Table 3.3 Multiplicações complexas por PRB e por T_c em cada DU. 32
Table 3.4 Modos de quantização e de estouro do tipo `ap_fixed`. 34

Table 4.1 Types of the values in the inversion module, for the format $\langle W, I \rangle$ with $F = W - I$ fractional bits. 41
Table 4.2 Simulated configurations. 43
Table 4.3 Carrier frequency and number of noise samples in each test condition. 43
Table 4.4 Maximum interval between inversions in each test condition. 45
Table 4.5 Sizes tested in the initial sweep. 46
Table 4.6 $\mathbf{G}_c$ at the operating point of the initial-sweep configurations. 47
Table 4.7 Configurations in which each format was chosen in the initial sweep. 48
Table 4.8 Batch of the inversion module in each test condition. 50

Table 5.1 Formatos escolhidos em cada grupo de configurações. 65
Table 5.2 Menor e maior uso de recursos, caminho crítico e intervalo entre inversões de cada arquitetura, entre os formatos. 73
Table 5.3 Recurso mais usado do XCU50, em porcentagem, por formato e arquitetura. 73
Table 5.4 Potência dinâmica, em W, por formato e arquitetura. 74
Table 5.5 Energia por inversão, em μJ, por formato e arquitetura. 74
Table 5.6 Throughput and delay of one block of $N_{\text{PRB}} = 275$ inversions, format $\langle 24, 12 \rangle$. 75
Table 5.7 Formato escolhido em cada condição de teste, com o pior aumento de SNR, em dB, e a potência dinâmica, em W. 80

Table A.1 Channel parameters set in the MATLAB script that runs QuaDRiGa. 84
Table A.2 Parameters of the propagation scenario `3GPP_38.901_UMa_NLOS`: valid range and values used in the simulation. 85
Table A.3 Signal and noise parameters. 85
Table A.4 Receiver parameters: partitioning, covariance estimation and equalization. 86
Table A.5 Evaluation parameters. 86

Table B.1 Recursos, caminho crítico e intervalo entre inversões da versão sem otimização, por formato. 87
Table B.2 Recursos, caminho crítico e intervalo entre inversões das arquiteturas otimizadas, por formato. 88

LIST OF ABBREVIATIONS AND ACRONYMS

3GPP	3rd Generation Partnership Project
5G	Quinta geração de redes móveis
AoA	Angle of Arrival
AoD	Angle of Departure
AWGN	Additive White Gaussian Noise
BCD	Block Coordinate Descent
BS	Base Station
DM-RS	Demodulation Reference Signal
DU	Distributed Unit
FR1	Frequency Range 1
FR2	Frequency Range 2
HST	High Speed Train
InF	Indoor Factory
InH	Indoor Hotspot
IoTh	Interference over Thermal
ITU	International Telecommunication Union
LMMSE	Linear Minimum Mean Square Error
LoS	Line of Sight
MIMO	Multiple-Input Multiple-Output
MMSE	Minimum Mean Square Error
NLoS	Non-Line of Sight
NR	New Radio
O2I	Outdoor-to-Indoor
OFDM	Orthogonal Frequency Division Multiplexing

PRB	Physical Resource Block
PUSCH	Physical Uplink Shared Channel
QAM	Quadrature Amplitude Modulation
QPSK	Quadrature Phase Shift Keying
QuaDRiGa	Quasi-Deterministic Radio Channel Generator
RB	Resource Block
RE	Resource Element
RMa	Rural Macro-cell
SCS	Subcarrier Spacing
SER	Symbol Error Rate
SNR	Signal-to-Noise Ratio
TR	Technical Report
TS	Technical Specification
UE	User Equipment
ULA	Uniform Linear Array
UMa	Urban Macro-cell
UMi	Urban Micro-cell
UPA	Uniform Planar Array
ZF	Zero-Forcing

CONTENTS

1 INTRODUCTION 12
2 TRABALHOS RELACIONADOS 14
2.1 Cholesky-Based Inversion in MIMO Hardware 14
2.2 Decentralized Detection on FPGA 15
2.3 Gap Synthesis 15
3 FUNDAMENTAÇÃO TEÓRICA 17
3.1 Sistemas MIMO 17
3.1.1 Uplink 17
3.1.2 Símbolos e constelação 17
3.1.3 Relação sinal-ruído 19
3.1.4 Modelo de canal 19
3.1.4.1 Tipos de modelo de canal 19
3.1.4.2 Multipath, desvanecimento... 20
3.1.4.3 Bloco de Coerência 20
3.1.4.4 Arranjo de canal 21
3.1.4.5 Modelo 3GPP e Simulador QuaDRiGa 21
3.1.4.6 Cenário de propagação 21
3.1.4.7 Parâmetros do simulador 22
3.1.5 Equalização 23
3.1.5.1 Detector de máxima verossimilhança 23
3.1.5.2 Zero-forcing 23
3.1.5.3 Covariância 23
3.1.5.4 Filtro de Wiener 23
3.1.5.5 Equalizador LMMSE 23
3.1.5.6 Detecção e taxa de erro de símbolo 23
3.2 Equalização descentralizada sob ruído colorido 23
3.2.1 Ruído 23
3.2.1.1 Ruído branco (AWGN) 24
3.2.1.2 Assinatura espacial e Interferência 24
3.2.1.3 Matriz de covariância do ruído 25
3.2.1.4 Interferência sobre ruído térmico (IoTh) 26
3.2.1.5 Estimação da matriz de covariância 27
3.2.1.6 Availability of noise samples within the coherence time 27
3.2.1.7 Condicionamento da estimativa 29
3.2.2 Processamento descentralizado 31
3.2.3 Equalizador BCD-MMSE 31
3.2.3.1 Complexidade 32
3.3 Inversão de Matrizes 33
3.3.1 Inversao de Matrizes 33
3.3.1.1 Decomposicao de Cholesky 33
3.3.1.2 Representação em ponto fixo 33
3.3.1.3 Latência, intervalo e vazão 33
3.3.1.4 Pipeline de tarefas 34
3.3.1.5 Arranjo sistólico 35
4 METHODOLOGY 36
4.1 Simulation workflow 36
4.1.1 Channel generation 37
4.1.2 Python simulator 38

4.1.3 Fixed-point inverse 38
4.2 Hardware setup 40
4.2.1 Tools and target device 40
4.2.2 Inversion module 40
4.3 Configurations and test conditions 42
4.3.1 Base station configurations 42
4.3.2 Test conditions 42
4.4 Inversion cycle limit 44
4.4.1 Sample collection 44
4.4.2 Throughput 44
4.5 Selection of the fixed-point format 45
4.5.1 SNR increase 45
4.5.2 Initial sweep 46
4.5.3 Configurations of the initial sweep 46
4.5.4 Synthesis and selection of formats for validation 47
4.5.5 Validation 48
4.6 Hardware architectures, test matrices and power 49
4.6.1 Hardware architectures 50
4.6.1.1 Loop pipelining 50
4.6.1.2 Dataflow between stages 51
4.6.1.3 Matrix batches 52
4.6.1.4 Systolic arrays 53
4.6.2 Test matrices 54
4.6.3 Power and energy 55
5 RESULTADOS 56
5.1 Simulation setup 56
5.1.1 Channel Generation in QuaDRiGa 56
5.1.2 Python Simulator 57
5.1.2.1 Signal and Noise 57
5.1.2.2 Receiver 58
5.1.2.3 Evaluation 58
5.2 Selection and analysis of word format 58
5.2.1 Grupo 1: $M = 128, C = 8$ 59
5.2.2 Grupo 2: $M = 256, C = 8$ 61
5.2.3 Grupo 3: $M = 256, C = 16$ 62
5.2.4 Formatos escolhidos 64
5.3 Mobility profiles 65
5.4 SER curves 67
5.5 Arquiteturas de hardware 69
5.5.1 Recursos por arquitetura 69
5.5.1.1 Pipeline de laços 70
5.5.1.2 Dataflow entre etapas 70
5.5.1.3 Lotes de matrizes 71
5.5.1.4 Arranjos sistólicos 71
5.5.2 Comparação entre as arquiteturas 72
5.5.3 Potência e energia 73
5.6 Vazão e atraso 74
5.7 Aumento de SNR e potência 77
6 CONCLUSIONS 81
6.1 Future work 81
REFERENCES 82

APPENDIX A — SIMULATION PARAMETERS **84**
APPENDIX B — HARDWARE RESOURCES **87**
APPENDIX C — SER CURVES **89**
APPENDIX D — SNR INCREASE PER CONFIGURATION **91**

1 INTRODUCTION

The evolution toward fifth-generation (5G) and sixth-generation (6G) wireless systems demands ever-increasing spectral efficiency and data throughput, driving the adoption of Massive Multi-User Multiple-Input Multiple-Output (MU-MIMO) as a key enabling technology (LI et al., 2017b). By equipping base stations with hundreds of antenna elements serving tens of users simultaneously, Massive MU-MIMO achieves fine-grained spatial multiplexing that small-scale MIMO systems cannot provide. Among the available uplink detection strategies, Linear Minimum Mean-Square Error (LMMSE) equalization is widely adopted for its near-optimal performance (ZHAO et al., 2025). Its closed-form solution, however, requires a matrix inversion whose complexity grows cubically with the number of antennas, and for a base station with 256 antennas operating at 80 MHz bandwidth with 12-bit converters, the raw fronthaul data rate approaches 1 Tbps, far beyond the capacity of existing interconnect standards such as the Common Public Radio Interface (CPRI) (LI et al., 2019).

Decentralized Baseband Processing (DBP) architectures address both bottlenecks by partitioning the base-station antenna array into clusters of M_c antennas, each served by a local Distributed Unit (DU) that performs baseband processing independently. This reduces per-cluster matrix dimensions and eliminates the need to transfer raw antenna data to a centralized processor (JEON et al., 2019). Existing decentralized LMMSE methods, however, almost universally assume that receiver noise follows an ideal Additive White Gaussian Noise (AWGN) model with a diagonal covariance matrix, an assumption that typically does not hold in practice, where interference from neighboring cells produces colored noise with a non-diagonal covariance structure (ZHAO et al., 2025). Under colored noise, the standard inversion-avoidance techniques used by AWGN-based methods lose their convergence guarantees (WANG et al., 2023), and all known decentralized LMMSE solutions for this regime require an explicit matrix inversion

The Block Coordinate Descent MMSE (BCD-MMSE) algorithm was recently proposed (ZHAO et al., 2025) as the first decentralized LMMSE equalizer to operate under colored noise. In BCD-MMSE, each DU performs a local $M_c \times M_c$ Hermitian matrix inversion once per coherence block, reusing the result across all received symbols. Despite its algorithmic completeness, no hardware implementation of this inversion has been reported: the work of (ZHAO et al., 2025) provides neither hardware resource use, nor power, nor latency figures for any target platform. Field-Programmable Gate Ar-

rays (FPGAs) are a promising target for this operation, offering higher energy efficiency and lower latency than GPU-based solutions at the power budgets required by distributed base-station nodes (JEON et al., 2019; LI et al., 2019).

This work proposes and evaluates an FPGA accelerator for the $M_c \times M_c$ Hermitian matrix inversion required by each DU in the BCD-MMSE algorithm. The accelerator is based on Cholesky decomposition, an exact inversion method applicable under a non-diagonal noise covariance. The contributions are evaluated in terms of detection quality (Bit Error Rate - BER), resource usage (LUT count and DSP blocks), and power consumption.

2 TRABALHOS RELACIONADOS

Related works found in the literature can be divided into: (a) works that implement Cholesky-based inversion in FPGA or ASIC in the context of MIMO technology and (b) works that implement decentralized equalization in FPGA.

2.1 Cholesky-Based Inversion in MIMO Hardware

The TL-BD-INSA detector of (WANG et al., 2021) implements Cholesky-based inversion of Hermitian block-diagonal sub-matrices on a Virtex-7 XC7VX980T FPGA and a TSMC 45 nm ASIC. The FPGA implementation targets a 128×8 MIMO configuration with 64-QAM. The inversion module uses a triangular systolic array for the Cholesky factorization, followed by parallel forward and backward substitution units, one per user equipment. This architecture is the closest available hardware reference for the $M_c \times M_c$ inversion required by BCD-MMSE, differing only in that TL-BD-INSA operates under AWGN and inverts block-diagonal sub-matrices of the Gram matrix, whereas BCD-MMSE inverts the full local noise-weighted matrix under colored noise.

Table 2.1 reports the implementation metrics for the complete TL-BD-INSA detector pipeline, which includes matched filtering, Gram matrix computation, the Cholesky inversion module, and symbol estimation (WANG et al., 2021). These figures therefore represent an upper bound on the resources attributable to the inversion step alone; the paper does not provide a per-module area or resource breakdown.

Table 2.1: TL-BD-INSA full detector pipeline hardware results (128×8, 64-QAM). FPGA: Virtex-7 XC7VX980T. ASIC: TSMC 45 nm((WANG et al., 2021)).

Metric	**FPGA**	**ASIC**
Gate count / LUT slices	13,640 LUT	572 kGE
Flip-flop slices	19,460 FF	—
DSP48 units	277	—
Clock frequency [MHz]	310	1,000
Throughput	57.3 Mbps	185 Mbps
Power [W]	—	0.144
Energy efficiency	1,731 bps/slice	1.28 Gbps/W

2.2 Decentralized Detection on FPGA

The FD-LAMA detector of (LI et al., 2017a), implemented on a Virtex-7 XC7VX690T using High-Level Synthesis (HLS) for a 128×8 system with QPSK and i.i.d. Rayleigh fading, achieves 22.2 Mb/s throughput at 429 MHz with 11,673 LUTs and 213 DSP48 blocks for $C = 1$ cluster. Resource utilization scales roughly linearly with C, while throughput degrades only slightly, reaching 17.8 Mb/s at $C = 4$ (LI et al., 2017a). This work assumes AWGN and does not include a matrix inversion module: FD-LAMA avoids explicit inversion entirely, operating only on local Gram matrix-vector products within each cluster.

The decentralized Newton-method detector of (KULKARNI; OUAMEUR; MASSICOTTE, 2021), implemented on a Virtex-7 XC7VX690T for a 128×8 system with $C = 4$ clusters in a ring topology, is architecturally equivalent to the BCD-MMSE daisy-chain in its inter-DU communication pattern. It achieves 253 Mbps throughput at 340 MHz, consuming 100,733 LUTs, 1,280 DSP48 blocks, and 7.248 W on-chip power for 16-QAM with $T = 3$ iterations (KULKARNI; OUAMEUR; MASSICOTTE, 2021). The ring topology maintains constant interconnect bandwidth as the MIMO configuration scales, a property shared by the BCD-MMSE daisy-chain. As with all other decentralized hardware works, this implementation assumes AWGN and provides no inversion metrics specific to the colored-noise regime.

2.3 Gap Synthesis

Algorithmic papers also explicitly identify FPGA or ASIC implementation as future work: (JEON et al., 2019) states that decentralized solutions are expected to be deployed on FPGA or ASIC clusters for higher computing efficiency and lower transfer latency, and (LI et al., 2019) notes that multi-FPGA and multi-ASIC designs are left for future work to reduce power relative to GPU baselines.

None of the surveyed hardware works simultaneously addresses the three conditions that define the gap motivating this work: (1) colored noise with a non-diagonal local covariance $\hat{\mathbf{R}}_{cc}$; (2) decentralized LMMSE equalization under the BCD-MMSE framework; and (3) FPGA implementation with measured resource metrics. The algorithmic solution exists in (ZHAO et al., 2025), which proves optimality within the decentralized constraint and near-centralized BER performance for BCD-MMSE but provides no LUT

count, power, or latency figures for any target platform. This work fills that gap by providing the first hardware analysis of the $M_c \times M_c$ Hermitian Cholesky inversion at the core of the BCD-MMSE update.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Sistemas MIMO

3.1.1 Uplink

Este trabalho considera o *uplink* de um sistema MIMO (*Multiple-Input Multiple-Output*) massivo, em que K Equipamentos de Usuário (UEs, *User Equipments*) alvo, cada um com uma antena, transmitem dados para uma Estação Base (BS, *Base Station*) com M antenas, sendo $M \gg K$ (ZHAO et al., 2025). Os UEs são atendidos ao mesmo tempo, nos mesmos recursos de tempo e frequência (ZHAO et al., 2025).

O sistema segue o 5G NR (*New Radio*), a tecnologia de acesso sem fio da quinta geração de redes móveis (5G) (LIN; LEE, 2021, p. 2). O NR é especificado pelo 3GPP (*3rd Generation Partnership Project*) em um conjunto de normas técnicas, como a TS 38.211, que define os canais físicos e a modulação (3GPP, 2025), e a TS 38.104, que define a transmissão e a recepção da BS (3GPP, 2020). O NR divide o espectro em duas faixas de frequência: FR1 (*Frequency Range* 1), de 410 MHz a 7125 MHz, e FR2, de 24250 MHz a 52600 MHz (3GPP, 2020, Tab. 5.1-1). No *uplink*, os dados dos UEs são transmitidos no canal físico compartilhado de *uplink* (PUSCH, *Physical Uplink Shared Channel*) (3GPP, 2025, Sec. 6.3.1).

3.1.2 Símbolos e constelação

Cada UE transmite símbolos de uma constelação de modulação, como a 16-QAM (*Quadrature Amplitude Modulation*) (ZHAO et al., 2025). A energia média por símbolo é E_s (ZHAO et al., 2025).

Neste trabalho, são usadas a 16-QAM e a QPSK (*Quadrature Phase-Shift Keying*). A QPSK tem 4 pontos e leva 2 bits por símbolo, e a 16-QAM leva 4 bits por símbolo (TSE; VISWANATH, 2005, p. 56 e 60). A probabilidade de erro depende da distância mínima entre os pontos da constelação (TSE; VISWANATH, 2005, p. 386). Com a mesma potência, quanto mais pontos a constelação tem, mais perto eles ficam uns dos outros (TSE; VISWANATH, 2005, p. 57 e 386). Assim, para a mesma probabilidade de erro, a 16-QAM precisa de mais SNR que a QPSK (TSE; VISWANATH, 2005, p. 60).

No NR, esses símbolos são transmitidos com OFDM (*Orthogonal Frequency Di-*

vision Multiplexing). O espaçamento entre subportadoras (SCS, *Subcarrier Spacing*) depende da numerologia μ, com $\Delta f = 2^{\mu} \cdot 15$ kHz (3GPP, 2025, Tab. 4.2-1). Por exemplo, $\mu = 1$ corresponde a 30 kHz, e $\mu = 3$ corresponde a 120 kHz.

No tempo, as transmissões são organizadas em quadros de 10 ms, e cada quadro tem dez subquadros de 1 ms (3GPP, 2025, Sec. 4.3.1). O slot é a divisão do subquadro usada na transmissão. Com prefixo cíclico normal, cada slot tem 14 símbolos OFDM, e cada subquadro tem 2^{μ} slots (3GPP, 2025, Tab. 4.3.2-1). Assim, um slot dura 0,5 ms com $\mu = 1$ e 0,125 ms com $\mu = 3$.

O UE só transmite o PUSCH quando a BS lhe concede recursos para isso. Essa concessão indica o slot e os recursos de frequência do PUSCH (LIN; LEE, 2021, p. 293). Assim, um slot só leva o PUSCH de um UE quando esse UE tem dados para enviar e foi escalonado. O *uplink* e o *downlink* também precisam dividir o espectro. Na duplexação por divisão no tempo (TDD, *Time Division Duplex*), os dois usam a mesma banda de frequência, em tempos diferentes (LIN; LEE, 2021, p. 104). Na duplexação por divisão na frequência (FDD, *Frequency Division Duplex*), os dois usam bandas diferentes ao mesmo tempo (LIN; LEE, 2021, p. 104). O TDD é cada vez mais comum nas bandas de frequência mais altas (LIN; LEE, 2021, p. 10). Com TDD, só os slots de *uplink* podem levar o PUSCH.

A grade de recursos combina subportadoras, na frequência, e símbolos OFDM, no tempo. Cada elemento dessa grade é um elemento de recurso (RE, *Resource Element*), que corresponde a uma subportadora em um símbolo OFDM (3GPP, 2025, Sec. 4.4.3). Um bloco de recursos (RB, *Resource Block*) tem 12 subportadoras consecutivas na frequência (3GPP, 2025, Sec. 4.4.4.1). Os blocos de recursos físicos (PRBs, *Physical Resource Blocks*) são os RBs usados dentro da parte da banda configurada para a transmissão (3GPP, 2025, Sec. 4.4.4.4).

O número de PRBs da transmissão é N_{PRB}. A parte da banda configurada pode ter de 1 a 275 RBs (LIN; LEE, 2021, p. 367). Esse limite vem da grade de recursos: a norma numera as suas subportadoras de 0 a 3299 (3GPP, 2025, Sec. 4.4.2). São, portanto, até 3300 subportadoras. Como cada RB tem 12 subportadoras, isso equivale a $3300/12 = 275$ RBs. As larguras de banda da norma ficam um pouco abaixo desse limite. O maior número de PRBs é 273 com SCS de 30 kHz e banda de 100 MHz, e 264 com SCS de 120 kHz e banda de 400 MHz (3GPP, 2020, Tabs. 5.3.2-1 e 5.3.2-2). Com 275 PRBs, as subportadoras ocupariam 3300×30 kHz $= 99$ MHz e 3300×120 kHz $= 396$ MHz, quase toda a banda. Como o receptor processa cada PRB, $N_{\text{PRB}} = 275$ é o pior caso do

5G NR (ZHAO et al., 2025).

~~Resource Element (RE) é a menor unidade básica de transmissão, correspondendo a uma subportadora no domínio da frequência e um símbolo OFDM no domínio do tempo. Slot é a unidade básica de tempo para o agendamento de transmissões na rede, representando um bloco temporal contínuo. No padrão 5G NR, cada PRB possui 12 subportadores consecutivas na frequência, e cada slot possui 14 símbolos.~~

3.1.3 Relação sinal-ruído

A relação sinal-ruído (SNR, *Signal-to-Noise Ratio*) compara a energia média por símbolo com a potência do ruído térmico em cada antena, σ^2 (Seção 3.2.1). Ela é definida como (ZHAO et al., 2025)

$$\mathrm{SNR} = 10 \log_{10} \left(\frac{E_s}{\sigma^2} \right). \tag{3.1}$$

3.1.4 Modelo de canal

A avaliação de um sistema de MIMO massivo precisa de um modelo que represente o canal com as principais características de um arranjo com muitas antenas. Esse modelo deve considerar a geometria do arranjo de antenas, a correlação espacial entre as respostas de canal de diferentes antenas e as características físicas do ambiente, como a localização e orientação das ~~Estações Base (BSs) e dos Equipamentos de Usuário (UEs.)~~ BSs e dos UEs.

3.1.4.1 Tipos de modelo de canal

Existem dois tipos principais de modelo de canal: determinísticos ou estocásticos. O modelo determinístico produz sempre o mesmo resultado sob as mesmas condições, pois depende de um ambiente específico com parâmetros fixos, como a localização dos transmissores, receptores, espalhadores e refletores de sinal. Exemplos desse modelo são o ray–tracing e medições reais de canal em campo. Nesse modelo há desvantagem para a reprodução de resultados, que exige o acesso aos dados de medição específicos do canal ou modelos 3D de prédios, que nem sempre estão disponíveis.

Modelos estocásticos são independentes de um ambiente em particular e general-

izam o comportamento do canal a partir de propriedades estatísticas selecionadas. Eles podem ser divididos em três categorias:

- Baseados em correlação: Nos modelos baseados em correlação as respostas do canal seguem uma distribuição gaussiana com média zero, definidas por meio de matrizes de correlação. Um exemplo é o correlated Rayleigh fading model, que considera um desvanecimento plano em frequência, ou seja, que todos os componentes multi-path chegam à antena com o mesmo delay. Esse modelo é suficiente para uma análise dentro de um bloco de coerência ~~(PRB)~~ (um PRB, na frequência, durante T_c, no tempo), em que o canal é considerado constante.
- Paramétricos: Esses modelos definem distribuições estatísticas para o número de espalhadores, atrasos, potências e ângulos de chegada e saída (AoA/AoD). Eles não consideram a geometria do ambiente, por isso não abrangem a análise da evolução temporal do canal que acontece com a movimentação dos ~~Equipamentos de Usuário~~ UEs.
- Baseados em geometria: Esses modelos equilibram as características dos modelos estocásticos e determinísticos. Eles definem uma distribuição da localização física dos espalhadores de sinal ao redor dos transmissores e receptores. Os caminhos de propagação do canal são modelados a partir da localização dos espalhadores de uma maneira quase-determinística (quasi-deterministic). Um modelo quase-determinístico define um processo regido por leis fixas mas com uma margem de aleatoriedade dada por ruído e/ou variabilidade.

3.1.4.2 Multipath, desvanecimento...

3.1.4.3 Bloco de Coerência

O tempo de coerência T_c corresponde ao intervalo durante o qual o canal pode ser considerado aproximadamente constante. Seu valor é calculado a partir do deslocamento Doppler máximo f_D, pela expressão $T_c = 1/(8f_D)$ (TSE; VISWANATH, 2005). Durante cada bloco de coerência, o canal e a matriz de covariância do ruído são considerados constantes, permitindo reutilizar a matriz de equalização $\mathbf{W}$ ao longo desse intervalo (ZHAO et al., 2025).

As condições de teste seguem os cenários de propagação do Anexo G da norma 3GPP TS 38.104 (3GPP, 2020). São adotados SCS de 30 kHz em FR1 e 120 kHz em FR2 (3GPP, 2020, Tabs. 5.3.5-1 e 5.3.5-2), com 14 símbolos OFDM por slot (3GPP, 2025,

Tab. 4.3.2-1). O tempo de coerência e o número de símbolos OFDM nesse intervalo, para cada condição de teste, estão na Tabela 3.2 (Seção 3.2.1.6).

3.1.4.4 Arranjo de canal

A escolha dos arranjos de antena (ou seja, suas distribuições no espaço) dependem do caso de uso, da frequência dos portadores e da área de cobertura. Os arranjos podem ser organizados em geometrias lineares (Uniform Linear Array - ULA), planares (Uniform Planar Array - UPA) ou em geometrias cilíndricas. Modelos lineares e planares separam os UEs baseados na elevação ou posição azimutal, enquanto os cilíndricos conseguem trabalhar nos dois domínios ao mesmo tempo. Para cenários urbanos, o arranjo cilíndrico é mais adequado, pois permite a representação dos UEs em diferentes localizações e andares de prédios.

3.1.4.5 Modelo 3GPP e Simulador QuaDRiGa

Linha de Visada (Line of Sight - LoS) é o trajeto livre de obstáculos que permite a visão direta entre dois pontos. Não é realista representar os caminhos de propagação apenas como linha de visada. A 3GPP definiu para sistemas MIMO um modelo estocástico baseado em geometria que representa um misto de caminhos de propagação NLos e LoS. Esse modelo une um layout determinístico do sistema MIMO (ocalização das BS e UE, orientação das antenas, dependência angular da intensidade das ondas de rádio transmitidas ou recebidas, frequência de subportadoras) e um conjunto de parâmetros aleatórios de larga escala gerados por distribuições estatísticas obtidas a partir de medições de canal. O simulador de canal de código aberto QuaDRiGa (Quasi-Deterministic Radio Channel Generator) implementa o modelo 3GPP 3D MIMO, e é o simulador utilizado neste trabalho.

3.1.4.6 Cenário de propagação

Um cenário de propagação descreve um conjunto de parâmetros estatísticos para um tipo de ambiente. Dentre os modelos de cenário, o trabalho utiliza o 3GPP NR (~~TR 39.901~~ TR 38.901), [que modela os parâmetros para o New Radio, que é uma frequencia maior, pior caso]. Os ambientes do modelo ~~3G– 38.901~~ 3GPP TR 38.901 podem ter as seguintes características:

- ~~Uma~~ UMa (*Urban Macro-cell*): cidade densa, com antenas acima dos telhados.
- ~~Umi~~ UMi (*Urban Micro-cell*): cidade densa, com uma antena pequena abaixo dos telhados, no nível da rua.
- RMa (*Rural Macro-cell*): área rural, com células grandes.
- InH (~~*Indoor Office*~~ *Indoor Hotspot*): dentro de prédios, como escritórios.
- InF (*Indoor Factory*): dentro de galpões de fábrica.

Além disso, cada ambiente pode ter variações:

- LOS/NLOS: com ou sem visada direta entre antena e usuário.
- O2I (*Outdoor-to-Indoor*): antena do lado de fora e usuário dentro de prédio.

Cada cenário do modelo é definido por um arquivo de configuração. A tabela 3.1 resume os cenários do modelo. Este trabalho usa o cenário `3GPP_38.901_UMa_NLOS`, que representa uma macrocélula urbana (*Urban Macro* - UMa) sem linha de visada (NLos) entre a BS e os UEs.

Table 3.1: Cenários do modelo 3GPP 38.901 no QuaDRiGa.

Cenário	Lugar	Dados do manual
`3GPP_38.901_UMa`	Cidade densa, antena no alto do prédio	Célula de até ∼1 km, BS a 25 m, usuário de 1,5 a 22,5 m
`3GPP_38.901_UMi`	Cidade densa, antena na rua, abaixo dos telhados	Célula de até ∼200 m, BS a 10 m
`3GPP_38.901_RMa`	Área rural	Célula de até ∼10 km, BS de 10 a 150 m
`3GPP_38.901_Indoor`	Escritório, como Wi-Fi ou femtocélula	BS a 3 m, usuário a 1 m. Tem dois tipos, *mixed office* e *open office*, que só mudam a probabilidade de LOS.
`3GPP_38.901_InF`	Galpão de fábrica com máquinas, linhas de montagem e prateleiras	Versões NLOS por densidade de obstáculos (D: denso, S: esparso) e altura da BS (H: alta, L: baixa): DH, DL, SH e SL

Fonte: manual do QuaDRiGa.

3.1.4.7 Parâmetros do simulador

Para além dos parâmetros fixos pelo cenário de simulação, uma série de parâmetros deve ser definida. Este trabalho utiliza os parâmetros definidos em (ZHAO et al., 2025).

3.1.5 Equalização

3.1.5.1 Detector de máxima verossimilhança

3.1.5.2 Zero-forcing

3.1.5.3 Covariância

3.1.5.4 Filtro de Wiener

3.1.5.5 Equalizador LMMSE

3.1.5.6 Detecção e taxa de erro de símbolo

Depois da equalização, cada entrada do vetor estimado $\hat{s}$ é levada ao ponto mais próximo da constelação (ZHAO et al., 2025). Essa é a regra do vizinho mais próximo (TSE; VISWANATH, 2005, p. 503). A detecção erra quando o ponto escolhido é diferente do símbolo transmitido. A taxa de erro de símbolo (SER, *Symbol Error Rate*) é a probabilidade desse erro (TSE; VISWANATH, 2005, p. 503). Ela cai quando a SNR aumenta, e o desempenho de um equalizador é mostrado pela curva de SER em função da SNR (ZHAO et al., 2025). Para comparar dois esquemas, é comum usar a diferença de SNR necessária para atingir a mesma probabilidade de erro, que é a distância horizontal entre as duas curvas (TSE; VISWANATH, 2005, p. 55).

3.2 Equalização descentralizada sob ruído colorido

3.2.1 Ruído

O equalizador LMMSE (*Linear Minimum Mean Square Error*) depende da matriz de covariância do ruído (ZHAO et al., 2025). Essa matriz descreve a potência do ruído em cada antena da BS e a relação entre as antenas. Quando o ruído é só térmico (AWGN), ela é proporcional à identidade. Os sinais dos UEs não-alvo também chegam às antenas da BS e tornam o ruído correlacionado entre elas. Neste trabalho, a soma dessa interferência com o ruído térmico é chamada de ruído colorido.

3.2.1.1 Ruído branco (AWGN)

No modelo AWGN (Additive White Gaussian Noise), considera-se que o ruído é gaussiano, aditivo e espacialmente branco. Sua matriz de covariância é dada por:

$$\mathbf{R}_n = \sigma^2 \mathbf{I}_M,$$

em que σ^2 representa a potência do ruído térmico por antena e $\mathbf{I}_M$ é a matriz identidade de dimensão $M \times M$. Como a matriz possui apenas elementos não nulos na diagonal, assume-se que os ruídos térmicos observados nas diferentes antenas são independentes. Nesse cenário, o equalizador LMMSE pode ser escrito como (JEON et al., 2019):

$$\mathbf{W}_{\text{LMMSE}} = \left(\mathbf{H}^H \mathbf{H} + \rho \mathbf{I}_K\right)^{-1} \mathbf{H}^H,$$

em que $\rho = \dfrac{\sigma^2}{E_s}$, E_s representa a energia média dos símbolos transmitidos e K é o número de usuários alvo.

3.2.1.2 Assinatura espacial e Interferência

O sinal de um UE chega às M antenas da BS multiplicado pelo ganho do canal em cada antena. Para um único transmissor, o sinal recebido é:

$$\mathbf{y} = \mathbf{h}s + \mathbf{n},$$

em que $\mathbf{h} \in \mathbb{C}^{M \times 1}$ reúne esses ganhos. Esse vetor é denominado assinatura espacial, porque descreve como o sinal do UE se distribui pelas antenas.

Cada um dos r UEs não-alvo também tem sua assinatura espacial. Assim, a mesma interferência chega a todas as antenas, com ganhos diferentes, e fica correlacionada entre elas. Já o ruído térmico é independente em cada antena.

Pode-se definir o modelo de sinal recebido no cenário de interferência como:

$$\mathbf{y} = \mathbf{H}\mathbf{s} + \mathbf{n} = \mathbf{H}\mathbf{s} + \overline{\mathbf{H}}\,\overline{\mathbf{s}} + \overline{\mathbf{n}},$$

em que $\mathbf{H} \in \mathbb{C}^{M \times K}$ é a matriz de canal dos K usuários-alvo, $\mathbf{s} \in \mathbb{C}^{K \times 1}$ é o vetor de símbolos transmitidos por esses usuários, $\overline{\mathbf{H}} \in \mathbb{C}^{M \times r}$ representa os canais dos usuários interferentes e $\overline{\mathbf{s}} \in \mathbb{C}^{r \times 1}$ reúne seus símbolos transmitidos. O vetor $\overline{\mathbf{n}} \in \mathbb{C}^{M \times 1}$ representa

o ruído térmico nas antenas receptoras.

$$\overline{\mathbf{H}} = \underbrace{\begin{bmatrix} \bar{h}_{11} & \cdots & \bar{h}_{1r} \\ \vdots & \ddots & \vdots \\ \bar{h}_{M1} & \cdots & \bar{h}_{Mr} \end{bmatrix}}_{M\times r}, \qquad \bar{\mathbf{s}} = \underbrace{\begin{bmatrix} \bar{s}_1 \\ \vdots \\ \bar{s}_r \end{bmatrix}}_{r\times 1}, \qquad \bar{\mathbf{n}} = \underbrace{\begin{bmatrix} \bar{n}_1 \\ \vdots \\ \bar{n}_M \end{bmatrix}}_{M\times 1}.$$

Para construir o modelo do ruído colorido, considera-se que os usuários interferentes transmitem de forma independente com potência média β, de modo que:

$$\mathbb{E}[\bar{\mathbf{s}}\,\bar{\mathbf{s}}^H] = \beta\mathbf{I}_r = \underbrace{\begin{bmatrix} \beta & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \beta \end{bmatrix}}_{r\times r}.$$

Os elementos em destaque na diagonal são a potência β de cada interferente. Os zeros fora da diagonal indicam que os interferentes são independentes entre si.

O ruído térmico tem média nula e covariância $\sigma^2\mathbf{I}_M$, como no modelo AWGN. Os símbolos dos interferentes e o ruído térmico são independentes entre si e de média nula. A matriz $\overline{\mathbf{H}}$ é constante durante o intervalo de coerência.

3.2.1.3 Matriz de covariância do ruído

O ruído colorido é $\mathbf{n} = \overline{\mathbf{H}}\,\bar{\mathbf{s}} + \bar{\mathbf{n}}$, e sua matriz de covariância é $\mathbf{R}_n \triangleq \mathbb{E}[\mathbf{n}\mathbf{n}^H]$. Como as duas parcelas são independentes e de média nula, a covariância da soma é a soma das covariâncias:

$$\mathbf{R}_n = \mathrm{Cov}\left(\overline{\mathbf{H}}\,\bar{\mathbf{s}}\right) + \mathrm{Cov}(\bar{\mathbf{n}}).$$

A primeira parcela pode ser obtida pela propriedade da covariância sob uma transformação linear. Para uma matriz determinística $\mathbf{A}$ e um vetor aleatório $\mathbf{z}$, vale:

$$\mathrm{Cov}(\mathbf{A}\mathbf{z}) = \mathbf{A}\,\mathrm{Cov}(\mathbf{z})\mathbf{A}^H.$$

Aplicando essa propriedade à contribuição dos interferentes, obtém-se:

$$\begin{aligned} \mathrm{Cov}\left(\overline{\mathbf{H}}\,\bar{\mathbf{s}}\right) &= \overline{\mathbf{H}}\mathbb{E}[\bar{\mathbf{s}}\,\bar{\mathbf{s}}^H]\overline{\mathbf{H}}^H \\ &= \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H. \end{aligned}$$

Como a covariância do ruído térmico é $\sigma^2\mathbf{I}_M$, a matriz de covariância do ruído colorido resulta em:

$$\boxed{\mathbf{R}_n = \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H + \sigma^2\mathbf{I}_M}$$

$$\underbrace{\overline{\mathbf{H}}}_{M\times r}\;\underbrace{\beta\mathbf{I}_r}_{r\times r}\;\underbrace{\overline{\mathbf{H}}^H}_{r\times M} + \underbrace{\sigma^2\mathbf{I}_M}_{M\times M} = \underbrace{\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1M} \\ r_{21} & r_{22} & \cdots & r_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ r_{M1} & r_{M2} & \cdots & r_{MM} \end{bmatrix}}_{\mathbf{R}_n\ (M\times M)}$$

O termo $\sigma^2\mathbf{I}_M$ só contribui para a diagonal. Já o termo $\beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H$, da interferência, pode ter elementos não nulos fora da diagonal. Na matriz acima, esses são os elementos em destaque: cada um indica o quanto o ruído de duas antenas está correlacionado. Os elementos da diagonal são a potência do ruído em cada antena.

O equalizador LMMSE usa essa matriz para levar a interferência em conta. Se os símbolos-alvo são independentes, com covariância $E_s\mathbf{I}_K$, e independentes do ruído, a matriz de covariância do sinal recebido é:

$$\mathbf{R}_y = E_s\mathbf{H}\mathbf{H}^H + \mathbf{R}_n = E_s\mathbf{H}\mathbf{H}^H + \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H + \sigma^2\mathbf{I}_M.$$

3.2.1.4 Interferência sobre ruído térmico (IoTh)

A IoTh mede a intensidade da interferência em relação ao ruído térmico. Ela é definida como (ZHAO et al., 2025):

$$\text{IoTh} = 10\log_{10}\left(\frac{\beta + N_0}{\sigma^2}\right), \tag{3.2}$$

em que N_0 é a potência do ruído térmico, ou seja, $N_0 = \sigma^2$, assim como na SNR medida por E_s/N_0 em (ZHAO et al., 2021).

A potência β dos UEs não-alvo controla a intensidade da interferência. Quando a IoTh é baixa, $\mathbf{R}_n$ se aproxima de $\sigma^2\mathbf{I}_M$ e o ruído é praticamente branco. À medida que a IoTh cresce, a parcela $\beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H$ domina e a correlação entre as antenas se torna mais forte.

3.2.1.5 Estimação da matriz de covariância

A matriz $\mathbf{R}_n$ é definida por uma esperança e depende de $\overline{\mathbf{H}}$ e de β, que o receptor não conhece. Por isso, o receptor estima $\mathbf{R}_n$ a partir de amostras do ruído.

A estimativa amostral é dada por:

$$\widehat{\mathbf{R}}_n = \frac{1}{N}\sum_{i=1}^{N} \mathbf{n}^i (\mathbf{n}^i)^H,$$

em que N é o número de amostras e $\mathbf{n}^i \in \mathbb{C}^{M\times 1}$ é a i-ésima amostra de ruído colorido, observada nas M antenas da BS.

As amostras vêm dos símbolos piloto. Nesses símbolos, o receptor conhece o sinal dos UEs alvo e pode removê-lo do sinal recebido. O que sobra é a interferência mais o ruído térmico, ou seja, uma amostra de ruído colorido.

O produto externo de uma amostra por sua transposta conjugada, $\mathbf{n}^i(\mathbf{n}^i)^H$, produz uma matriz de dimensão $M \times M$. Cada elemento dessa matriz corresponde ao produto entre o ruído observado em duas antenas na mesma amostra:

$$\left[\mathbf{n}^i(\mathbf{n}^i)^H\right]_{ab} = n_a^i (n_b^i)^*.$$

$$\underbrace{\begin{bmatrix} n_1^i \\ n_2^i \\ \vdots \\ n_M^i \end{bmatrix}}_{M\times 1} \underbrace{\begin{bmatrix} (n_1^i)^* & (n_2^i)^* & \cdots & (n_M^i)^* \end{bmatrix}}_{1\times M} = \underbrace{\begin{bmatrix} n_1^i(n_1^i)^* & n_1^i(n_2^i)^* & \cdots & n_1^i(n_M^i)^* \\ n_2^i(n_1^i)^* & n_2^i(n_2^i)^* & \cdots & n_2^i(n_M^i)^* \\ \vdots & \vdots & \ddots & \vdots \\ n_M^i(n_1^i)^* & n_M^i(n_2^i)^* & \cdots & n_M^i(n_M^i)^* \end{bmatrix}}_{M\times M}$$

Quando $a = b$, o produto corresponde à potência observada na antena considerada. Esses são os elementos em destaque na diagonal da matriz acima. Quando $a \neq b$, o produto relaciona os sinais de ruído observados em duas antenas distintas.

A média desses produtos ao longo das N amostras estima cada elemento de $\mathbf{R}_n$. Uma única amostra não basta, e a estimativa melhora à medida que N cresce.

3.2.1.6 Availability of noise samples within the coherence time

All samples used in noise estimation must represent the same covariance matrix. Therefore, the sampling interval is limited by the variation in time and frequency of the

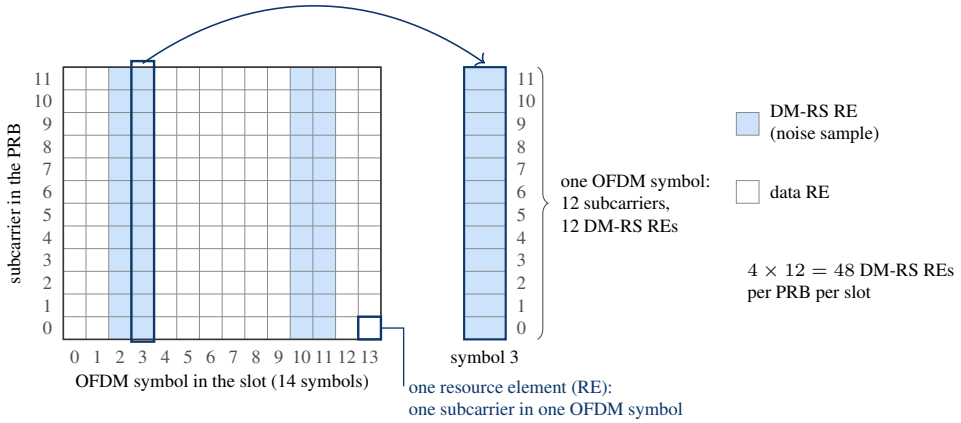
channels that determine the interference. The variation in time is defined by the coherence time. The variation in frequency is defined by the PRB. It is considered that the channels of the interfering users remain constant within the PRB (ZHAO et al., 2025). The estimate R_{cc}, therefore, is unique per PRB within each coherence time.

In 5G NR, the Demodulation Reference Signal (DM-RS) is the reference signal transmitted by the UE to allow the receiver to estimate channel conditions. The DM-RS is transmitted alongside the PUSCH (Section 3.1.1) within the same PRBs (3GPP, 2025, Sec. 6.4.1.1.3). Each UE transmits its own DM-RS within the same OFDM symbols, and the receiver must distinguish the DM-RS of each one. The double-symbol DM-RS occupies two consecutive OFDM symbols at each position within the slot and allows for the differentiation of up to 12 UEs (3GPP, 2025, Tab. 6.4.1.1.3-5). To achieve this, each UE employs different subcarriers or sequences of $+1$ and -1 values (3GPP, 2025, Tab. 6.4.1.1.3-2). For instance, when two UEs utilize the same subcarriers, one may transmit $(+1, +1)$ across two adjacent REs while the other transmits $(+1, -1)$; the sum and difference of the received REs then allow for the isolation of each UE.

Noise samples are obtained from the REs occupied by the DM-RS. Como o DM-RS só é transmitido junto com o PUSCH, só há amostras nos slots em que os UEs alvo transmitem o PUSCH (Seção 3.1.2). The number of samples therefore depends on how many DM-RS REs are present within the interval over which the covariance remains constant. With a maximum of two positions per slot, the double-symbol DM-RS occupies 4 OFDM symbols (3GPP, 2025, Tab. 6.4.1.1.3-4). Since each PRB contains 12 subcarriers (Section 3.1.2), a maximum of 4 $\times$ 12 = 48 DM-RS REs per slot are available within each PRB. Figure 3.1 illustrates these REs within a PRB. Consequently, the quantity of samples collected within an interval T_c depends on the number of OFDM symbols contained within that interval.

A sample represents the noise covariance for a given data symbol only if both fall within the same interval of duration T_c. As a result, the number of available samples may vary from one data symbol to another. For any given symbol, the number of samples corresponds to the maximum count of DM-RS REs found within an interval of duration T_c that encompasses that symbol. The number of samples available for all symbols in the slot is the smallest of these values. Figure 3.2 shows an example with the DM-RS located at symbols 2, 3, 10, and 11 of each slot. Under the TDLB100-400 condition, symbols 5 through 8 represent the worst-case scenario, with 24 samples. Therefore, the number of samples available for all symbols is 24. Table 3.2 presents this value for each

Figure 3.1: Resource elements of one PRB in one slot, with the DM-RS symbols.



Source: the author, with the DM-RS positions of 3GPP, 2025. Double-symbol DM-RS, mapping type A, symbols 2, 3, 10 and 11.

test condition.

Table 3.2: Number of DM-RS REs per PRB available to every data symbol of the slot, in each test condition.

Test condition	**Frequency range**	T_c **(ms)**	**OFDM symbols in** T_c	**DM-RS REs available**
TDLA30-5	FR1	25.0000	700	2400
TDLA30-10		12.5000	350	1200
TDLA30-75	FR2	1.6667	186	648
TDLA30-300		0.4167	46	168
TDLC300-100	FR1	1.2500	35	132
TDLB100-400		0.3125	8	24
HST350		0.0536	1	0
HST500		0.0375		

Source: the author, from 3GPP, 2020; 3GPP, 2025; TSE; VISWANATH, 2005. Double-symbol DM-RS in symbols 2, 3, 10 and 11, with every slot carrying PUSCH.

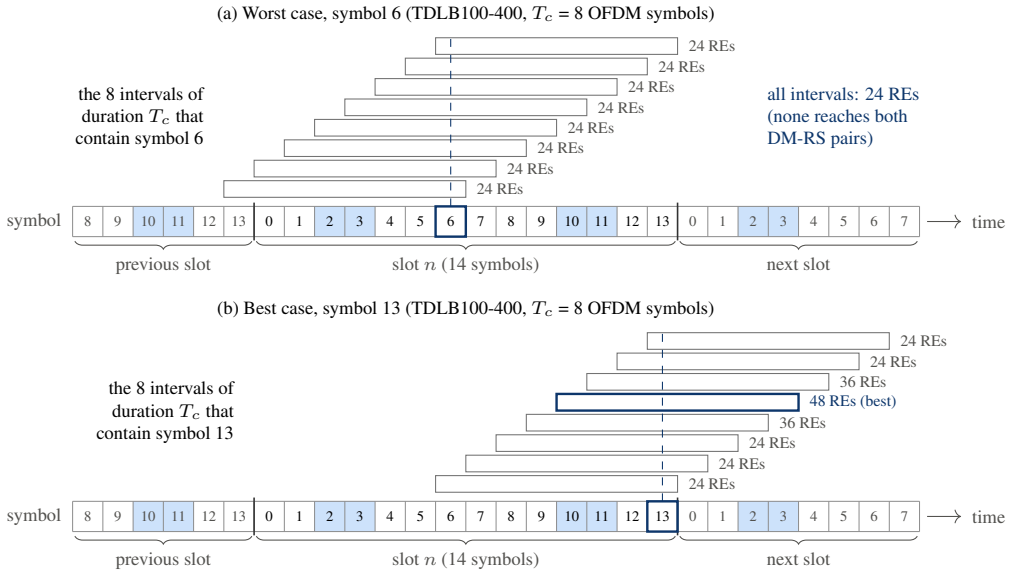
The best range of a data symbol may include DM-RS that arrive after that symbol. In Figure 3.2(b), the best range of symbol 13 goes up to symbol 3 of the next slot. To use these samples, the receiver needs to store symbol 13 for 4 more symbols.

3.2.1.7 Condicionamento da estimativa

O posto de uma matriz é o número máximo de linhas ou colunas linearmente independentes. Cada parcela $\mathbf{n}^i(\mathbf{n}^i)^H$ tem posto 1, porque todas as suas colunas são múltiplas de $\mathbf{n}^i$. Então, o posto da soma de N parcelas é no máximo N:

$$\text{rank}(\widehat{\mathbf{R}}_n) \leq \min(M, N)$$

Figure 3.2: Intervals of duration T_c that contain a data symbol, and their DM-RS REs per PRB, in the TDLB100-400 condition.



Source: the author, with the DM-RS positions of 3GPP, 2025. Double-symbol DM-RS, mapping type A, symbols 2, 3, 10 and 11; continuous uplink. In (a) and (b), each bar is one position of an interval of duration T_c that contains the marked symbol. Symbol 6 is the worst case of the slot, and symbol 13 is a best case.

Assim, quando $N < M$, $\hat{\mathbf{R}}_n$ é singular e não tem inversa.

O artigo base adota $N \gg K$ para obter uma estimativa precisa e usa $N = 192$ amostras por realização do canal (ZHAO et al., 2025). Um trabalho anterior dos mesmos autores cita 96 e 192 como valores típicos (ZHAO et al., 2021).

Uma forma de tornar a estimativa inversível é o encolhimento (*shrinkage*) de Ledoit e Wolf. Ele troca $\hat{\mathbf{R}}_n$ por uma média ponderada entre $\hat{\mathbf{R}}_n$ e um múltiplo da identidade (LEDOIT; WOLF, 2004):

$$\hat{\mathbf{R}}_{\mathrm{LW}} = (1-\alpha)\,\hat{\mathbf{R}}_n + \alpha\,\mu\,\mathbf{I}_M, \qquad \mu = \frac{\mathrm{tr}(\hat{\mathbf{R}}_n)}{M}. \tag{3.3}$$

O peso α, entre 0 e 1, é calculado a partir das próprias amostras, de forma a minimizar o erro quadrático esperado entre a estimativa e a covariância verdadeira (LEDOIT; WOLF, 2004). Como $\hat{\mathbf{R}}_n$ não tem autovalores negativos, todo autovalor de $\hat{\mathbf{R}}_{\mathrm{LW}}$ é pelo menos $\alpha\mu$. Quando $\alpha > 0$, a matriz resultante é inversível.

3.2.2 Processamento descentralizado

Na arquitetura descentralizada, as M antenas da ~~estação base~~ BS são particionadas em C clusters, cada um associado a uma DU (*Distributed Unit*), uma unidade de processamento local (ZHAO et al., 2025). O vetor de ruído pode, então, ser organizado em componentes locais:

$$\mathbf{n} = \begin{bmatrix} \mathbf{n}_1 \\ \mathbf{n}_2 \\ \vdots \\ \mathbf{n}_C \end{bmatrix}.$$

Essa partição induz uma estrutura em blocos na matriz de covariância $\mathbf{R}_n$. Os blocos diagonais $\mathbf{R}_{cc}$ descrevem a covariância do ruído observado pelas antenas do cluster c, enquanto os blocos fora da diagonal, $\mathbf{R}_{mn}$, representam as correlações entre antenas pertencentes a clusters distintos.

Cada DU dispõe localmente das amostras de ruído de suas próprias antenas. Assim, a DU c pode estimar seu bloco diagonal sem trocar dados com as demais unidades:

$$\hat{\mathbf{R}}_{cc} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{n}_c^i (\mathbf{n}_c^i)^H.$$

Os blocos fora da diagonal, por outro lado, dependem de amostras provenientes de duas ou mais DUs. Sua estimação direta exige a troca de informações entre as unidades, aumentando o tráfego de comunicação.

Essa limitação estabelece uma relação entre as propriedades estatísticas do ruído e a estratégia de equalização descentralizada.

3.2.3 Equalizador BCD-MMSE

O equalizador BCD-MMSE (*Block Coordinate Descent–Minimum Mean Square Error*) (ZHAO et al., 2025), por sua vez, emprega atualizações distribuídas para tratar o acoplamento entre os blocos do sistema, mantendo a necessidade de definir quais informações devem ser compartilhadas entre as unidades.

No BCD-MMSE considerado neste trabalho, a estimativa local $\hat{\mathbf{R}}_{cc}$ participa da inicialização do equalizador e da construção da matriz utilizada nas atualizações da DU.

Em particular, a matriz local a ser invertida é definida por:

$$\mathbf{G}_c = E_s \mathbf{H}_c \mathbf{H}_c^H + \hat{\mathbf{R}}_{cc},$$

em que $\mathbf{H}_c$ representa a parcela da matriz de canal associada ao cluster c. Dessa forma, a estimação e o condicionamento da covariância local afetam diretamente a viabilidade das operações matriciais necessárias à equalização.

A DU c usa $\hat{\mathbf{R}}_{cc}$ de duas formas. Na inicialização, usa a inversa $\hat{\mathbf{R}}_{cc}^{-1}$, pois o BCD-MMSE começa pela aproximação bloco-diagonal (ZHAO et al., 2025). Nas iterações, usa $\mathbf{G}_c$, que soma $\hat{\mathbf{R}}_{cc}$ ao termo do canal (ZHAO et al., 2025). Assim, $\mathbf{G}_c$ usa $\hat{\mathbf{R}}_{cc}$, e não a sua inversa. Além disso, $\mathbf{G}_c$ não depende da iteração (ZHAO et al., 2025). Ela é formada e invertida uma vez, e a inversa é usada em todas as iterações.

O número N de amostras também tem um custo de armazenamento e de comunicação. Cada DU guarda as N amostras de ruído das suas antenas (ZHAO et al., 2025). Trocar essas amostras entre as DUs exigiria transferir dados de tamanho $M \times N$ (ZHAO et al., 2025). No BCD-MMSE, a banda de comunicação em cada iteração também depende de N (ZHAO et al., 2025). Como a estimativa só muda a cada bloco de coerência, esse custo é pago uma vez a cada N_{coh} símbolos (ZHAO et al., 2025).

3.2.3.1 Complexidade

No artigo base, o pré-processamento de cada DU tem complexidade $\mathcal{O}(M_c^3 + NM_c^2)$, e cada iteração do BCD-MMSE tem complexidade $\mathcal{O}(NMK + MM_cK)$ no total (ZHAO et al., 2025). A Tabela 3.3 mostra o número de multiplicações complexas de cada parte do algoritmo em uma DU, por PRB e por T_c. Várias partes dependem de N, que muda com a condição de teste (Tabela 4.3). Neste trabalho, só a inversa de $\mathbf{G}_c$ é calculada em ponto fixo (Seção 4.1).

Table 3.3: Multiplicações complexas por PRB e por T_c em cada DU.

Parte	**Multiplicações complexas**
Estimar $\hat{\mathbf{R}}_{cc}$ e aplicar Ledoit-Wolf	$NM_c^2 + NM_c + M_c^2$
Inicialização, com a inversa de $\hat{\mathbf{R}}_{cc}$	$M_c^3 + M_c^2K + M_cK^2 + 2K^2M_c + NKM_c$
Formar $\mathbf{G}_c$	M_c^2K
Inverter $\mathbf{G}_c$	M_c^3
Cada iteração	$2NKM_c + 3K^2M_c + KM_c^2$
Inverter a matriz $K \times K$ (só na DU C)	K^3
Filtro, por símbolo de dados	$12KM_c$

3.3 Inversão de Matrizes

3.3.1 Inversao de Matrizes

3.3.1.1 Decomposicao de Cholesky

3.3.1.2 Representação em ponto fixo

Em hardware, os números costumam ser representados em ponto fixo. No Vitis HLS, o tipo `ap_fixed`$\langle W, I\rangle$ tem W bits no total, dos quais I são da parte inteira (KASTNER; MATAI; NEUENDORFFER, 2018, p. 73–74). A parte fracionária tem $F = W - I$ bits. Neste trabalho, o formato é escrito como $\langle W, I\rangle$.

Quando um valor não cabe na parte inteira, ocorre um estouro (*overflow*). Uma forma comum de tratá-lo é descartar os bits mais significativos, o que se chama *wrapping* (KASTNER; MATAI; NEUENDORFFER, 2018, p. 70). Em complemento de dois, isso pode até trocar o sinal do número (KASTNER; MATAI; NEUENDORFFER, 2018, p. 71). Quando um valor não cabe na parte fracionária, ele precisa ser arredondado (KASTNER; MATAI; NEUENDORFFER, 2018, p. 71). Descartar os bits extras arredonda em direção a $-\infty$. Outras formas arredondam em direção a $+\infty$, em direção a zero, para longe de zero, ou para o valor mais próximo, com o empate indo para o valor par (KASTNER; MATAI; NEUENDORFFER, 2018, p. 71).

O tipo `ap_fixed` tem 7 modos de quantização e 5 modos de estouro, o que dá 35 combinações. A Tabela 3.4 descreve cada um.

3.3.1.3 Latência, intervalo e vazão

Em síntese de alto nível, uma tarefa é uma chamada de função. A latência da tarefa é o tempo entre o início e o fim de uma tarefa. O intervalo da tarefa é o tempo entre o início de uma tarefa e o início da seguinte (KASTNER; MATAI; NEUENDORFFER, 2018, p. 20). Em um circuito com *pipeline*, uma nova tarefa pode começar antes de a anterior terminar. Assim, o intervalo pode ser menor que a latência (KASTNER; MATAI; NEUENDORFFER, 2018, p. 21). O intervalo entre dois dados que entram no *pipeline* é chamado de intervalo de iniciação (II, *Initiation Interval*) (KASTNER; MATAI; NEUENDORFFER, 2018, p. 21).

No começo, o *pipeline* está vazio, e só os primeiros estágios trabalham. O tempo até todos os estágios ficarem ocupados é o tempo de enchimento. No fim, o *pipeline* tam-

Table 3.4: Modos de quantização e de estouro do tipo `ap_fixed`.

Modo	Comportamento
Quantização (afeta os bits fracionários)	
`AP_TRN`	Descarta os bits menos significativos. Arredonda em direção a $-\infty$.
`AP_TRN_ZERO`	Arredonda em direção a zero. Para números positivos, é igual ao `AP_TRN`.
`AP_RND`	Arredonda para o valor mais próximo. No empate, arredonda em direção a $+\infty$.
`AP_RND_ZERO`	Arredonda para o valor mais próximo. No empate, arredonda em direção a zero.
`AP_RND_MIN_INF`	Arredonda para o valor mais próximo. No empate, arredonda em direção a $-\infty$.
`AP_RND_INF`	Arredonda para o valor mais próximo. No empate, afasta-se de zero.
`AP_RND_CONV`	Arredonda para o valor mais próximo. No empate, vai para o valor par.
Estouro (afeta os bits inteiros)	
`AP_SAT`	Satura no maior ou no menor valor representável.
`AP_SAT_ZERO`	Em caso de estouro, o resultado vira zero.
`AP_SAT_SYM`	Satura de forma simétrica: os limites positivo e negativo têm o mesmo módulo.
`AP_WRAP`	Descarta os bits mais significativos que não cabem (estouro em complemento de dois).
`AP_WRAP_SM`	Faz o mesmo descarte, mas em sinal e magnitude.

Fonte: TODO (UG1399).

bém precisa esvaziar. Esse custo fixo pesa mais quando há poucas iterações (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 114 e 146).

Em uma sequência de tarefas, a primeira termina depois da latência L, e cada uma das seguintes, um intervalo I depois da anterior. Assim, n tarefas terminam em

$$t_n = L + (n-1)\,I. \tag{3.4}$$

Muitas ferramentas chamam o intervalo de vazão (*throughput*). Um intervalo maior quer dizer menos tarefas por unidade de tempo, ou seja, uma vazão menor (KASTNER; MATAI; NEUENDORFFER, 2018, p. 22). Neste trabalho, a vazão é medida pelo intervalo entre inversões, e a latência é o tempo de uma inversão.

3.3.1.4 Pipeline de tarefas

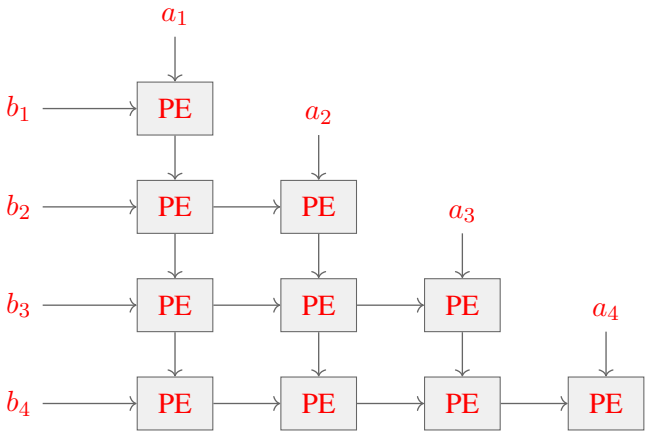
Uma tarefa pode ser dividida em etapas. No *pipeline* de tarefas, as etapas trabalham ao mesmo tempo, cada uma em um dado diferente. Assim, a tarefa seguinte começa quando a anterior termina a primeira etapa. Em HLS, isso é feito com a diretiva `dataflow`, e cada etapa vira um processo separado. O intervalo do conjunto nunca é menor que o intervalo da etapa mais lenta (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 113–115).

3.3.1.5 Arranjo sistólico

Um arranjo sistólico é um conjunto de elementos de processamento (PEs, *processing elements*) iguais, em que cada PE só troca dados com os vizinhos. Esse tipo de projeto pode surgir quando um laço interno é desenrolado: cada cópia do corpo do laço vira um PE (KASTNER; MATAI; NEUENDORFFER, 2018, p. 180).

A Figura 3.3 mostra um arranjo sistólico em grade que calcula a matriz de Gram de $\mathbf{H}$. Como essa matriz é hermitiana, só o triângulo inferior é calculado. A cada ciclo, o arranjo lê uma linha de $\mathbf{H}$. Os dados entram por cima e pela esquerda e passam para o PE de baixo e para o da direita. Cada PE multiplica as duas entradas e acumula o produto. Depois de ler todas as linhas, cada PE guarda um elemento da matriz (WANG et al., 2021, p. 2979).

Figure 3.3: Lower triangular systolic array that computes a Gram matrix.



Source: adapted from WANG et al., 2021, p. 2979. a_j and b_i: elements of one row of $\mathbf{H}$, one row per cycle. Each PE multiplies a_j by b_i, adds the product to its accumulator, and passes a_j down and b_i to the right.

4 METHODOLOGY

This chapter describes the tools used and the tests performed for this work. The uplink transmission of a massive MIMO system with decentralized processing (Section 3.1.1) was simulated to analyze the effect of the approximation used in the inversion module of the BCD-MMSE algorithm. Based on this analysis, the fixed-point formats for the inversion module, implemented on an FPGA, are selected. The chapter begins by presenting the simulation workflow (Section 4.1) and the hardware tools (Section 4.2). It then covers test configurations and conditions (Section 4.3), timing constraints for the inversion process (Section 4.4), the selection of the numerical format (Section 4.5), the hardware architectures, the matrices used to test them and the power estimation (Section 4.6), and the organization of the results (Section **??**).

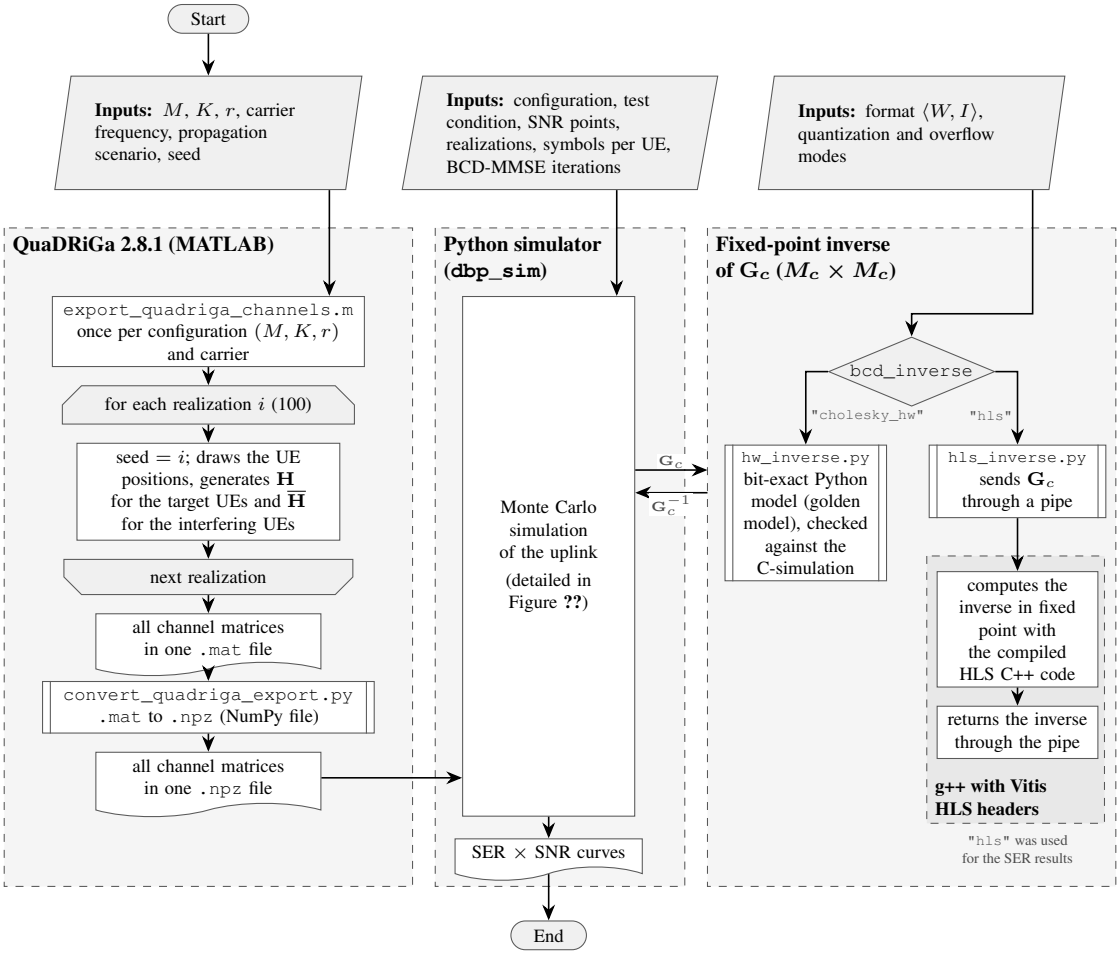
4.1 Simulation workflow

The simulation workflow 4.1 employs three tools: the QuaDRiGa channel generator, a Python simulator developed in this work, and C++ code developed in Vitis HLS, which computes the inverse of $\mathbf{G}_c$ using fixed-point arithmetic. QuaDRiGa generates the channel matrices $\mathbf{H}$ for the target UEs and $\overline{\mathbf{H}}$ for the interfering UEs.

The Python simulator handles the transmission process. It generates symbols for the target UEs and creates colored noise, with a covariance derived from $\overline{\mathbf{H}}$ (Section 3.2.1), to construct the received signal using $\mathbf{H}$. Subsequently, it computes the equalizers, performs symbol detection, and measures the SER at various SNR levels. The BCD-MMSE algorithm is simulated in two ways: using the inverse of $\mathbf{G}_c$ in 64-bit floating-point format (which serves as a reference) and using the inverse in fixed-point format. In the latter case, whenever the BCD-MMSE algorithm requires $\mathbf{G}_c^{-1}$, the Python simulator invokes the C++ code.

The Python simulator models the same uplink as in (ZHAO et al., 2025). According to the authors, that paper was the first work on decentralized processing to consider colored noise (ZHAO et al., 2025). Therefore, the simulation parameters follow that work, except where the text indicates otherwise.

Figure 4.1: Simulation flow: QuaDRiGa channels, Python simulator and fixed-point inverse.



Source: the author.

4.1.1 Channel generation

The channels are generated using QuaDRiGa 2.8.1 in MATLAB. They depend on the number of antennas M, the number of target UEs K, the number of interfering UEs r, and the carrier frequency. Consequently, they are generated once for each combination of these values. Other parameters, such as C, the modulation scheme, and the IoTh, only affect the processing steps within the Python-based simulator. The simulation employs 100 channel realizations. Each realization consists of a random selection of UE positions, along with the corresponding channels $\mathbf{H}$ and $\overline{\mathbf{H}}$ generated by QuaDRiGa for those specific positions. Every realization uses its own unique random seed.

MATLAB saves the channels for all 100 realizations into a `.mat` file. A Python script then converts this file into an `.npz` file, NumPy's format for storing multiple arrays within a single compressed file. The Python-based simulator reads the matrices $\mathbf{H}$ and $\overline{\mathbf{H}}$ from this file.

4.1.2 Python simulator

The Python simulator runs two nested loops (Figure 4.2). The outer loop iterates through the 100 realizations, while the inner loop iterates through the SNR points. The SNR points range from -10 to 10 dB, with a step of 2 dB. For each realization, the simulator takes $\mathbf{H}$ and $\overline{\mathbf{H}}$ corresponding to that realization and uses these channels across all SNR points. Each point on the SER curves represents the average over the 100 realizations. This is a Monte Carlo simulation, as in (ZHUANG et al., 2025): the SER (Section 3.1.5.6) is estimated by the fraction of erroneous symbols across many random draws of the channel, noise, and symbols.

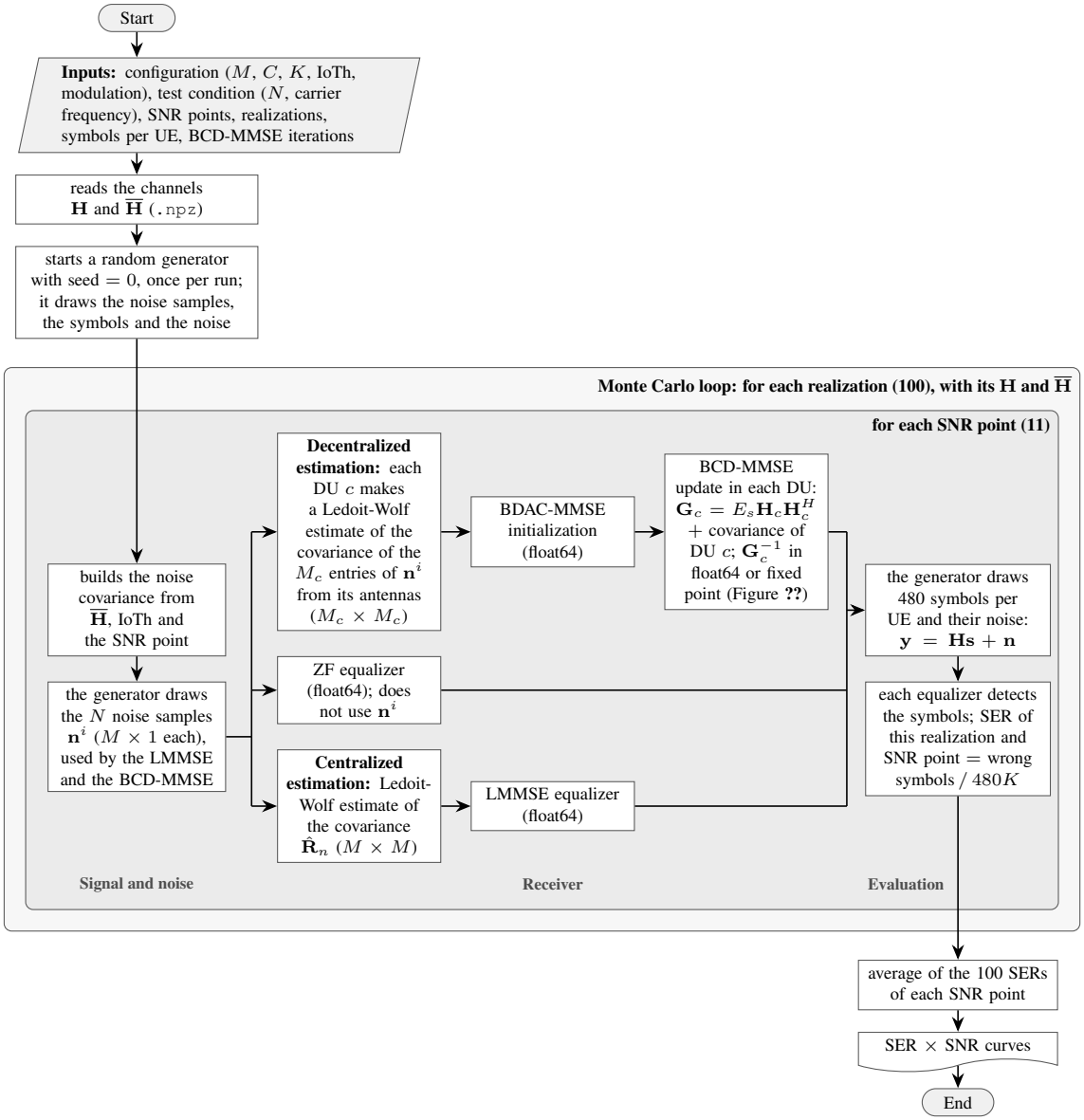
At each SNR point, the simulator constructs the noise covariance matrix based on $\overline{\mathbf{H}}$, the IoTh (interference-over-thermal), and the SNR. It then generates N noise samples, $\mathbf{n}^i$ (Section 3.2.1.5). It also generates 480 data symbols for each UE, along with the noise associated with these symbols. Based on this, the equalizers are calculated. The channel remains constant within the coherence block (Section 3.1.4.3); therefore, each equalizer is computed once and applied to all 480 symbols. The BCD-MMSE is compared against two centralized equalizers, the LMMSE and the ZF, which serve as benchmarks. The ZF does not utilize the noise samples $\mathbf{n}^i$. In contrast, both the LMMSE and the BCD-MMSE employ these samples to estimate the noise covariance (Section 3.2.1.7). This estimate is always subjected to Ledoit-Wolf shrinkage (Equation 3.3) before being used, even in cases where the estimate is already invertible.

In the LMMSE, this adjustment is applied to the estimate $\hat{\mathbf{R}}$ corresponding to all M antennas. In the BCD-MMSE, each DU performs the adjustment on its own block, $\hat{\mathbf{R}}_{cc}$, using only the samples from its M_c antennas. Consequently, no DU requires access to the samples from any other DU. The adjusted block forms the matrix $\mathbf{G}_c$ (Section 3.2.3). In the reference version of the BCD-MMSE, this matrix is inverted using 64-bit floating-point arithmetic; alternatively, it can be inverted using fixed-point arithmetic (Section 4.1.3).

4.1.3 Fixed-point inverse

The inverse of $\mathbf{G}_c$ in fixed-point arithmetic is derived from the C++ code of the inversion module (Section 4.2.2). This code is compiled using `g++` along with the `ap_fixed` header files from Vitis HLS 2026.1, with a separate compilation performed

Figure 4.2: Detailed flow of the Python simulator: Monte Carlo loop, equalizers and SER.



Source: the author.

for each combination of format $\langle W, I \rangle$, quantization mode, overflow mode (Table 3.4), and size M_c. The program remains active throughout the simulation. Other calculations within the simulator, including the inverses used for BDAC initialization and LMMSE, utilize 64-bit floating-point arithmetic.

To verify this program, the same $\mathbf{G}_c$ matrices used in the simulator were inverted both by this program and by the Vitis HLS C-simulation. This process involved 30 matrices for each of the 35 mode combinations, with M_c set to 16 and 32. In every instance, the resulting inverses were bit-identical. Consequently, the Symbol Error Rate (SER) is measured using the exact same inverse as that calculated by the hardware. The simulator also

includes a Python-based model of the inversion, which has been verified bit-for-bit against the C++ code; however, the reported SER results are based on the C++ implementation.

4.2 Hardware setup

4.2.1 Tools and target device

The inversion module was described in C++ and synthesized in Vitis HLS 2026.1. Vitis HLS generates the module's RTL. Then, the RTL was synthesized, placed, and routed in Vivado 2023.2. The target device is the AMD Alveo U50 board, with the XCU50-FSVH2104-2-E FPGA. This FPGA has 5,952 DSP blocks and 871,680 LUTs. All circuits use a clock frequency of 100 MHz, or a period of 10 ns.

In Vivado, the inversion module is synthesized and implemented alone, as the top-level design, without any other surrounding board logic. The flow performs synthesis, optimization, placement, physical optimization, and routing. The only timing constraint is the clock period, and the paths entering and exiting the module's ports are not constrained. Thus, the resource, timing, and power figures for this work are only for the module and come from the routed design in Vivado 2023.2.

Before the Vivado workflow, each version went through the Vitis HLS C synthesis. A version only went to Vivado if the HLS estimate fit within the FPGA: a maximum of 5,952 DSP blocks and a maximum of 90% of the LUTs.

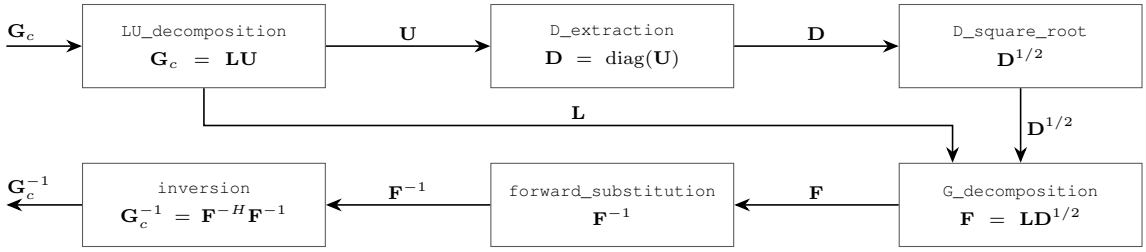
4.2.2 Inversion module

The module receives the $M_c \times M_c$ matrix $\mathbf{G}_c$ and returns its inverse. It calculates the inverse using Cholesky decomposition (Section 3.3). Two sizes for $\mathbf{G}_c$, corresponding to the configurations described in Section 4.3.1, have been implemented: 16×16 and 32×32. The input, output, and internal values use the `ap_fixed` data type, following the formats selected in Section 4.5.

The module consists of six stages, each implemented as a C++ function (Figure 4.3). These stages are executed sequentially, with each stage passing its output to the next.

Additions, multiplications, and divisions utilize `ap_fixed` type operators. Com-

Figure 4.3: Steps of the inversion module. Each block is one C++ function of the HLS code.



Source: the author.

plex multiplication and division operations are explicitly implemented in the code, with a specific type defined for each intermediate value; thus, the bit-width of every intermediate value is explicitly set within the code. The addition of two N-bit numbers requires $N + 1$ bits, while their product requires $2N$ bits (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 72–73). When using `ap_fixed` types, the result of an operation inherently possesses a sufficient number of bits; overflow only occurs when this result is assigned to a type with a smaller bit-width (KASTNER; MATAI; NEUENDORFFER, 2018, p. 74). Consequently, intermediate values employ types with greater bit-widths than the $\langle W, I \rangle$ format (Table 4.1), with the result reverting to the specified format only at the output of each stage. Square root calculations are performed using the `hls::sqrt` function from Vitis HLS, operating directly within the defined format.

Table 4.1: Types of the values in the inversion module, for the format $\langle W, I \rangle$ with $F = W - I$ fractional bits.

Value	**Integer bits**	**Fractional bits**
Input, output and output of each step	I	F
Square root (`hls::sqrt`)	I	F
Accumulated sums, numerator and divisor of the complex division, quotient	$2I$	$2F$
Product of two real numbers	$4I$	$4F$
Real or imaginary part of a complex product (sum of two products)	$4I + 1$	$4F$

Source: the author. The intermediate values use the same quantization and overflow modes as the format; the square root uses AP_TRN and AP_WRAP.

4.3 Configurations and test conditions

4.3.1 Base station configurations

The uplink of the massive MIMO system was investigated using two types of parameters. Hard constraints refer to the physical characteristics of the base station: $M = 128$ or 256 antennas, $C = 8$ or 16 clusters, and the size $M_c = M/C$ of the inversion module. Soft constraints depend on the UEs being served and the level of interference: $K = 8$ or 12 UEs, QPSK or 16-QAM modulation, and an IoTh ratio of 10 dB as a baseline case, or 20 dB representing an extreme, high-interference scenario. These configurations were adapted from (ZHAO et al., 2025). Table 4.2 presents these configurations.

Each configuration is identified by a label consisting of a number and a letter. The number denotes the hard constraints: 1 for $M = 128$ and $C = 8$; 2 for $M = 256$ and $C = 8$; and 3 for $M = 256$ and $C = 16$. The letter indicates the soft constraints and remains consistent across all three groups.

4.3.2 Test conditions

The simulated test conditions of Table 3.2 define the carrier frequency and the number of noise samples in the Python simulator, as shown in Table 4.3. The simulation does not include a time axis, and the channel does not account for the Doppler effect, so only these two variables change between test conditions. A frequency of 3.5 GHz was used for FR1 conditions, within the n78 band (3300 to 3800 MHz, TDD), and 28 GHz was used for FR2 conditions, within the n257 band (26500 to 29500 MHz, TDD) (3GPP, 2020, Tabs. 5.2-1 and 5.2-2). In each realization, the channels at both frequencies use the same UE positions.

This work employs a double-symbol DM-RS, located at symbols 2, 3, 10, and 11 of each slot (3GPP, 2025, Tab. 6.4.1.1.3-4), as it enables the differentiation of DM-RS signals among $K = 8$ or 12 UEs (Section 3.2.1.6). The value of N for each condition is taken from Table 3.2, with an upper limit of 192, the value used in (ZHAO et al., 2025). This N assumes that the receiver waits for samples arriving after the data symbol (Section 4.4.1). The HST350 and HST500 conditions were not simulated, as they correspond to $N = 0$.

TDLA30-5 and TDLA30-10 share the same carrier frequency and the same N

Table 4.2: Simulated configurations.

Configuration	**Hard constraints**			**Soft constraints**		
	M	C	M_c	**IoTh (dB)**	**Modulation**	K
–	M	C	–	iot_db	modulation	K
1A	128	8	16	10	16-QAM	8
1B						12
1C					QPSK	8
1D						12
1E				20	16-QAM	8
1F						12
1G					QPSK	8
1H						12
2A	256	8	32	10	16-QAM	8
2B						12
2C					QPSK	8
2D						12
2E				20	16-QAM	8
2F						12
2G					QPSK	8
2H						12
3A	256	16	16	10	16-QAM	8
3B						12
3C					QPSK	8
3D						12
3E				20	16-QAM	8
3F						12
3G					QPSK	8
3H						12

Source: 1A, 1B, 1C, 1E, 2A and 3A adapted from ZHAO et al., 2025; the others, the author. $r = 8$ in all configurations. Second header row: field names of `Panel` in the code.

Table 4.3: Carrier frequency and number of noise samples in each test condition.

Test condition	**Frequency range**	**Carrier frequency (GHz)**	**Noise samples N**	**UE speed (km/h)**
M1_Driver.test_conditions	–	M1_Driver.test_conditions	M5_Noise.n_noise_samples	–
TDLA30-5	FR1	3.5	192	1.5
TDLA30-10	FR1	3.5	192	3.1
TDLA30-75	FR2	28	192	2.9
TDLA30-300	FR2	28	168	11.6
TDLC300-100	FR1	3.5	132	30.8
TDLB100-400	FR1	3.5	24	123.3

Source: 3GPP, 2020 and the author. Speed from $v = f_D c / f_c$ TSE; VISWANATH, 2005.

value. Consequently, they exhibit identical SER curves, and the simulations cover 5 distinct test conditions. The only difference between them lies in the inversion cycle limit (Section 4.4.2).

The test conditions are grouped into two mobility profiles based on UE speed (Table 4.3). The low-mobility profile includes TDLA30-5, TDLA30-10, and TDLA30-75, with speeds up to 3.1 km/h. The high-mobility profile comprises TDLA30-300, TDLC300-100, and TDLB100-400, with speeds ranging from 11.6 to 123.3 km/h. The ITU-R uses 3 km/h for indoor and pedestrian users, and 30 and 120 km/h for users in vehicles (ITU-R, 2017, Sec. 8.2 and Tab. 5). In each profile, the fastest condition is the most demanding: it has the smallest T_c and the smallest N. This condition is TDLA30-75 in the low-mobility profile and TDLB100-400 in the high-mobility profile.

4.4 Inversion cycle limit

This section calculates, based on the coherence time, the maximum number of cycles between two inversions under each test condition.

4.4.1 Sample collection

The receiver first collects the N noise samples for each PRB. Some of these samples arrive only after the data symbol (Section 3.2.1.6). Therefore, this work assumes that the receiver buffers the data symbols until the N samples have been received. Subsequently, it processes the samples and equalizes the buffered data. A new set of samples arrives every interval T_c, and the processing must keep pace with this rate.

4.4.2 Throughput

The estimate $\hat{\mathbf{R}}_{cc}$ is unique per PRB within each interval T_c (Section 3.2.1.6). The matrix $\mathbf{G}_c$ is inverted once and serves all iterations of the BCD-MMSE algorithm (Section 3.2.3). Thus, each DU performs one inversion of $\mathbf{G}_c$ per PRB every T_c. With N_{PRB} PRBs, this results in N_{PRB} inversions per DU every T_c. This work uses $N_{\text{PRB}} = 275$, which is the worst-case scenario for 5G NR (Section 3.1.2).

To track data collection, the module needs to complete the N_{PRB} inversions within

T_c. Let T_{inv} be the interval between the start of two inversions (Section 3.3.1.3). Then:

$$T_{\text{inv}} \leq \frac{T_c}{N_{\text{PRB}}}. \tag{4.1}$$

With a clock speed of 100 MHz (Section 4.2.1), this limit is given in cycles, rounded down. Table 4.4 shows the limit for each test condition.

Table 4.4: Maximum interval between inversions in each test condition.

Test condition	T_c **(ms)**	T_c/N_{PRB} **(µs)**	**Limit at 100 MHz (cycles)**
TDLA30-5	25.0000	90.9091	9090
TDLA30-10	12.5000	45.4545	4545
TDLA30-75	1.6667	6.0606	606
TDLA30-300	0.4167	1.5152	151
TDLC300-100	1.2500	4.5455	454
TDLB100-400	0.3125	1.1364	113

Source: the author. T_c from Table 3.2; $N_{\text{PRB}} = 275$.

The hardware architectures in this work aim to meet this throughput limit under each test condition. The following are outside the scope: the delay between the arrival of a data symbol and its equalization; the throughput of the links between the DUs; and the other matrix inversions required by the BCD-MMSE algorithm, which the simulator performs using floating-point arithmetic (Section 4.1.3). The delay encompasses both the time spent waiting for samples and the latency of each inversion (i.e., the time elapsed from the start to the completion of an inversion).

4.5 Selection of the fixed-point format

4.5.1 SNR increase

To select the $\langle W, I \rangle$ format (Section 3.3.1.2), the BCD-MMSE algorithm was simulated using both fixed-point and 64-bit floating-point matrix inversion. Only the inversion of $\mathbf{G}_c$ differs between the two simulations; however, the SNR and SER are measured for the entire uplink. Each format was evaluated based on the SNR increase, that is, the additional SNR required by the uplink to achieve an $\text{SER} = 10^{-2}$ compared to the floating-point case. This increase corresponds to the horizontal distance between the two SER curves, which is the standard method for comparing two schemes (TSE; VISWANATH, 2005, p. 55).

As curvas de SER foram simuladas de -10 a 10 dB, com passo de 2 dB. Com QPSK, a grade começa em -20 dB, porque algumas curvas já atingem $\mathrm{SER} = 10^{-2}$ antes de -10 dB: quase todas as do LMMSE no grupo 1 e todas as do LMMSE e do BCD-MMSE nos grupos 2 e 3.

Two thresholds were employed. A 0.1 dB threshold represents a near-ideal format, where the SER curve is virtually identical to that of the floating-point implementation. A 3 dB threshold serves as the maximum acceptable increase. An increase of 3 dB is equivalent to doubling the power (TSE; VISWANATH, 2005, p. 73). Consequently, at this threshold, each UE would need to transmit with twice the energy per symbol to compensate for the error introduced by the matrix inversion (Section 3.1.3).

4.5.2 Initial sweep

The first stage involved varying the integer and fractional parts separately, while keeping the other part fixed (Table 4.5). Each size was tested across 35 combinations of quantization and overflow modes (Table 3.4), using the BCD-MMSE algorithm with both 1 and 4 iterations.

Table 4.5: Sizes tested in the initial sweep.

Sweep	**Integer bits** (I)	**Fractional bits** (F)
Integer part	6, 8, 10, 12, 14, 16, 20, 24, 32	32
Fractional part	14	4, 6, 8, 10, 12, 16, 20, 24, 32

Source: the author.

4.5.3 Configurations of the initial sweep

The initial sweep utilized the 16 configurations for which the floating-point SER curve reaches 10^{-2} within the -10 to 10 dB range. Running all of these across the 5 test conditions would have resulted in 80 cases, each involving 17 sizes and 35 mode combinations; therefore, only 5 configurations were selected.

The selection process relied on the $\mathbf{G}_c$ matrices at two SNR points surrounding the operating point (defined as $\mathrm{SER} = 10^{-2}$ in floating-point arithmetic). From each matrix, the largest absolute value between the real and imaginary parts of the elements was taken, since the circuit stores the two parts separately. This value is referred to here as the

"maximum |part|". In each case, the 99th percentile of these values was used to ensure that the overall ranking would not be disproportionately influenced by a few outlier matrices. Larger elements necessitate a greater number of integer bits. Configurations 1G, 1H, 2E, and 2F contain the largest elements and are the most demanding, while configuration 1A contains the smallest elements and was used as a baseline (Table 4.6).

These configurations also span the extremes of precision requirements. With a limited number of bits, the rounding error in matrix inversion increases with the condition number (OSINSKY et al., 2023, p. 1603). When the smallest eigenvalue is very small, the matrix approaches a singular state, making the result more dependent on the fractional part (Section 3.3.1.2). Configurations 2E and 2F exhibit the highest condition numbers (at the 99th percentile), whereas configuration 1A features the smallest eigenvalues (at the 1st percentile).

Table 4.6: $\mathbf{G}_c$ at the operating point of the initial-sweep configurations.

	SNR (dB)		**Largest |part|**		**Condition number**		**Smallest eigenvalue**	
Configuration	**192; 3.5**	**168; 28**	**192; 3.5**	**168; 28**	**192; 3.5**	**168; 28**	**192; 3.5**	**168; 28**
1A	-2.6	-2.4	63	66	57	58	1.9	1.9
1G	-9.3	-9.1	2266	2383	187	169	22.4	24.1
1H	-8.7	-8.6	2470	2385	195	172	22.1	24.7
2E	-5.8	-5.6	962	999	286	254	8.8	9.7
2F	-5.6	-5.5	983	987	282	285	8.8	9.5

Source: the author. Subcolumns: N; carrier in GHz.

4.5.4 Synthesis and selection of formats for validation

The sizes and modes of the initial sweep were also synthesized in Vitis HLS to measure resource occupancy (LUT, DSP, FF, and BRAM). The occupancy of each circuit is that of the most used resource in the FPGA. In each configuration and test condition, the integer and fractional parts with the lowest occupancy were chosen, with an SNR increase of up to 3 dB and of up to 0.1 dB, in 1 and in 4 iterations. Combining the two parts resulted in 10 formats (Table 4.7).

Table 4.7: Configurations in which each format was chosen in the initial sweep.

	1 iteration		4 iterations	
Format	**Up to 3 dB**	**Up to 0.1 dB**	**Up to 3 dB**	**Up to 0.1 dB**
$\langle 14, 6\rangle$	1A	–	–	–
$\langle 16, 6\rangle$	–	–	1A	–
$\langle 18, 8\rangle$	–	1A	–	–
$\langle 20, 8\rangle$	–	–	–	1A
$\langle 22, 12\rangle$	1H, 2E, 2F	–	–	–
$\langle 24, 12\rangle$	1G	2E, 2F	1G, 1H, 2E, 2F	–
$\langle 26, 12\rangle$	–	1G[a], 1H[a]	–	2E, 2F
$\langle 28, 12\rangle$	–	–	–	1G[a], 1H[a]
$\langle 28, 14\rangle$	–	1G[b], 1H[b]	–	–
$\langle 30, 14\rangle$	–	–	–	1G[b], 1H[b]

Source: the author. Columns: SNR increase. No mark: all 5 test conditions. [a]Only TDLB100-400. [b]The other 4 conditions.

4.5.5 Validation

The initial sweep fixed one part of the word while varying the other, and used only 5 configurations. This can change the most used resource and the best mode. Therefore, the 10 formats were simulated again, with the 35 mode combinations, in the 24 configurations and 5 test conditions.

In this step, BCD-MMSE used only 1 iteration. The 4 iterations of the initial sweep had already helped to choose the formats of the most demanding cases: $\langle 16, 6\rangle$, $\langle 20, 8\rangle$, $\langle 28, 12\rangle$ and $\langle 30, 14\rangle$ only appeared with 4 iterations (Table 4.7). With 1 iteration, BCD-MMSE already has a good result in SER. Furthermore, the inverse of $\mathbf{G}_c$ is calculated once and used in all iterations, so the number of inversions does not change (Section 3.2.3).

The circuit has only one mode. Therefore, each format received a fixed mode in each group of configurations (Section 4.3). All configurations in a group use the same circuit. The mode was chosen according to the following criteria, in this order:

1. the mode that leaves more pairs (configuration, test condition) with an increase in SNR of up to 3 dB. A pair that does not reach $\mathrm{SER} = 10^{-2}$ counts as out of bounds;
2. the lowest resource occupancy, by the most used resource in the synthesis;
3. the quantization `AP_RND_ZERO`, which never uses more resources than the other tied modes;
4. the smallest increase in SNR in the worst pair;

5. the order of the modes in the list.

Nessa etapa, a simulação em ponto fixo usou só as SNRs em volta do ponto de operação, com passo de 2 dB. Elas começam no ponto da grade logo abaixo da SNR em que o BCD-MMSE em ponto flutuante atinge $\mathrm{SER} = 10^{-2}$ e vão de 4 a 7 dB além dela. Depois de escolhido o modo fixo, cada formato foi simulado de novo, só com esse modo, na grade inteira de SNR (Seção 4.5.1). Essas simulações usam a mesma semente da simulação em ponto flutuante. A escolha dos três formatos de cada grupo e os resultados da Seção 5.2 vêm delas.

Depois, com o modo fixo, foram escolhidos dois formatos em cada grupo: o menor em que todas as configurações e condições do grupo atingem $\mathrm{SER} = 10^{-2}$ com aumento de SNR de até 3 dB, e o menor com aumento de até 0,1 dB. O aumento é arredondado em duas casas decimais. O menor formato que só atinge $\mathrm{SER} = 10^{-2}$, com qualquer aumento, não foi usado como critério, porque esse aumento chega a mais de 10 dB (Seção 5.2). Esses formatos foram implementados no Vivado e estão na Seção 5.2.

4.6 Hardware architectures, test matrices and power

The chosen formats were synthesized using Vitis HLS. In all versions, the automatic loop pipelining feature was disabled (AMD, 2026, p. 42). The first version, without loop optimizations, was implemented for both 16×16 and 32×32 matrix sizes. In this version, the iterations of each loop execute sequentially, and the $N_{\mathrm{PRB}} = 275$ matrices within a block are inverted one by one. Consequently, the interval between inversions exceeds the limits defined for all test conditions (Table 4.4), and the module does not track the arrival of the matrices at each T_c.

To meet the cycle limits for each test condition, a specific architecture was developed for every case, only in 16×16, with Vitis HLS directives that reduce the interval between inversions. All versions were implemented in Vivado, tested using $\mathbf{G}_c$ matrices obtained from the simulator (Section 4.6.2), and evaluated for power consumption and energy per inversion (Section 4.6.3). Each version produces the same inverse as the non-optimized implementation, a result verified bit-for-bit against the C-language code. As arquiteturas otimizadas não foram implementadas em 32×32. Uma síntese C de teste das seis, em 32×32 e no formato $\langle 24, 12 \rangle$, estimou intervalos acima do limite de ciclos de todas as condições de teste. Nas sistólicas, o intervalo cresce 4 vezes, com o quadrado do

tamanho da matriz. A da TDLB100-400 também não caberia no FPGA. Para atender os limites em 32×32, seria preciso outra arquitetura, com mais área, e isso não foi testado.

4.6.1 Hardware architectures

All optimized architectures employ pipelining (Section 3.3.1.3). Every T_c, the DU inverts 275 matrices $\mathbf{G}_c$, one for each PRB (Section 4.4). For the 9,090 and 4,545-cycle limits, the module inverts one matrix at a time. For tighter limits, it inverts the matrices in batches of n. In all cases, the module must receive the 275 matrices for a given block within the duration T_c.

The matrices within a batch are independent of one another. Consequently, c of these matrices are computed in the same cycle using loop unrolling, which creates a copy of the loop body for each iteration. The loop that iterates over the c matrices is nested within a pipelined loop, and the HLS tool performs the unrolling automatically (AMD, 2026, pp. 41, 372, and 428). When c is less than n, groups of c matrices are processed in alternating cycles. Each copy has its own dedicated memory bank, though it may share operators with the others.

The values for n and c were determined through trial and error and are presented in Table 4.8.

Table 4.8: Batch of the inversion module in each test condition.

Test condition	**Limit per matrix (cycles)**	**Matrices per batch (n)**	**Copies (c)**	**Batches per block**
Without optimizations	–	1	1	275
TDLA30-5	9090	1	1	275
TDLA30-10	4545	1	1	275
TDLA30-75	606	12	6	23
TDLC300-100	454	16	8	18
TDLA30-300	151	32	2	9
TDLB100-400	113	32	4	9

Source: the author. Limit per matrix from Table 4.4.

4.6.1.1 Loop pipelining

The module consists of six sequential stages (Figure 4.3): LU decomposition, extraction of the diagonal $\mathbf{D}$, calculation of the square root $\mathbf{D}^{1/2}$, computation of $\mathbf{F}$, forward substitution, and inversion. For the 9,090-cycle limit, the internal loops of

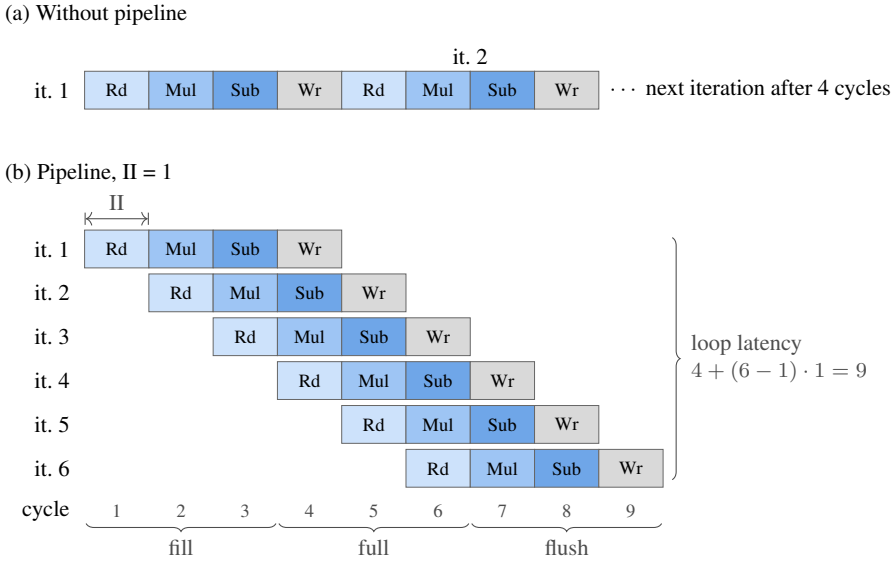
these stages are annotated with the `#pragma HLS pipeline II=1` directive. The `D_extraction` stage is the only one without this directive, as it merely copies the diagonal from **U** to **D** and requires significantly fewer cycles than the other stages.

This directive enables loop pipelining, specifying that a new iteration should begin every cycle, that is, an initiation interval (II) of 1 cycle (Section 3.3.1.3). If data dependencies or resource constraints prevent this, the HLS tool will automatically use a larger II (KASTNER; MATAI; NEUENDORFFER, 2018, p. 45).

Figure 4.4 illustrates a loop from the LU decomposition process, both without and with pipelining. The individual stages and loops for a given matrix are executed sequentially. Consequently, the next inversion process cannot begin until the current one has finished; in this case, the interval between inversions is equal to the latency (KASTNER; MATAI; NEUENDORFFER, 2018, p. 114).

Figure 4.4: Loop pipelining in the LU decomposition. The code is simplified.

```
for (line = column + 1; line < n; line++)
  for (k = column; k < n; k++) {
#pragma HLS pipeline II=1
    t = L[line][column] * U[column][k];
    U[line][k] = U[line][k] - t;
  }
```



Source: the author. Rd: read, Mul: multiply, Sub: subtract, Wr: write. Iteration latency of 4 cycles, for illustration.

4.6.1.2 Dataflow between stages

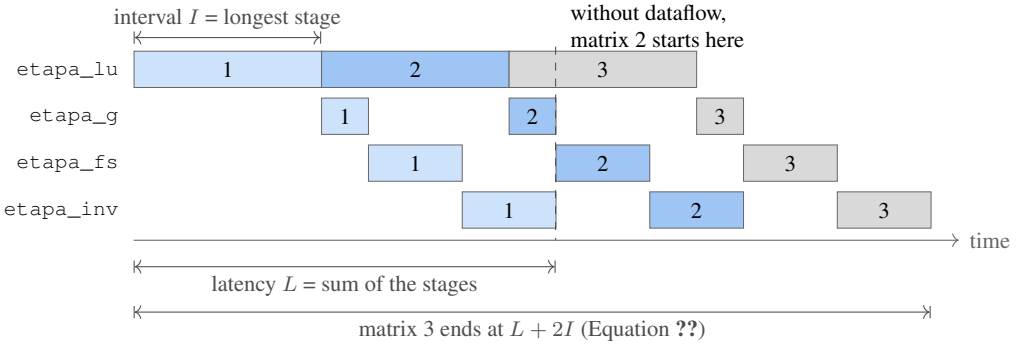
The architecture of the 4,545-cycle limit maintains the loop pipeline and combines the six steps into four: LU decomposition, calculation of **F** (`D_extraction`, `D_square_root` and `G_decomposition`), forward substitution, and inversion. The

`#pragma HLS dataflow` directive, applied to the top-level function, implements a task-level pipeline across the four stages, with each stage assigned to a specific matrix (Section 3.3.1.4). Each step carries the directive `#pragma HLS inline off`, which keeps it as a separate function (KASTNER; MATAI; NEUENDORFFER, 2018, p. 52). In Figure 4.5, the latency of a matrix is the sum of the four steps, and the interval is the duration of the longest step.

Figure 4.5: Dataflow between the stages of the inversion. The code is simplified.

```
void cholesky_inverse(A, inverse) {
#pragma HLS dataflow
  etapa_lu(A, L, U);
  etapa_g(U, L, G);
  etapa_fs(G, G_inv);
  etapa_inv(G_inv, inverse);
}

static void etapa_fs(G, G_inv) {
#pragma HLS inline off
  forward_substitution(G, G_inv);
}
```



Source: the author. Numbers in the boxes: index of the matrix.

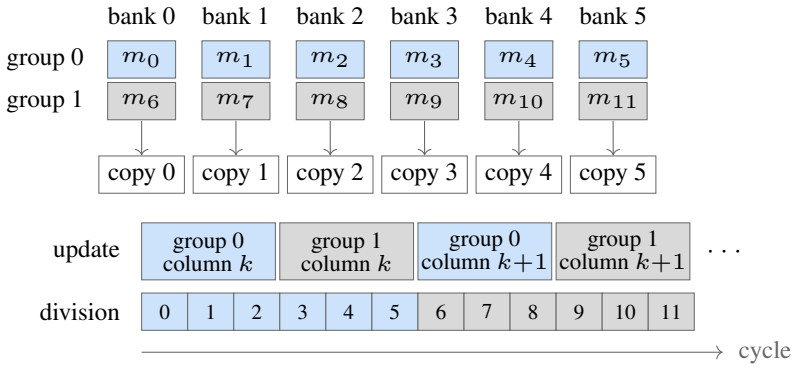
4.6.1.3 Matrix batches

Within the 606 and 454 cycle limits, the steps form a dataflow, as in the 4,545 cycle limit. The c copies read different matrices in the same cycle, but a BRAM memory only allows access to two addresses per cycle. Therefore, the batch memory is divided into c BRAM banks, one per copy, using the directive `#pragma HLS array_partition cyclic` and factor c. Thus, matrix m is located in bank $m \bmod c$ (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 87–88).

Figure 4.6 shows the architecture of the 606 cycle limit. Since the two groups of 6 matrices alternate in each cycle, one iteration does not wait for the result of the previous one. Each copy has its own multipliers, but there is a single divider in each step, shared by the copies. It receives one matrix of the batch per cycle and, since it has a pipeline, it also does not wait for the previous division.

Figure 4.6: Batch of 12 matrices with 6 copies, for the 606-cycle limit. The code is simplified.

```
#pragma HLS array_partition variable=U \
                        cyclic factor=COPIES dim=1
for (k = column; k < n; k++)
  for (g = 0; g < GRUPOS; g++) {
#pragma HLS pipeline II=1
    for (p = 0; p < COPIES; p++) {
      m = g * COPIES + p;
      t = L[m][line][column] * U[m][column][k];
      U[m][line][k] = U[m][line][k] - t;
    }
  }
```



Source: the author. Bank p holds the matrices with $m \bmod 6 = p$. Update: 6 matrices per cycle. Division: one matrix m per cycle, in a single divider.

4.6.1.4 Systolic arrays

Within the 151 and 113-cycle limits, the stages also form a dataflow, with each copy featuring its own divider. LU decomposition, forward substitution, and inversion utilize systolic arrays: the k-loop, which iterates over a row or a summation, is nested within a pipelined loop, which HLS unrolls into 16 processing elements (PEs) (Section 3.3.1.5). Each stage employs an array structure per copy, resulting in a total of $3c$ such arrays within the module.

In forward substitution and inversion, each PE adds a term and passes the partial sum to the next PE, following the same sequence as the unoptimized version, with each cycle computing a complete sum. In the LU decomposition, each cycle updates an entire row, with one element assigned to each PE. To achieve this, each row element is stored in its own dedicated register, configured using the `#pragma HLS array_partition complete` directive (KASTNER; MATAI; NEUENDORFFER, 2018, p. 87).

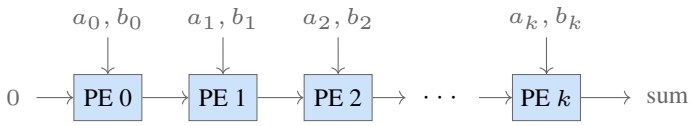
In both LU decomposition and forward substitution, each step reads rows that were written during the previous step, with a cycle offset corresponding to the step size. During the final steps of the LU decomposition and the initial steps of the forward substitution, which are relatively short, the required row might still be within the pipeline.

Consequently, these steps are padded with empty cycles until they reach a minimum duration (Figure 4.7). Since HLS cannot automatically infer this from the source code, the `#pragma HLS dependence ... dependent=false` directive is used to specify that the iterations are independent of one another (KASTNER; MATAI; NEUENDORFFER, 2018, p. 149).

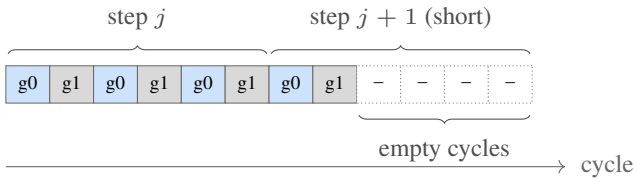
Figure 4.7: Systolic array of the forward substitution and minimum step of the LU decomposition. The code is simplified.

```
#pragma HLS array_partition variable=Ua \
                      complete dim=3
for (it = 0; it < LU_ITER; it++) {
#pragma HLS pipeline II=1
#pragma HLS dependence variable=Ua \
          type=inter dependent=false
  if (idx < reais)       // else: empty
    for (p = 0; p < COPIES; p++)
      for (k = 0; k < n; k++)  // PEs
        Ua[m][line][k] = row[k] - l*piv[k];
}
```

(a) PEs of one sum



(b) Steps of the LU decomposition



Source: the author. g0, g1: groups of matrices of the batch, one per cycle; there are 16 groups for the 151-cycle limit and 8 for the 113-cycle limit.

4.6.2 Test matrices

The hardware was tested with the same $\mathbf{G}_c$ matrices that BCD-MMSE inverts in the SER curves, with 1 iteration. These matrices were saved before inversion, in each of the 5 distinct test conditions (Section 4.3.2). In each condition, a sample was saved: some matrices randomly selected by configuration and SNR point, and the most extreme in range and precision, such as those with the largest element and the largest condition number. QPSK configurations were excluded because the modulation does not change $\mathbf{G}_c$.

The testbench inverts 20 matrices randomly selected from this sample. In the

16×16 circuit, these matrices come only from configurations 1A–1H. In architectures with batch, the 20 matrices are repeated until at least 3 batches are completed.

4.6.3 Power and energy

Power was estimated from the switching activity of the routed design: the version without optimizations under the 5 test conditions, and each optimized architecture only under its own. Vivado generates a functional netlist, which is simulated with the test-bench. The simulation checks the inverses byte by byte against the C code and records how many times each signal changes value in a SAIF (Switching Activity Interchange Format) file. Vivado then reads this file and reports the power.

Energy per inversion is defined as the dynamic power multiplied by the average time required for an inversion. This duration is calculated as the interval between two inversions, measured during the netlist simulation, multiplied by the 10 ns clock period. In architectures that employ batch processing, the interval between two batches is divided by n. A potência estática é a potência total menos a dinâmica.

5 RESULTADOS

5.1 Simulation setup

The simulation parameters are organized as the simulation flow of Figure 4.1. The base station configurations are inputs of both parts of the flow. The channels are generated before the simulation, by a MATLAB script that runs QuaDRiGa (Section 5.1.1). The Python simulator reads the stored channels and sets the other parameters: test conditions, signal and noise, receiver and evaluation (Section 5.1.2). In the tables, the source column gives the reference of each value, and the name in the code gives the variable that sets it. The parameter tables are in Appendix A.

5.1.1 Channel Generation in QuaDRiGa

Table A.1 shows the parameters of the MATLAB script that generates the channel matrices $\mathbf{H}$ and $\overline{\mathbf{H}}$. The remaining QuaDRiGa parameters were kept at their default values (JAECKEL; RASCHKOWSKI; THIELE, 2023).

The BS features a uniform linear array (ULA) of M antennas, with a standard spacing of half a wavelength ($\lambda/2$) of the operating signal. The antennas at both the BS and the UEs are omnidirectional, meaning they provide the same gain in all directions.

For each realization, the script randomly determines the positions of the K target UEs and the r interferers. The distance is sampled such that the UE density remains uniform across the entire area. The random seed used for this sampling is always the realization number. Based on these positions, QuaDRiGa generates the frequency-domain channel response between each BS antenna and each UE within the propagation scenario.

In this work, the `3GPP_38.901_UMa_NLOS` propagation scenario was used, which represents an urban macrocell (UMa) with no line-of-sight (NLoS) between the base station (BS) and the user equipment (UEs). Table A.2 presents the valid range for the UMa propagation scenario and the values selected. The system model assumes a flat-fading channel, in which all subcarriers within a physical resource block (PRB) experience the same channel conditions.

5.1.2 Python Simulator

The Python simulator reads the stored channels and runs the rest of the simulation flow. Its parameters are grouped as in Figure 4.2: test conditions, signal and noise, receiver and evaluation. The test conditions are described in Section 4.3.2.

5.1.2.1 Signal and Noise

Table A.3 shows the parameters used to read the channels and to generate the transmitted symbols and the noise.

The SNR depends solely on the ratio between the symbol energy E_s and the thermal noise power σ^2, as defined by the parameter `M5_Noise.snr_definition`. Accordingly, the constellations were normalized to have unit average energy (parameter `M4_Source.unit_energy_constellation`), and the symbol energy was set to $E_s = 1$ (parameter `M4_Source.Es`). Thus, the power σ^2 could be calculated based on the SNR to be simulated.

Furthermore, the power of the UEs was adjusted via channel normalization, defined by the parameter `M2_Channel.normalization`, ensuring that signals from UEs located at varying distances arrived at the BS with the same average power.

With the channel normalized, each of the r interferers reaches every BS antenna with the same average power. In this scenario, the interference received at each antenna is scaled by the number of interferers, r. To ensure that the ratio of interference to thermal noise at each antenna equals the IoTh value specified in the configuration (parameter `M5_Noise.iot_db`), the power β of each interferer is divided by r. This convention is governed by the parameter `M5_Noise.beta_convention` and follows the approach of averaging interference at each antenna, a method also employed in (TOMASIN et al., 2024).

The parameter `M5_Noise.data_noise_independent` specifies that the noise affecting the data symbols is drawn independently from the N noise samples used to estimate the covariance matrix $\hat{\mathbf{R}}$. The parameter `M5_Noise.resample_noise_per_realization` specifies that the N samples are re-drawn for each channel realization and each SNR point. This mimics the real behavior of the system, where samples come from the pilot REs and the data occupies other REs.

5.1.2.2 Receiver

Table A.4 shows the parameters for antenna partitioning, covariance estimation, and the equalizers.

Each cluster is assigned neighboring antennas (parameter `M3_Clustering.mapping`). The first cluster is assigned antennas 1 to M_c, the second is assigned antennas M_c+1 to $2M_c$, and so on. All clusters have the same number of antennas (parameter `M3_Clustering.equal_cluster_size`).

The equalizers receive the true $\mathbf{H}$ channel, without channel estimation (parameter `M7_Equalizers.perfect_csi`), as in (ZHAO et al., 2025). The BCD-MMSE was simulated with 1 iteration (parameter `M7_Equalizers.bcd_iterations`). The DUs update sequentially, from $c = 1$ to C, and each one uses the result of the previous one (parameter `M7_Equalizers.bcd_update_rule`) (ZHAO et al., 2025).

5.1.2.3 Evaluation

Table A.5 shows how the results are measured. The random number generator seed was set to 0, so that the results can be replicated.

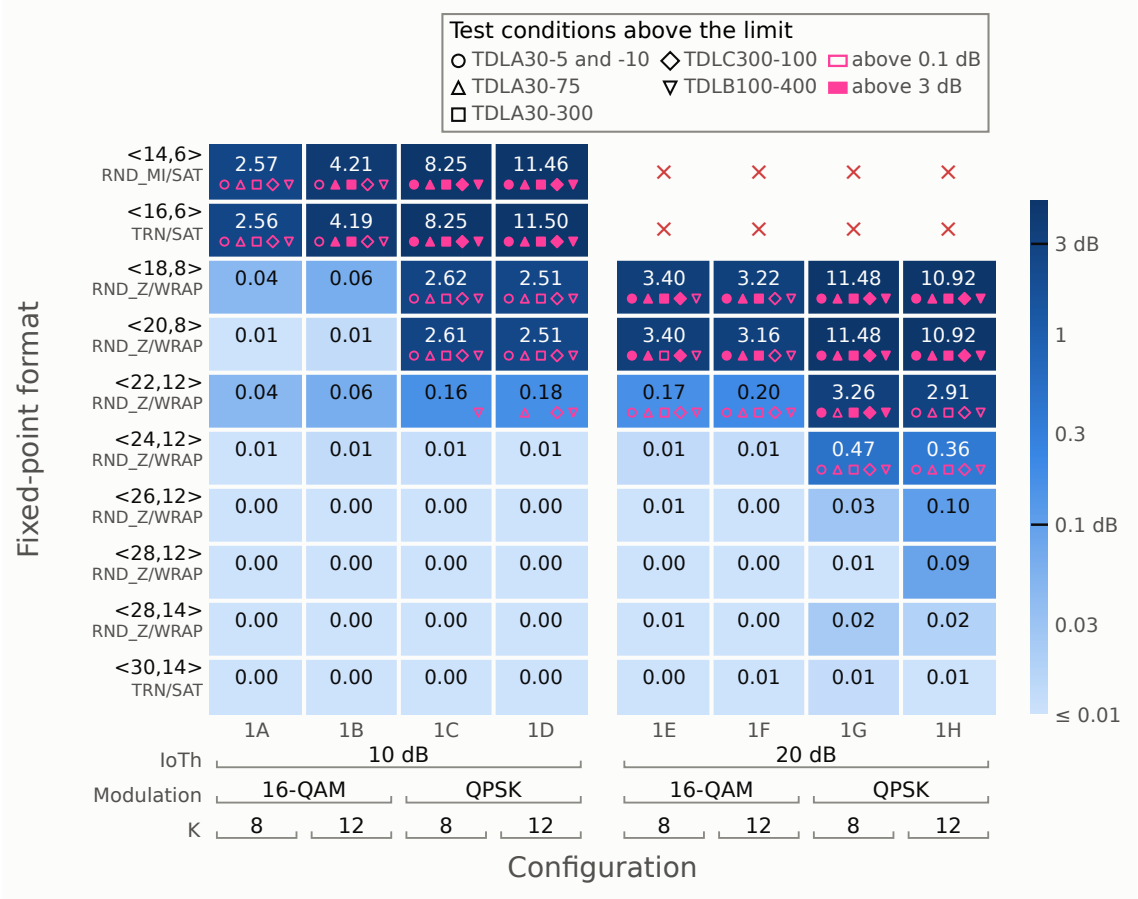
5.2 Selection and analysis of word format

This section presents the results of the validation of the 10 formats (Section 4.5.5). The results are grouped by the number of the configuration, which sets M, C and the size of the inversion circuit (Section 4.3.1): group 1 ($M = 128$, $C = 8$, 16×16 circuit), group 2 ($M = 256$, $C = 8$, 32×32 circuit) and group 3 ($M = 256$, $C = 16$, 16×16 circuit). All configurations of a group use the same circuit, and only the modulation, K and IoTh change between them.

For each group, the section presents the effect of the format on the SNR increase, the chosen formats, and how much of the increase over LMMSE comes from decentralization and how much from the format. Then, Section 5.3 shows whether the choice of format changes with the mobility profile. Depois, a Seção 5.4 mostra as curvas de SER dos três grupos. The hardware, throughput and delay results are presented by test condition, because each condition has its own architecture, set by its cycle limit (Section 5.5).

5.2.1 Grupo 1: $M = 128$, $C = 8$

Figure 5.1: Maior aumento de SNR, em dB, no grupo 1, configurações 1A a 1H ($M = 128$, $C = 8$, $M_c = 16$).

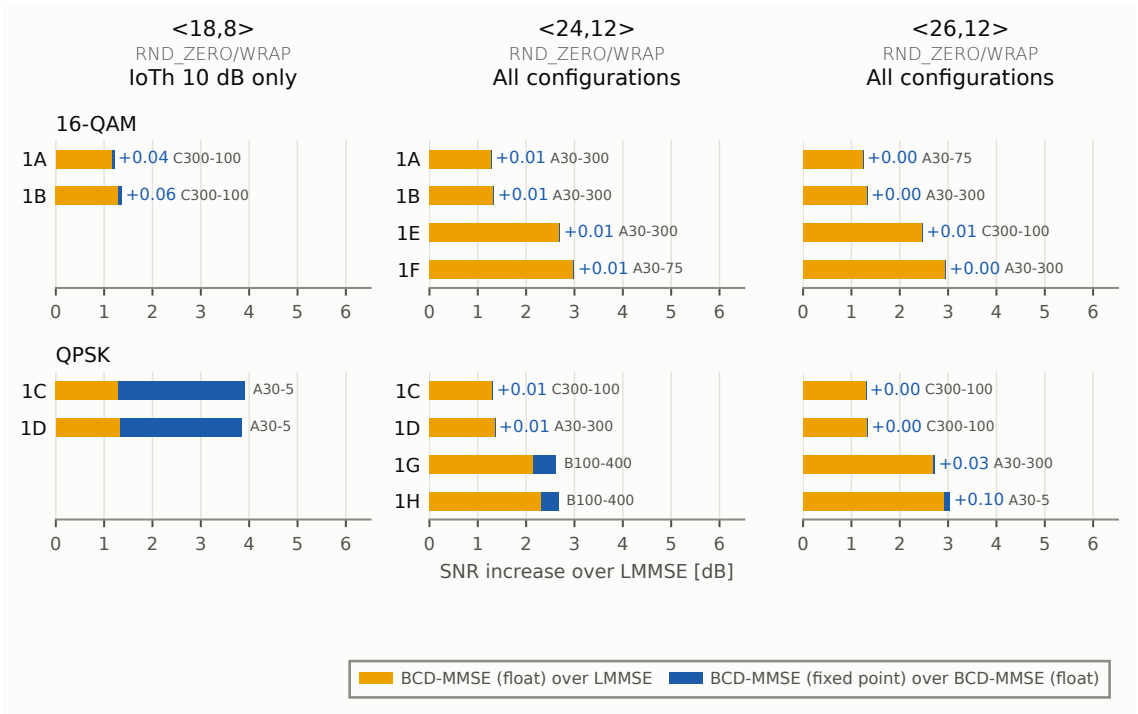


Fonte: a autora. Abaixo do formato, o modo de quantização e o de estouro, sem o prefixo AP_.

Figure 5.1 shows, on a logarithmic color scale, the largest SNR increase of each format (rows) in each configuration (columns), among the 5 simulated test conditions (Section 4.3.2). Below the value, unfilled symbols mark the test conditions above 0.1 dB, and filled symbols, those above 3 dB. The circle stands for TDLA30-5 and TDLA30-10, which share the same SER curves. A $\times$ marks a cell where some test condition does not reach $\mathrm{SER} = 10^{-2}$. The maps of groups 2 and 3 follow the same layout.

For each threshold, the chosen format is the smallest one in which all configurations stay within it (Section 4.5): $\langle 24, 12\rangle$ for 3 dB and $\langle 26, 12\rangle$ for 0.1 dB. The QPSK configurations with IoTh of 20 dB (1G and 1H) set both formats. When only the configurations with IoTh of 10 dB (1A to 1D) are considered, the chosen formats are $\langle 18, 8\rangle$ and $\langle 24, 12\rangle$, respectively.

Figure 5.2: Aumento de SNR do BCD-MMSE sobre o LMMSE no grupo 1 ($M = 128$, $C = 8$, $M_c = 16$), no pior caso de cada barra.



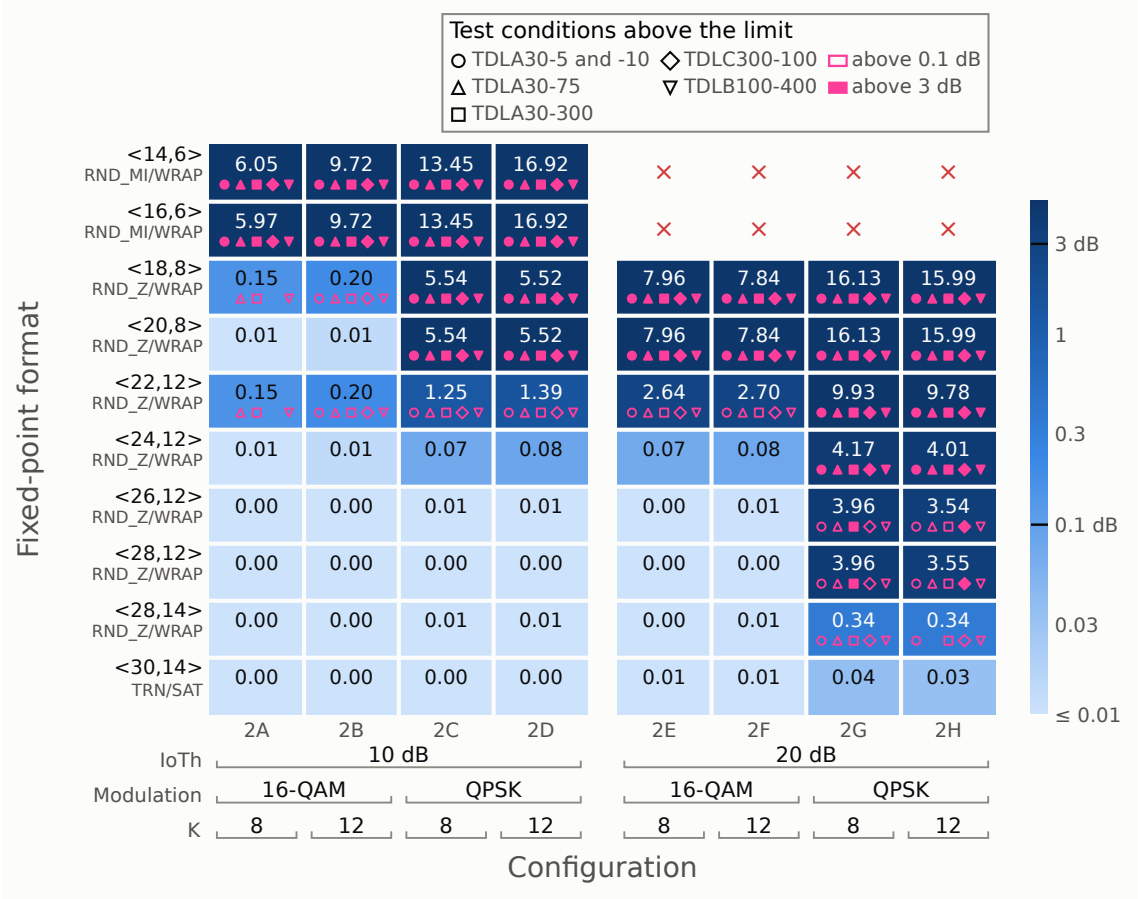
Fonte: a autora.

Figure 5.2 illustrates the source of the SNR gain provided by BCD-MMSE over LMMSE at an SER of 10^{-2}. The yellow bar represents the gain resulting from decentralization, that is, the difference between LMMSE and floating-point BCD-MMSE. The blue bar represents the gain (or impact) attributed to the numerical format, that is, the difference between floating-point and fixed-point BCD-MMSE, with the specific value indicated at the end of the bar. Each plot corresponds to a chosen format from the group (Table 5.1), and each bar represents a specific configuration. For each configuration, the test condition under which the format yields the greatest SNR increase was selected, and the name of this condition is labeled next to the bar.

In Group 1, decentralization provides the largest SNR gain when the IoTh is 20 dB. With 16-QAM, the gain attributed to the numerical format is negligible. In contrast, with QPSK, this gain is significantly larger, reaching 2.6 dB for the $\langle 18, 8 \rangle$ format, which exceeds the gain provided by decentralization. Thus, QPSK appears to be more sensitive to the approximations introduced by the numerical format than 16-QAM.

5.2.2 Grupo 2: $M = 256$, $C = 8$

Figure 5.3: Maior aumento de SNR, em dB, no grupo 2, configurações 2A a 2H ($M = 256$, $C = 8$, $M_c = 32$).

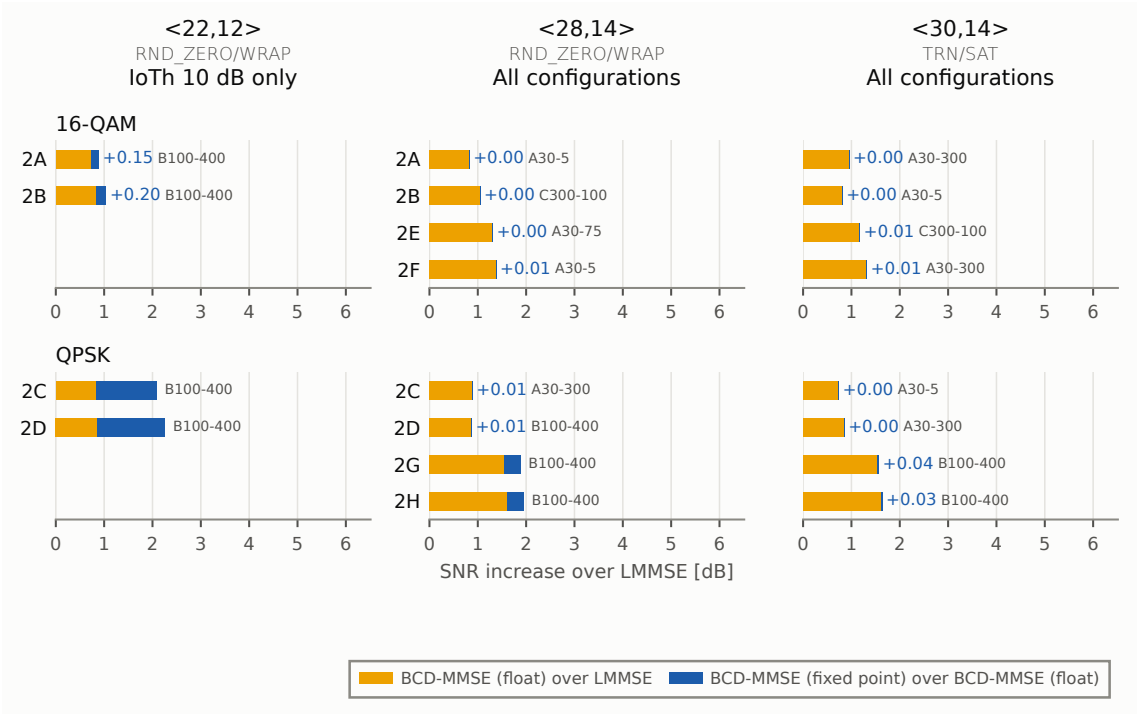


Fonte: a autora. Abaixo do formato, o modo de quantização e o de estouro, sem o prefixo AP_.

Group 2 has twice as many antennas as group 1, with the same number of clusters. Thus, each cluster has 32 antennas, not 16. In the map (Figure 5.3), the greatest increase in SNR in the smaller formats is greater than in group 1. Furthermore, the increase remains above 3 dB even in larger formats. In 2G and 2H, this goes up to $\langle 28, 12\rangle$, while 1G and 1H remain below 3 dB from $\langle 24, 12\rangle$. Therefore, the chosen formats are larger: $\langle 28, 14\rangle$ at the 3 dB limit and $\langle 30, 14\rangle$ at the 0.1 dB limit. As in group 1, the two are defined by 2G and 2H, with QPSK and IoTh of 20 dB.

In 2G and 2H, from $\langle 24, 12\rangle$ to $\langle 28, 12\rangle$, the increase hardly changes. These formats only gain fractional bits. The increase only decreases at $\langle 28, 14\rangle$, which has 2 more integer bits. Therefore, group 2 needs more integer bits than group 1. A likely explanation is the SNR at which the group operates. With more antennas, the equalizers need less

Figure 5.4: Aumento de SNR do BCD-MMSE sobre o LMMSE no grupo 2 ($M = 256$, $C = 8$, $M_c = 32$), no pior caso de cada barra.



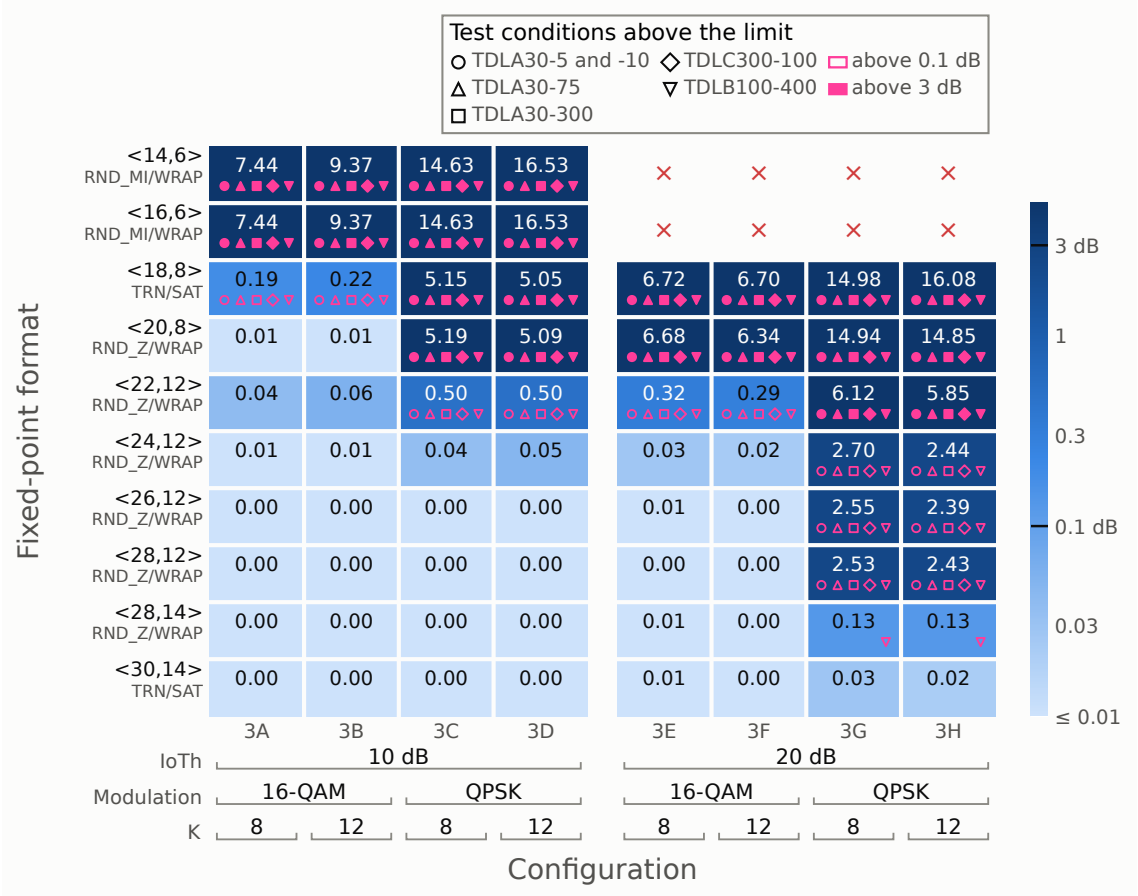
Fonte: a autora.

SNR, as also happens in (ZHAO et al., 2025). In the simulation, the symbol energy E_s is fixed, and the SNR varies according to the thermal noise power, σ^2 (Equation 3.1). With a fixed IoTh, the interference scales with σ^2 (Equation 3.2). Consequently, as the SNR decreases, the combined noise and interference become stronger, causing $\hat{\mathbf{R}}_{cc}$ to increase as well. Since $\mathbf{G}_c$ is formed by adding $\hat{\mathbf{R}}_{cc}$ to the channel term (Section 3.2.3), the elements of $\mathbf{G}_c$ also increase. At the same SNR, an IoTh of 20 dB results in ten times more noise-plus-interference than an IoTh of 10 dB, leading to even larger values for these elements. In Figure 5.4, decentralization increases the SNR less than in group 1. That is, with more antennas, the BCD-MMSE gets closer to the LMMSE, as also happens in (ZHAO et al., 2025). The increase in format size is also smaller because the chosen formats are larger. However, as in group 1, it is larger with QPSK than with 16-QAM.

5.2.3 Grupo 3: $M = 256$, $C = 16$

Group 3 has the same number of antennas as Group 2, but twice the number of clusters. Consequently, each cluster contains 16 antennas, just as in Group 1. As shown in the map (Figure 5.5), the most significant gain for smaller formats occurs near Group

Figure 5.5: Maior aumento de SNR, em dB, no grupo 3, configurações 3A a 3H ($M = 256$, $C = 16$, $M_c = 16$).



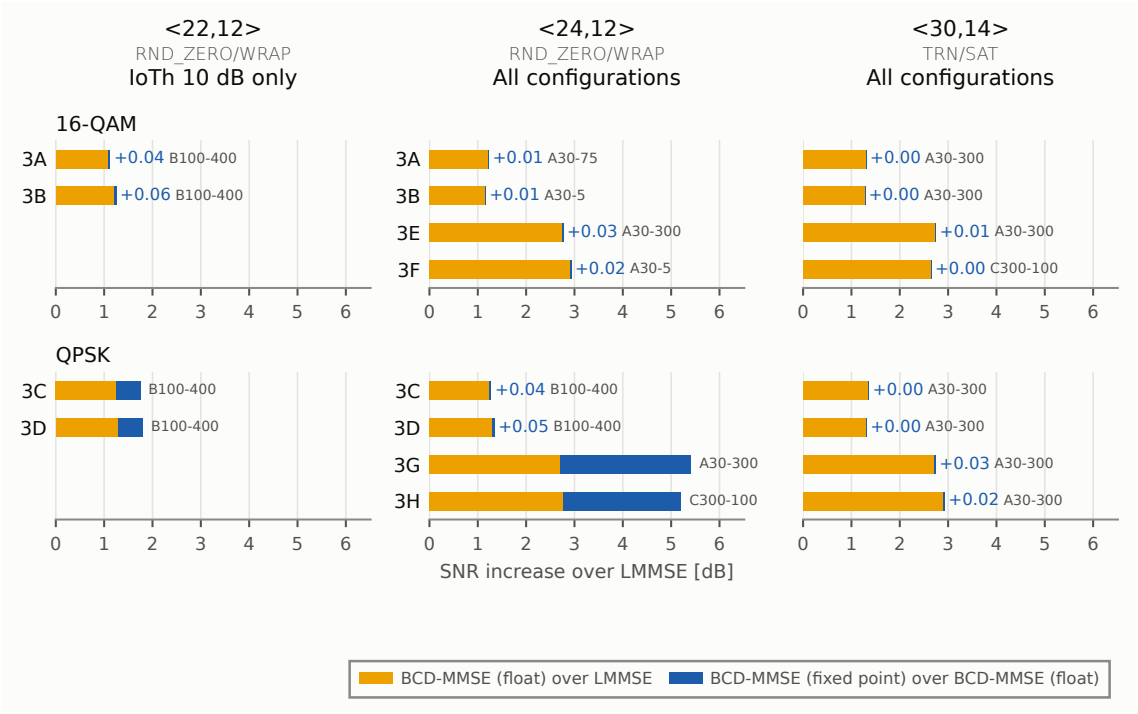
Fonte: a autora. Abaixo do formato, o modo de quantização e o de estouro, sem o prefixo AP_.

2. However, at the 3 dB threshold, the format is $\langle 24, 12\rangle$, the same as in Group 1 and smaller than the $\langle 28, 14\rangle$ format found in Group 2. This suggests that, in addition to the total number of antennas, the cluster size also influences the format.

For the $\langle 24, 12\rangle$ format, 3G and 3H lie close to the 3 dB threshold. In subsequent formats, similar to what is observed in Group 2, the gain for these cases remains nearly constant up to $\langle 28, 12\rangle$, only decreasing when two additional bits are introduced in the $\langle 28, 14\rangle$ format. Even then, it remains above 0.1 dB. Therefore, at the 0.1 dB threshold, the format is $\langle 30, 14\rangle$, which is the same as in Group 2.

In Figure 5.6, decentralization yields a greater increase in SNR compared to Group 2, bringing it closer to the levels seen in Group 1. Since the clusters contain fewer antennas, each one utilizes a smaller block of the covariance matrix. With a higher number of clusters, the BCD-MMSE performance deviates further from the LMMSE, a phenomenon also noted in (ZHAO et al., 2025). Using QPSK, for the $\langle 24, 12\rangle$ format, the gain provided by the format for 3G and 3H is nearly equivalent to the gain achieved through decentral-

Figure 5.6: Aumento de SNR do BCD-MMSE sobre o LMMSE no grupo 3 ($M = 256$, $C = 16$, $M_c = 16$), no pior caso de cada barra.



Fonte: a autora.

ization. For $\langle 22, 12\rangle$, with an IoTh of only 10 dB, the gap is smaller than in group 2.

Thus, across the three groups, the primary factor driving the divergence between BCD-MMSE and LMMSE depends on the modulation scheme. With 16-QAM, the impact of the numerical format is almost negligible, and the decentralization effect carries more weight. With QPSK, for the formats at the 3 dB threshold, the numerical format carries as much weight as the decentralization, or even more; this is the case for $\langle 18, 8\rangle$ in group 1, $\langle 22, 12\rangle$ in group 2, and $\langle 24, 12\rangle$ in group 3. For the formats at the 0.1 dB threshold, the decentralization effect is the dominant factor in all cases.

5.2.4 Formatos escolhidos

Table 5.1 presents the formats selected across the three groups, both with all configurations and with only the 10 dB IoTh configurations (A through D). Excluding the 20 dB IoTh configurations results in formats that are 2 to 6 bits smaller, with the 0.1 dB format becoming $\langle 24, 12\rangle$ in all three groups. Since some formats overlap, there are three distinct formats to synthesize for groups 1 and 3, and four for group 2 (Section 5.5).

Table 5.1: Formatos escolhidos em cada grupo de configurações.

Criterion	Group 1	Group 2	Group 3
All configurations (A to H)			
Increase up to 3 dB	$\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 28, 14\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP
Increase up to 0.1 dB	$\langle 26, 12\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 30, 14\rangle$ AP_TRN/AP_SAT	$\langle 30, 14\rangle$ AP_TRN/AP_SAT
IoTh of 10 dB (A to D)			
Increase up to 3 dB	$\langle 18, 8\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 22, 12\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 22, 12\rangle$ AP_RND_ZERO/AP_WRAP
Increase up to 0.1 dB	$\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP	$\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP

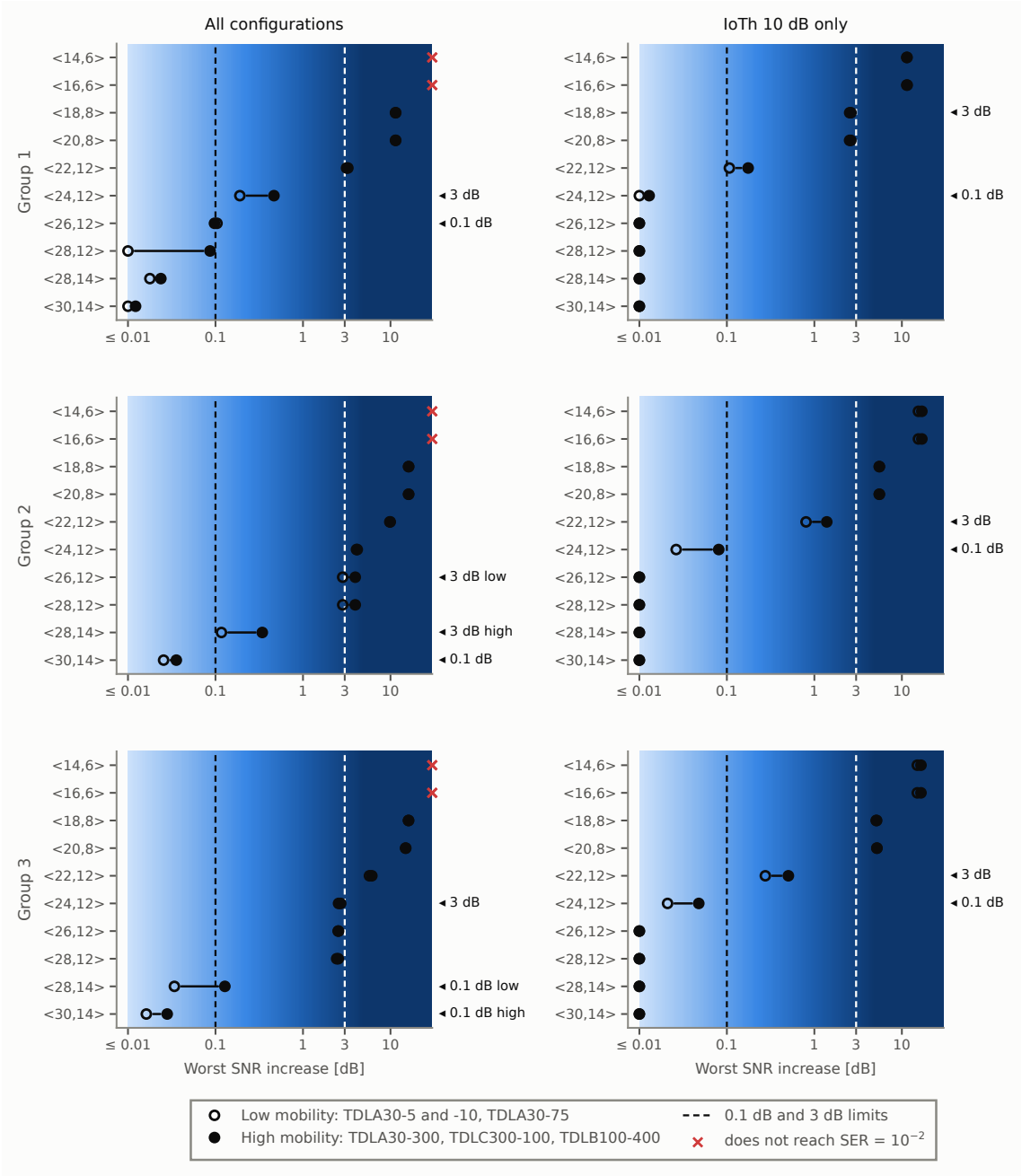
Fonte: a autora. Abaixo do formato, o modo de quantização e o de estouro.

5.3 Mobility profiles

Figure 5.7 shows the largest SNR increase of each of the 10 validated formats (Section 4.5.5) for each mobility profile (Section 4.3.2). Each row represents a group. The first column includes all configurations, while the second column is restricted to those with an IoTh of 10 dB. Each data point represents the largest increase among the configurations within that column under the conditions of the respective profile. The triangle on the right-hand side marks the first format, from top to bottom, that falls within each limit; this is the format selected for that profile.

For most formats, the data points for both profiles are close to one another, with the high-mobility point appearing either to the right of or directly atop the low-mobility point. In other words, the mobility profile has little impact on the SNR increase. However, under low-mobility conditions alone, the selected format would be 2 bits smaller in two instances: $\langle 26, 12\rangle$ instead of $\langle 28, 14\rangle$ in group 2, with the 3 dB limit, and $\langle 28, 14\rangle$ instead of $\langle 30, 14\rangle$ in group 3, with the 0.1 dB limit. In all other cases, both profiles select the same format. Consequently, the high-mobility profile determines the format. Appendix D presents the SNR increase of each configuration with the selected formats.

Figure 5.7: Largest SNR increase, in dB, of each format in each mobility profile, in the three groups.



Source: the author. Each format uses the same mode as in the SNR increase maps (Figures 5.1 to 5.5): RND_ZERO/WRAP, except $\langle 14, 6\rangle$ in RND_MIN_INF/SAT in group 1 and RND_MIN_INF/WRAP in groups 2 and 3; $\langle 16, 6\rangle$ in TRN/SAT in group 1 and RND_MIN_INF/WRAP in groups 2 and 3; $\langle 18, 8\rangle$ in TRN/SAT in group 3; and $\langle 30, 14\rangle$ in TRN/SAT in all three groups.

5.4 SER curves

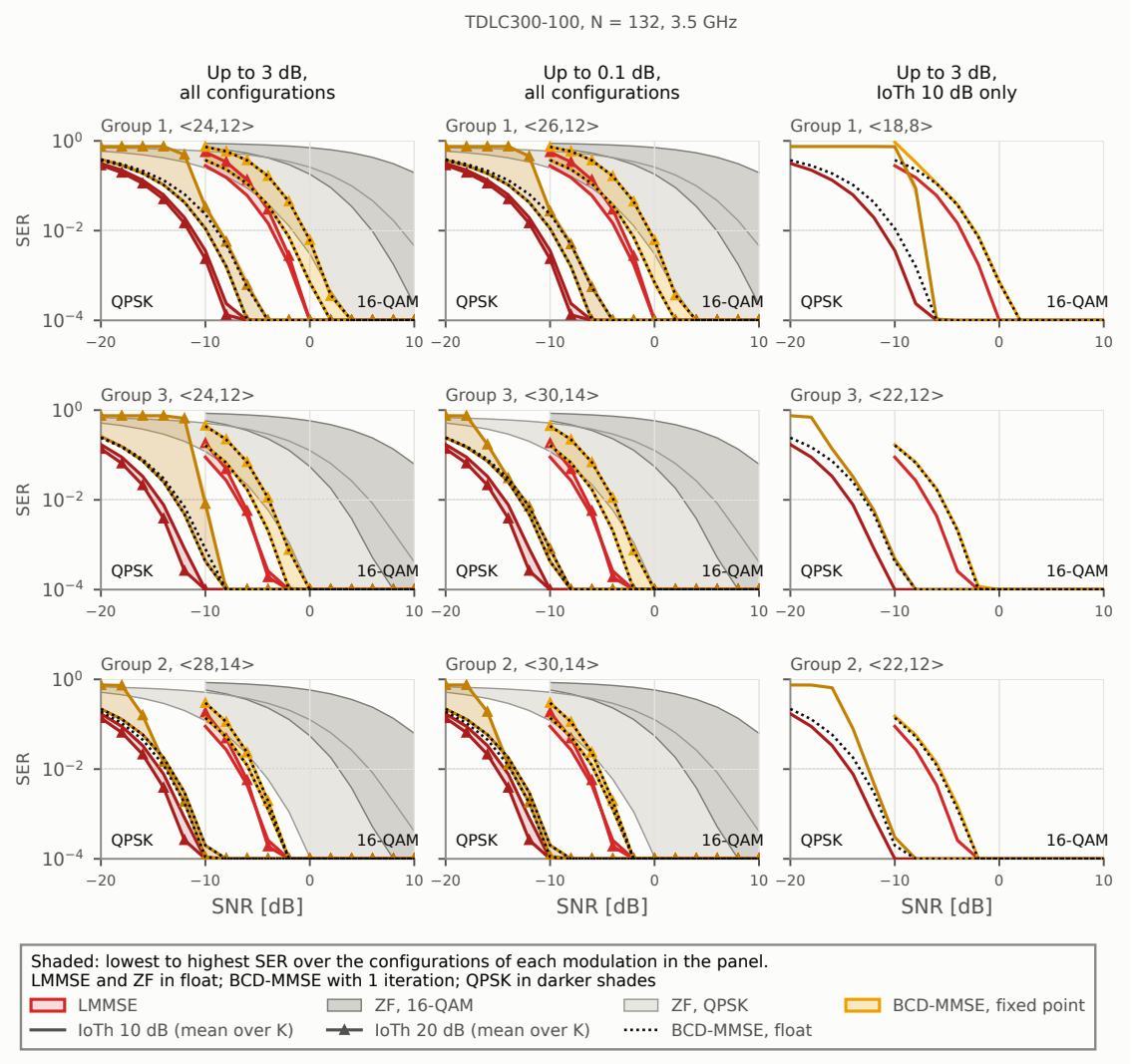
Figure 5.8 shows the SER curves for LMMSE and ZF (in floating-point) and BCD-MMSE (in fixed-point) for the TDLC300-100 channel model. The analysis here is not limited to the $\mathrm{SER} = 10^{-2}$ point; the curves cover the entire SNR range, from -20 to 10 dB. Each row of plots represents a group of configurations, while each column corresponds to a criterion from Table 5.1; the BCD-MMSE in each plot utilizes the format selected for that specific case. The first two columns include all configurations, the first with the 3 dB format and the second with the 0.1 dB format. The third column contains only the configurations with an IoTh of 10 dB, using the 3 dB format selected for them. The ZF performance remains unchanged by the format and is included in the plots as a worst-case reference; for the sake of clarity, it is omitted from the third column. The curves for the remaining test conditions can be found in Appendix C.

Within a group, the configurations vary only in terms of soft constraints: modulation, K, and IoTh (Section 4.3.1). In each plot, the QPSK curves appear on the left in darker shades, while the 16-QAM curves appear on the right in lighter shades. The dotted line represents the floating-point BCD-MMSE, which serves as the reference for the format. Each curve represents the average between $K = 8$ and $K = 12$; these values differ by at most 0.9 dB at an SER of 10^{-2}. The shaded region surrounding the curves spans from the lowest to the highest SER observed across the configurations of that modulation. Since K has little impact, the width of this region in the first two columns primarily reflects the difference between IoTh values of 10 dB and 20 dB.

In floating-point arithmetic, QPSK operates at lower SNRs than 16-QAM (Section 3.1.2). In fixed-point arithmetic, however, some QPSK curves begin with a plateau: a flat, horizontal segment at the top of the plot, corresponding to an SER of 0.75. An SER of 0.75 implies that 3 out of every 4 symbols are incorrect, equivalent to a random selection among the 4 constellation points. Within this plateau, increasing the SNR does not improve the SER; beyond it, the curve drops sharply.

With a greater number of integer bits, the plateau ends at a lower SNR. In groups 1 and 2, the 0.1 dB format differs from the 3 dB format only by having more fractional bits, and the plateau ends at the same SNR. In group 3, the $\langle 30, 14 \rangle$ format has two more integer bits than the $\langle 24, 12 \rangle$ format, causing the plateau to end earlier. 16-QAM, on the other hand, operates at higher SNRs and shows almost no variation across the different formats. The exception is the $\langle 18, 8 \rangle$ format, where the 16-QAM curve begins above the

Figure 5.8: Curvas de SER dos três grupos na TDLC300-100, nos formatos escolhidos.



Fonte: a autora.

floating-point curve, near $\mathrm{SER} = 1$, and quickly converges with it.

It should be noted that the LMMSE in the graphs is in floating point. Consequently, one cannot draw conclusions regarding how its performance varies with different IoTh values compared to the BCD-MMSE approach. It is observable, however, that the groups with smaller clusters (groups 1 and 3) exhibit the greatest gap between the BCD-MMSE curves for IoTh values of 10 dB and 20 dB, particularly when using 16-QAM (Figures 5.2 and 5.6).

Considering only the configurations with an IoTh of 10 dB, the 3 dB formats feature 2 to 6 fewer bits (Table 5.1). As shown in the third column, these formats remain within the 3 dB increase threshold at an SER of 10^{-2}. Regarding the SNR increase of BCD-MMSE over LMMSE, no specific target was defined; nevertheless, this increase

exceeds 3 dB only in the case of QPSK. This occurs in the smaller formats, $\langle 18, 8\rangle$ for group 1 and $\langle 24, 12\rangle$ for group 3, and, marginally, for the 1H configuration in $\langle 26, 12\rangle$ (Figures 5.2 and 5.6).

QPSK is the modulation scheme most sensitive to the number of bits because it operates at lower SNRs, where the elements of $\mathbf{G}_c$ are larger (Section 5.2.2). When these elements exceed the capacity of the chosen format, an integer overflow occurs, which results in a plateau (Section 3.3.1.2). For instance, the $\langle 18, 8\rangle$ format can store values up to 128. In the 1C case, the median of the largest element in $\mathbf{G}_c$ ranges from 70 at -6 dB to 164 at -10 dB. It exceeds the limit for nearly all matrices at -10 dB, while it does not exceed the limit for any matrix at -6 dB. It is within this range that the QPSK curve emerges from the plateau. Although the modulation scheme itself does not alter $\mathbf{G}_c$ (Section 4.6.2), 16-QAM operates at higher SNRs, where such overflows occur less frequently.

5.5 Arquiteturas de hardware

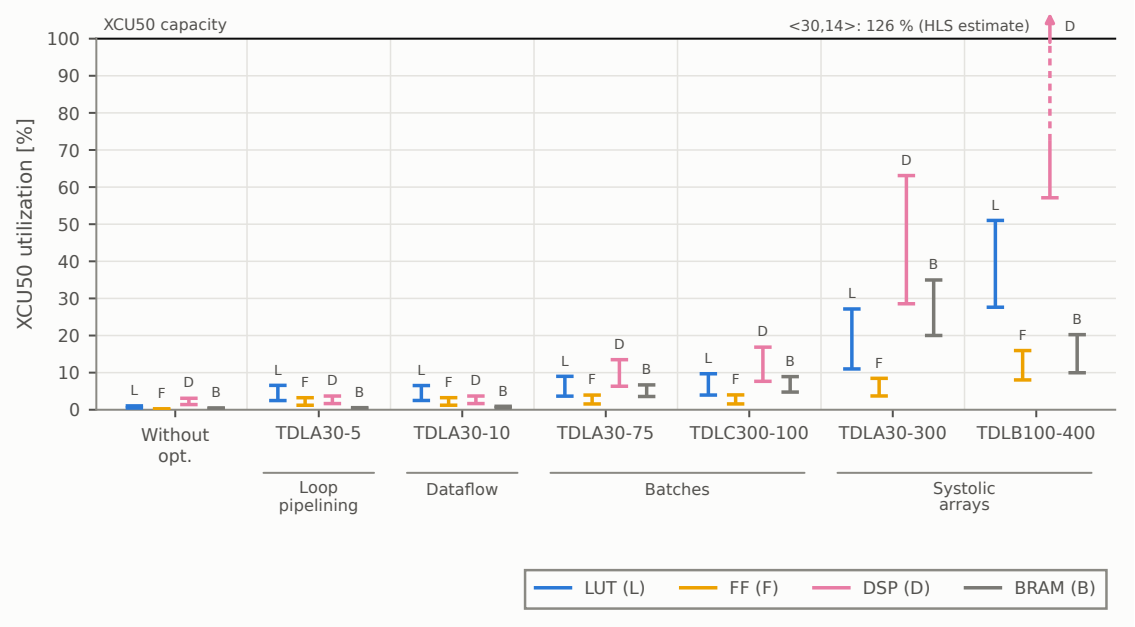
Esta seção mostra os resultados de implementação das arquiteturas no XCU50, a 100 MHz. Em todos os formatos, nas arquiteturas otimizadas, o intervalo entre duas inversões fica abaixo do limite de ciclos da Tabela 4.4. O módulo aceita um bloco novo de 275 matrizes a cada T_c e acompanha a chegada das amostras. O timing é alcançado em todas as arquiteturas, com maior caminho crítico de 9,74 ns, na TDLB100-400.

A Seção 5.5.1 analisa os recursos de cada arquitetura, na mesma ordem da Seção 4.6.1, e mostra qual recurso cada uma mais usa e por quê. A Seção 5.5.2 compara as arquiteturas e os formatos, e a Seção 5.5.3 mostra a potência e a energia por inversão.

5.5.1 Recursos por arquitetura

A Figura 5.9 mostra o uso de cada recurso em cada arquitetura, em porcentagem do XCU50. Cada traço vai do menor ao maior uso entre os formatos. O recurso mais usado de cada arquitetura é o traço mais alto do grupo. Nas seções a seguir, cada arquitetura é comparada com a anterior. Quando o texto diz, por exemplo, que as LUTs crescem de 4,6 a 6,8 vezes, a conta é feita com o mesmo formato nas duas arquiteturas, e a faixa vai do formato que cresce menos ao que cresce mais (Tabelas B.2 e B.1).

Figure 5.9: Uso de cada recurso do XCU50 por arquitetura, do menor ao maior entre os formatos.



Fonte: a autora. Circuito 16×16. No DSP da TDLB100-400, a seta indica que o $\langle 30, 14 \rangle$ passa da capacidade do XCU50: a estimativa do HLS é de 126%, e ele não cabe no FPGA.

5.5.1.1 Pipeline de laços

Na versão sem otimização, o recurso mais usado são os DSPs, com 1,4 a 3,1%. A arquitetura da TDLA30-5 reduz o intervalo de 3,6 a 4,5 vezes. Os DSPs crescem só de 1,1 a 1,2 vez, mas as LUTs crescem de 4,6 a 6,8 vezes, e os FFs, de 9,4 a 13,2 vezes. Com isso, o recurso mais usado passa a ser a LUT, com 2,5 a 6,6%.

Esse aumento vem dos divisores. No relatório do HLS, os divisores da versão sem otimização são sequenciais: cada um faz uma divisão por vez e só aceita a próxima quando termina. Com o pipeline de laços, o laço da divisão começa uma iteração por ciclo, e o HLS usa divisores em pipeline, que aceitam uma divisão por ciclo. O único relatório do HLS com o uso de cada operador dessa arquitetura é o de palavras de 24 bits. Nele, os quatro divisores da LU e da substituição direta somam 91% dos FFs e 71% das LUTs, e nenhum DSP.

5.5.1.2 Dataflow entre etapas

A arquitetura da TDLA30-10 reduz o intervalo mais 2,5 a 2,7 vezes, e as LUTs, os FFs e os DSPs não mudam. O dataflow não cria operadores novos (Seção 4.6.1.2). O custo fica na memória, e as BRAMs crescem de 1,3 a 1,7 vez. Entre as etapas, o HLS

usa buffers ping-pong, que guardam cada matriz pelo menos duas vezes (KASTNER; MATAI; NEUENDORFFER, 2018, p. 115). Como os divisores são os mesmos, o recurso mais usado continua sendo a LUT, com 2,5 a 6,5%.

5.5.1.3 Lotes de matrizes

Com lote, as BRAMs seguem o número de bancos de memória, e não o número de matrizes guardadas. Cada banco ocupa pelo menos um bloco de BRAM, mesmo quando guarda poucas palavras.

Na TDLA30-75, com 6 cópias, o intervalo por matriz é de 5,7 a 6,3 vezes menor que na TDLA30-10. Os DSPs crescem de 3,6 a 3,8 vezes, e as LUTs, só de 1,3 a 1,5 vez. Os multiplicadores das atualizações são copiados 6 vezes, mas o divisor de cada etapa é um só para as cópias (Seção 4.6.1.3). Assim, a parte do circuito que mais usa LUTs não é copiada. As BRAMs crescem de 6,9 a 8,2 vezes, porque cada matriz passa a ficar em 6 bancos, um por cópia. Da TDLA30-75 para a TDLC300-100, as cópias vão de 6 para 8, e as BRAMs crescem na mesma proporção, 1,33 vez, em todos os formatos.

Assim, a BRAM e os DSPs ficam próximos, e o recurso mais usado depende do formato. Do $\langle 20, 8\rangle$ ao $\langle 24, 12\rangle$, é a BRAM, com 6,7% na TDLA30-75 e 8,9% na TDLC300-100. No $\langle 18, 8\rangle$, essas arquiteturas usam cerca de metade das BRAMs dos outros formatos, e a partir do $\langle 26, 12\rangle$, os DSPs crescem. Nesses formatos, o recurso mais usado são os DSPs.

Os DSPs crescem com o formato porque os multiplicadores ficam mais largos. Nos relatórios do HLS, os operandos dos multiplicadores das atualizações têm 28 bits no $\langle 18, 8\rangle$, 36 bits no $\langle 24, 12\rangle$ e 46 bits no $\langle 30, 14\rangle$. Uma multiplicação mais larga é feita com vários DSPs (AMD, 2026, p. 499). Do $\langle 18, 8\rangle$ ao $\langle 24, 12\rangle$, o total de DSPs quase não muda, e no relatório do $\langle 24, 12\rangle$ cada multiplicador usa 4 DSPs. A partir do $\langle 26, 12\rangle$, com 40 bits, o total cresce. No $\langle 30, 14\rangle$, cada multiplicador usa 9 DSPs.

5.5.1.4 Arranjos sistólicos

Na TDLA30-300, com só 2 cópias, o intervalo é de 3,0 a 3,3 vezes menor que na TDLC300-100. Em cada etapa, as 2 cópias somam 32 multiplicadores, um por PE, contra 8 na TDLC300-100, e cada cópia tem o seu divisor (Seção 4.6.1.4). Os DSPs crescem de 3,7 a 3,8 vezes, e as LUTs, de 2,7 a 2,9 vezes. Cada linha da matriz fica em 16 bancos, um por PE, e cada cópia tem os seus. Assim, as BRAMs crescem de 3,8 a 4,2 vezes.

No relatório do HLS de palavras de 24 bits, cada um desses bancos ocupa um bloco de 18 Kb, mas guarda só 256 palavras, cerca de um terço do bloco. Como na TDLC300-100, o recurso mais usado é a BRAM do $\langle 20, 8\rangle$ ao $\langle 24, 12\rangle$, com 34,2 a 34,4%, e os DSPs nos outros formatos. Os multiplicadores são os mesmos dos lotes, e os DSPs crescem com o formato pelo mesmo motivo (Seção 5.5.1.3). Mas, como a TDLA30-300 já usa muitos DSPs, esse crescimento pesa mais no FPGA: os DSPs vão de 28,6% no $\langle 24, 12\rangle$ para 63,1% no $\langle 30, 14\rangle$.

Na TDLB100-400, as cópias vão de 2 para 4, e o intervalo cai à metade, de 128 para 64 ciclos. Os DSPs também dobram. As LUTs crescem de 2,4 a 2,5 vezes, e as BRAMs caem quase à metade. Com 4 cópias, cada banco guarda metade das palavras, e o HLS fez esses bancos em LUTs, e não em BRAM. Uma memória pode ser feita em BRAM, em LUTs (LUTRAM) ou em URAM (AMD, 2026, p. 53). No relatório do Vivado, as LUTs usadas como memória crescem de 7,4 a 9,8 vezes, e as LUTs de lógica, 2,0 vezes, como as cópias. Assim, em todos os formatos, o recurso mais usado são os DSPs, com 57,1 a 72,4%, e as BRAMs ficam entre 10,0 e 20,2%.

5.5.2 Comparação entre as arquiteturas

A Tabela 5.2 mostra o melhor e o pior caso de cada arquitetura entre os formatos: o menor e o maior uso de cada recurso, de caminho crítico e de intervalo entre inversões. Os valores de cada formato estão nas Tabelas B.2 e B.1 (Apêndice B).

As arquiteturas sem lote (TDLA30-5 e TDLA30-10) usam no máximo 6,6% de cada recurso, e a versão sem otimização, no máximo 3,1%. Em 32×32, a versão sem otimização usa quase os mesmos recursos que em 16×16, mas o intervalo cresce de 5 a 6 vezes.

A Tabela 5.3 mostra o recurso mais usado em cada formato. O maior efeito do formato está no $\langle 30, 14\rangle$, em que os DSPs quase dobram em relação ao $\langle 28, 14\rangle$: na TDLA30-300, vão de 36 para 63%. Na TDLB100-400, o $\langle 30, 14\rangle$ não cabe no FPGA: a estimativa do HLS é de 7.512 DSPs, e o XCU50 tem 5.952.

O gráfico da esquerda da Figura 5.11 mostra o recurso mais usado. Como ele muda de uma arquitetura para outra, a ocupação não segue o limite de ciclos. Da TDLA30-10 para a TDLA30-75, o limite fica 7,5 vezes menor, e a ocupação cresce só de 1,4 a 2,5 vezes. Da TDLA30-300 para a TDLB100-400, o limite fica só 1,3 vez menor, e a ocupação cresce de 1,7 a 2,0 vezes.

Table 5.2: Menor e maior uso de recursos, caminho crítico e intervalo entre inversões de cada arquitetura, entre os formatos.

Test condition	**LUT (%)**	**FF (%)**	**DSP (%)**	**BRAM (%)**	**Critical path (ns)**	**Interval (cycles)**	**Limit (cycles)**
TDLA30-5	2.5–6.6	1.2–3.2	1.7–3.7	0.4–0.6	7.24–9.03	7,081–8,627	9,090
TDLA30-10	2.5–6.5	1.2–3.3	1.7–3.7	0.5–0.9	7.54–8.36	2,765–3,320	4,545
TDLA30-75	3.7–9.0	1.6–4.0	6.3–13.5	3.6–6.7	7.87–8.98	483.9–528.9	606
TDLC300-100	4.0–9.7	1.6–4.0	7.7–16.9	4.8–8.9	7.79–9.37	387.3–421.1	454
TDLA30-300	11.0–27.2	3.7–8.5	28.6–63.1	20.0–35.0	8.73–9.66	128	151
TDLB100-400	27.6–51.0	8.0–15.9	57.1–72.4	10.0–20.2	9.38–9.74	64	113
Without opt., 16×16	0.5–1.0	0.1–0.2	1.4–3.1	0.3–0.5	7.80–9.15	26,702–38,567	–
Without opt., 32×32	0.5–1.0	0.1–0.3	1.4–3.1	0.3–0.7	7.72–8.58	159,086–194,367	–

Fonte: a autora. Recursos em porcentagem do XCU50. Em quase todos os casos, o menor valor vem do $\langle 18, 8\rangle$ e o maior, do $\langle 30, 14\rangle$ ($\langle 28, 14\rangle$ na TDLB100-400). O caminho crítico não segue essa ordem. Em 32×32, sem o intervalo do $\langle 18, 8\rangle$ e do $\langle 20, 8\rangle$, que não foram medidos.

Table 5.3: Recurso mais usado do XCU50, em porcentagem, por formato e arquitetura.

Test condition	$\langle 18, 8\rangle$	$\langle 18, 8\rangle^*$	$\langle 20, 8\rangle$	$\langle 22, 12\rangle$	$\langle 24, 12\rangle$	$\langle 26, 12\rangle$	$\langle 28, 14\rangle$	$\langle 30, 14\rangle^*$
TDLA30-5	2.5 (L)	2.6 (L)	3.2 (L)	3.2 (L)	4.1 (L)	4.9 (L)	5.4 (L)	6.6 (L)
TDLA30-10	2.5 (L)	2.6 (L)	3.2 (L)	3.3 (L)	4.1 (L)	5.0 (L)	5.5 (L)	6.5 (L)
TDLA30-75	6.3 (D)	6.3 (D)	6.7 (B)	6.7 (B)	6.7 (B)	7.7 (D)	7.9 (D)	13.5 (D)
TDLC300-100	7.7 (D)	7.7 (D)	8.9 (B)	8.9 (B)	8.9 (B)	9.4 (D)	9.7 (D)	16.9 (D)
TDLA30-300	28.6 (D)	28.6 (D)	34.2 (B)	34.3 (B)	34.4 (B)	35.1 (D)	36.2 (D)	63.1 (D)
TDLB100-400	57.2 (D)	57.2 (D)	57.6 (D)	57.3 (D)	57.1 (D)	70.3 (D)	72.4 (D)	–

Fonte: a autora. Entre parênteses, o recurso: L = LUT, D = DSP, B = BRAM. *`AP_TRN`/`AP_SAT`; os outros, `AP_RND_ZERO`/`AP_WRAP`. –: não cabe no FPGA.

5.5.3 Potência e energia

A Tabela 5.4 mostra a potência dinâmica. Em cada formato, a da TDLB100-400 é de 15 a 20 vezes a da TDLA30-5. O formato pesa menos que a arquitetura. Do $\langle 18, 8\rangle$ ao $\langle 28, 14\rangle$, a potência dinâmica cresce de 1,6 a 2,0 vezes em cada arquitetura. Nesse intervalo, cada bit a mais na palavra custa, em média, 15 mW na TDLA30-5 e 180 mW na TDLB100-400. A potência estática fica entre 2,21 e 2,30 W em todas as arquiteturas e formatos.

A Tabela 5.5 e a Figura 5.10 mostram a energia por inversão. Nas arquiteturas com lote, ela fica entre 1,75 e 4,59 μJ, e as quatro ficam próximas em cada formato. A da TDLA30-5 gasta de 5,7 a 8,8 vezes mais que elas, no mesmo formato. Ela tem a menor potência dinâmica, mas cada inversão leva de 7.081 a 8.627 ciclos, contra 64 a 529 nas arquiteturas com lote. A da TDLA30-10 gasta cerca de metade da TDLA30-5.

Table 5.4: Potência dinâmica, em W, por formato e arquitetura.

Test condition	$\langle 18,8\rangle$	$\langle 18,8\rangle^*$	$\langle 20,8\rangle$	$\langle 22,12\rangle$	$\langle 24,12\rangle$	$\langle 26,12\rangle$	$\langle 28,14\rangle$	$\langle 30,14\rangle^*$
TDLA30-5	0.151	0.158	0.187	0.194	0.234	0.280	0.305	0.377
TDLA30-10	0.209	0.208	0.252	0.253	0.305	0.362	0.391	0.496
TDLA30-75	0.369	0.400	0.450	0.474	0.546	0.618	0.647	0.868
TDLC300-100	0.452	0.482	0.570	0.573	0.666	0.759	0.805	1.037
TDLA30-300	1.368	1.425	1.679	1.753	1.973	2.087	2.180	3.065
TDLB100-400	2.908	3.167	3.313	3.477	4.062	4.456	4.703	–
Without opt., 16×16	0.031	0.034	0.035	0.037	0.041	0.046	0.048	0.059
Without opt., 32×32	–	–	–	0.040	0.046	0.051	0.053	0.065

Fonte: a autora. Arquitetura de cada condição de teste em 16×16. *`AP_TRN`/`AP_SAT`; os outros, `AP_RND_ZERO`/`AP_WRAP`. –: sem medida ou não cabe no FPGA.

Table 5.5: Energia por inversão, em μJ, por formato e arquitetura.

Test condition	$\langle 18,8\rangle$	$\langle 18,8\rangle^*$	$\langle 20,8\rangle$	$\langle 22,12\rangle$	$\langle 24,12\rangle$	$\langle 26,12\rangle$	$\langle 28,14\rangle$	$\langle 30,14\rangle^*$
TDLA30-5	10.69	11.60	13.74	14.25	17.77	21.96	24.66	32.52
TDLA30-10	5.81	5.75	7.31	7.34	9.21	11.37	12.51	16.47
TDLA30-75	1.79	1.94	2.22	2.34	2.75	3.18	3.36	4.59
TDLC300-100	1.75	1.87	2.25	2.26	2.68	3.11	3.33	4.37
TDLA30-300	1.75	1.82	2.15	2.24	2.52	2.67	2.79	3.92
TDLB100-400	1.86	2.03	2.12	2.23	2.60	2.85	3.01	–
Without opt., 16×16	8.28	9.09	10.64	11.25	13.31	15.81	17.00	22.75
Without opt., 32×32	–	–	–	63.63	76.96	89.24	94.92	126.34

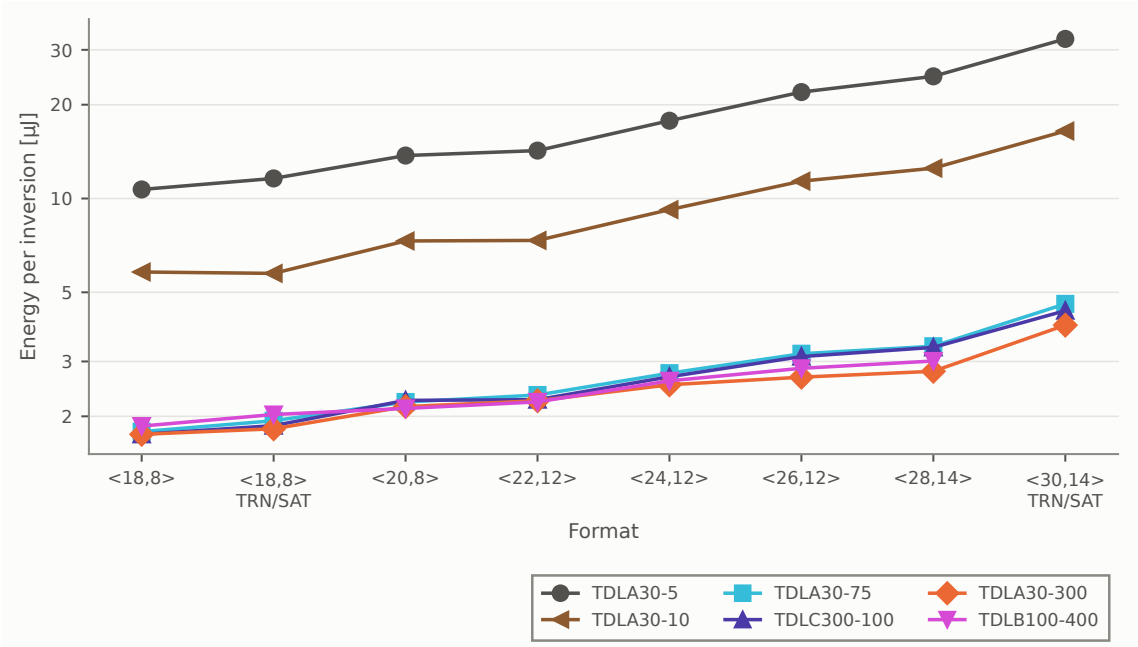
Fonte: a autora. Cálculo na Seção 4.6.3. *`AP_TRN`/`AP_SAT`; os outros, `AP_RND_ZERO`/`AP_WRAP`. –: sem medida ou não cabe no FPGA.

A Figura 5.11 junta, por formato, a ocupação do recurso mais usado (Tabela 5.3) e a potência dinâmica. Da TDLA30-5 à TDLC300-100, a potência dinâmica fica entre 7 e 46% da estática. Na TDLA30-300, ela vai de 61 a 135%, e na TDLB100-400, de 128 a 204%.

5.6 Vazão e atraso

As arquiteturas foram feitas para o limite de vazão, e atender um limite de atraso ficou fora do escopo (Seção 4.4.2). Esta seção mostra o atraso que resulta delas e por que ele não atende a latência prevista para o 5G. O atraso vai da chegada da primeira amostra de um bloco até a última inversão desse bloco. A primeira parte é a coleta das N amostras (Seção 4.4.1). Com 48 REs de DM-RS por PRB e por slot, ela leva $\lceil N/48 \rceil$ slots, limitada a T_c. A segunda é o tempo para inverter o bloco (Equação 3.4). A Tabela 5.6 mostra essas

Figure 5.10: Energia por inversão por formato e condição de teste.



Fonte: a autora. Eixo vertical em escala logarítmica. Sem TRN/SAT no rótulo, o formato usa `AP_RND_ZERO`/`AP_WRAP`. O $\langle 30, 14\rangle$ da TDLB100-400 não cabe no FPGA.

duas partes no formato $\langle 24, 12\rangle$.

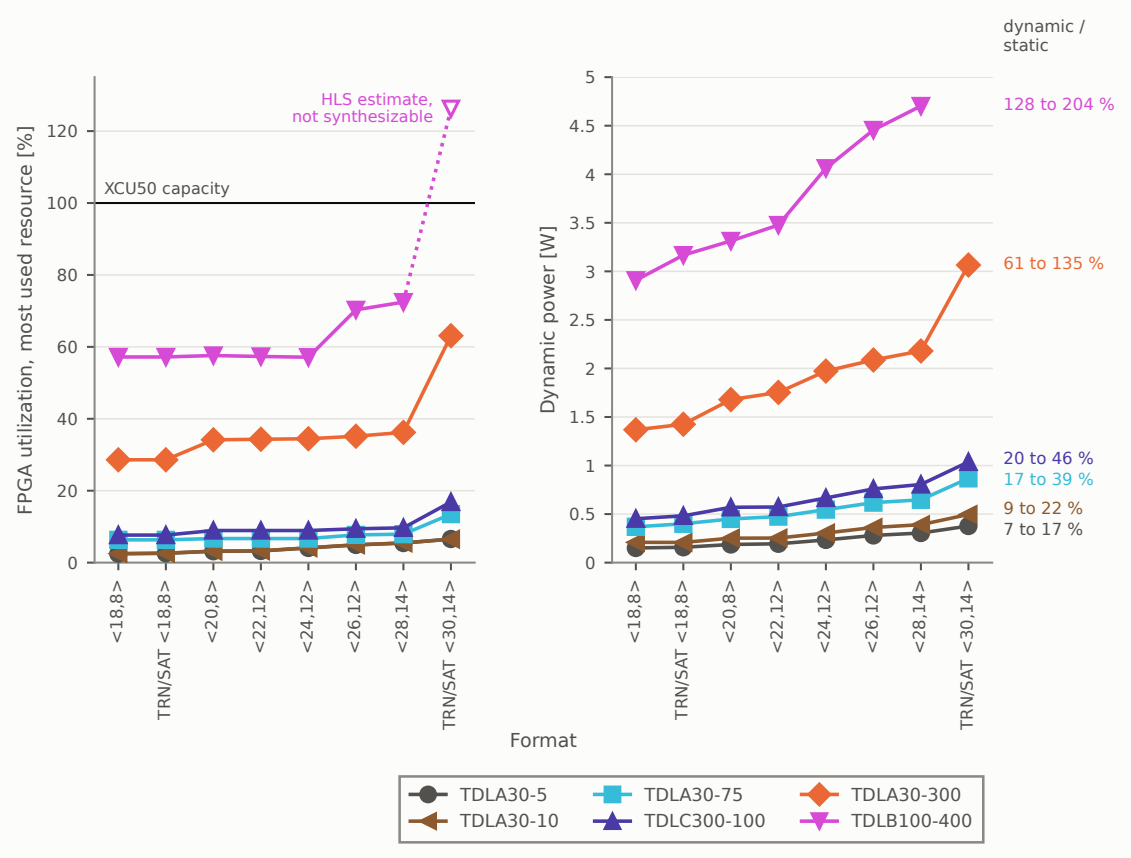
Table 5.6: Throughput and delay of one block of $N_{\mathrm{PRB}} = 275$ inversions, format $\langle 24, 12\rangle$.

Test condition	T_c (ms)	Block input (ms)	Block end (ms)	Collection (ms)	Total delay (ms)
TDLA30-5	25.000	20.884	20.884	2.000	22.884
TDLA30-10	12.500	8.305	8.354	2.000	10.354
TDLA30-75	1.667	1.391	1.543	0.500	2.043
TDLC300-100	1.250	1.159	1.336	1.250	2.586
TDLA30-300	0.417	0.369	0.506	0.417	0.923
TDLB100-400	0.313	0.184	0.265	0.313	0.578

Source: the author, from the netlist simulation at 100 MHz. Block input: time to accept the 275 matrices. Block end: time until the last inversion of the block. Total delay: collection plus block end.

Como todas as arquiteturas atendem o limite de ciclos (Seção 5.5), o bloco inteiro entra no módulo antes de T_c, e não se forma fila. Nas condições TDLC300-100 e TDLA30-300, a última inversão do bloco termina depois de T_c. Esse excesso não se acumula, pois o bloco seguinte já entra no módulo a tempo. Ele é um atraso fixo, de cerca de uma latência de batch, e não uma perda de vazão. Além disso, cada PRB só precisa da sua própria inversão: os primeiros PRBs ficam prontos cerca de uma latência de batch depois da chegada do bloco, e o atraso da tabela é o do último PRB.

Figure 5.11: Ocupação do recurso mais usado do XCU50 (esquerda) e potência dinâmica (direita), por formato e condição de teste.



Fonte: a autora. À direita, a potência dinâmica em porcentagem da estática, do menor ao maior formato. No $\langle 30, 14 \rangle$ da TDLB100-400, a ocupação é a estimativa do HLS.

A ITU-R prevê que o IMT-2020 possa oferecer latência de 1 ms pela interface aérea (ITU-R, 2015, p. 14). Essa latência é a contribuição da rede de rádio para o tempo entre o envio e a recepção de um pacote (ITU-R, 2015, p. 13). O processamento na BS é uma das parcelas da latência do plano de usuário (ITU-R, 2017, Sec. 7.2.6 e Tab. 3). Só nas condições TDLA30-300 e TDLB100-400 o atraso fica abaixo de 1 ms. Nas outras, o atraso vai de 2,0 a 22,9 ms.

Parte desse atraso vem do método de estimação, e não do hardware. Com $N = 192$ em FR1, a coleta leva 4 slots de 0,5 ms, ou 2 ms, e já passa de 1 ms antes de qualquer inversão. A outra parte vem da escolha de projeto. As arquiteturas das condições de baixa mobilidade distribuem as 275 inversões por quase todo o T_c. Com isso, elas gastam menos área e potência (Seção 5.5), mas o último PRB fica pronto mais tarde. Uma arquitetura mais rápida reduziria essa parte do atraso, com mais área e potência. A arquitetura da TDLB100-400, por exemplo, termina um bloco em 0,27 ms.

A coleta também depende do PUSCH. O DM-RS só é transmitido junto com

o PUSCH, e o PUSCH só ocupa os slots em que os UEs alvo foram escalonados (Seção 3.1.2). Este trabalho supõe que todo slot leva PUSCH. As bandas n78 e n257 usam TDD (3GPP, 2020, Tabs. 5.2-1 e 5.2-2), e só os slots de uplink levam o PUSCH. Com um padrão TDD real, os slots de uplink ficam mais espaçados. A coleta das N amostras leva mais tempo ou, se passar de T_c, o número de amostras disponível diminui.

5.7 Aumento de SNR e potência

As Figuras 5.12 e 5.13 juntam o aumento de SNR e a potência dinâmica nos grupos 1 e 3. Cada ponto é um formato numa condição de teste. No eixo x, está o pior aumento de SNR entre as configurações do grupo com o IoTh do gráfico, naquela condição: 10 dB (A a D) à esquerda e 20 dB (E a H) à direita. No gráfico da direita, o pior caso é o mesmo de todo o grupo (A a H). No eixo y, está a potência dinâmica do formato na arquitetura da condição, e a linha horizontal marca a potência estática.

Só os grupos 1 e 3 aparecem, porque eles usam o circuito 16×16. No grupo 2 (32×32), só a versão sem otimização foi implementada, e ela não atende nenhum limite. A potência foi medida só com matrizes do grupo 1 (Seção 4.6.2), e isso é uma limitação para o grupo 3. Mas os dados de entrada parecem mudar pouco a potência: na versão sem otimização do $\langle 18, 8 \rangle$, medida com as matrizes de cada uma das 5 condições de teste, a potência dinâmica ficou entre 0,031 e 0,032 W.

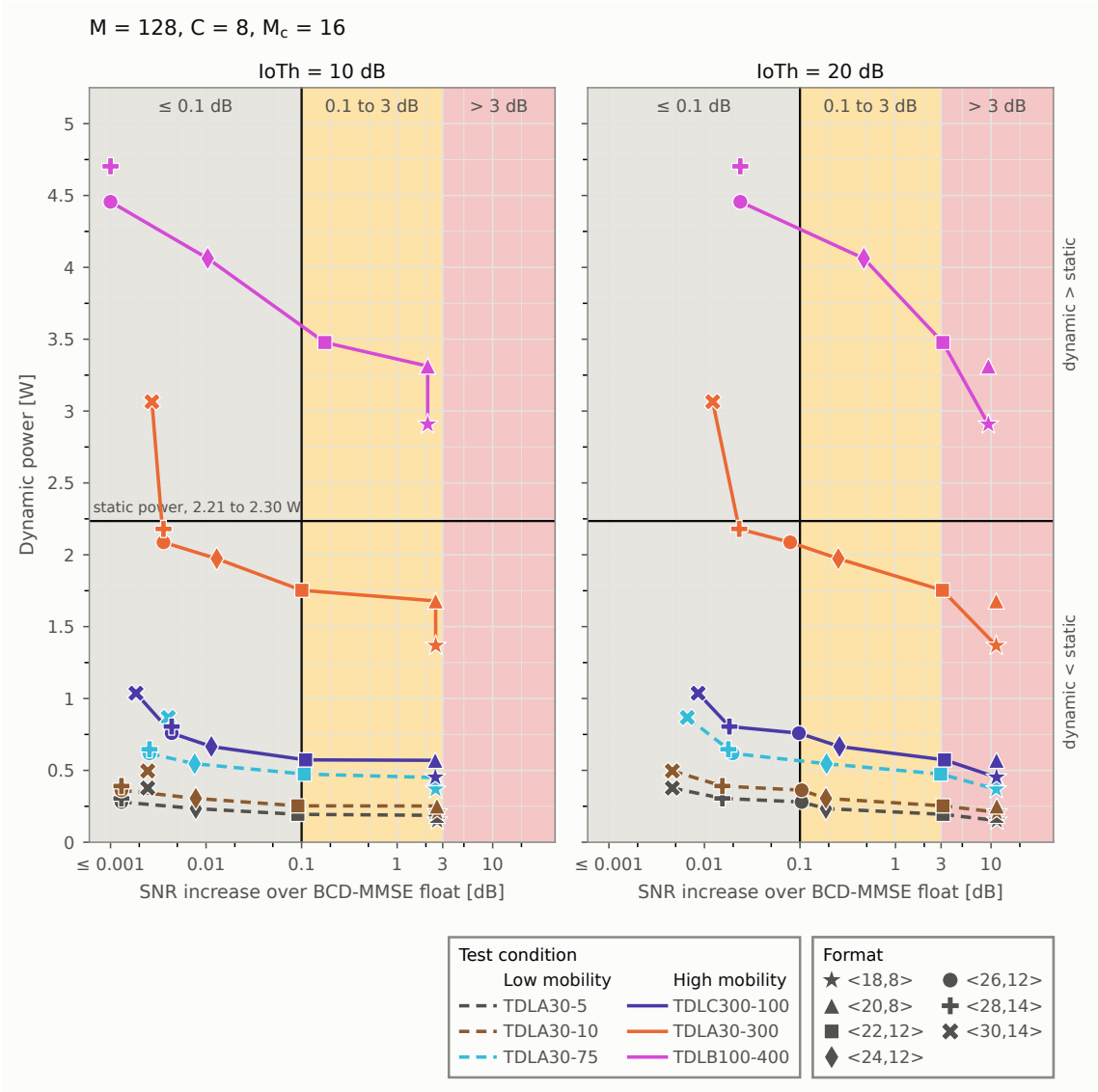
Como a potência muda mais com a arquitetura do que com o formato (Seção 5.5.3), cada condição forma uma faixa de altura própria nas figuras. A Tabela 5.7 mostra o formato escolhido em cada condição de teste: o menor que cabe na arquitetura e fica dentro do limite.

No grupo 1, com IoTh de 20 dB, o limite de 3 dB fica no $\langle 24, 12 \rangle$ em quase todas as condições. Na TDLA30-75, o $\langle 22, 12 \rangle$ já basta, com 2,97 dB. Para 0,1 dB, todas as condições usam o $\langle 26, 12 \rangle$. Na TDLA30-5 e na TDLA30-10, ele fica em 0,104 dB, que arredonda para 0,10 dB. Passar de 3 dB para 0,1 dB aumenta a potência dinâmica de 6% (TDLA30-300) a 30% (TDLA30-75).

Na TDLB100-400, o $\langle 24, 12 \rangle$ fica 0,47 dB acima do BCD-MMSE em ponto flutuante. Esse aumento pede que cada UE transmita 11% mais energia por símbolo (Seção 4.5.1). O $\langle 26, 12 \rangle$ tira essa diferença com 0,39 W a mais de potência dinâmica no FPGA.

No grupo 3, com IoTh de 20 dB, o limite de 3 dB fica no $\langle 24, 12 \rangle$ em todas as

Figure 5.12: Aumento de SNR e potência dinâmica de cada formato nas condições de teste, grupo 1.

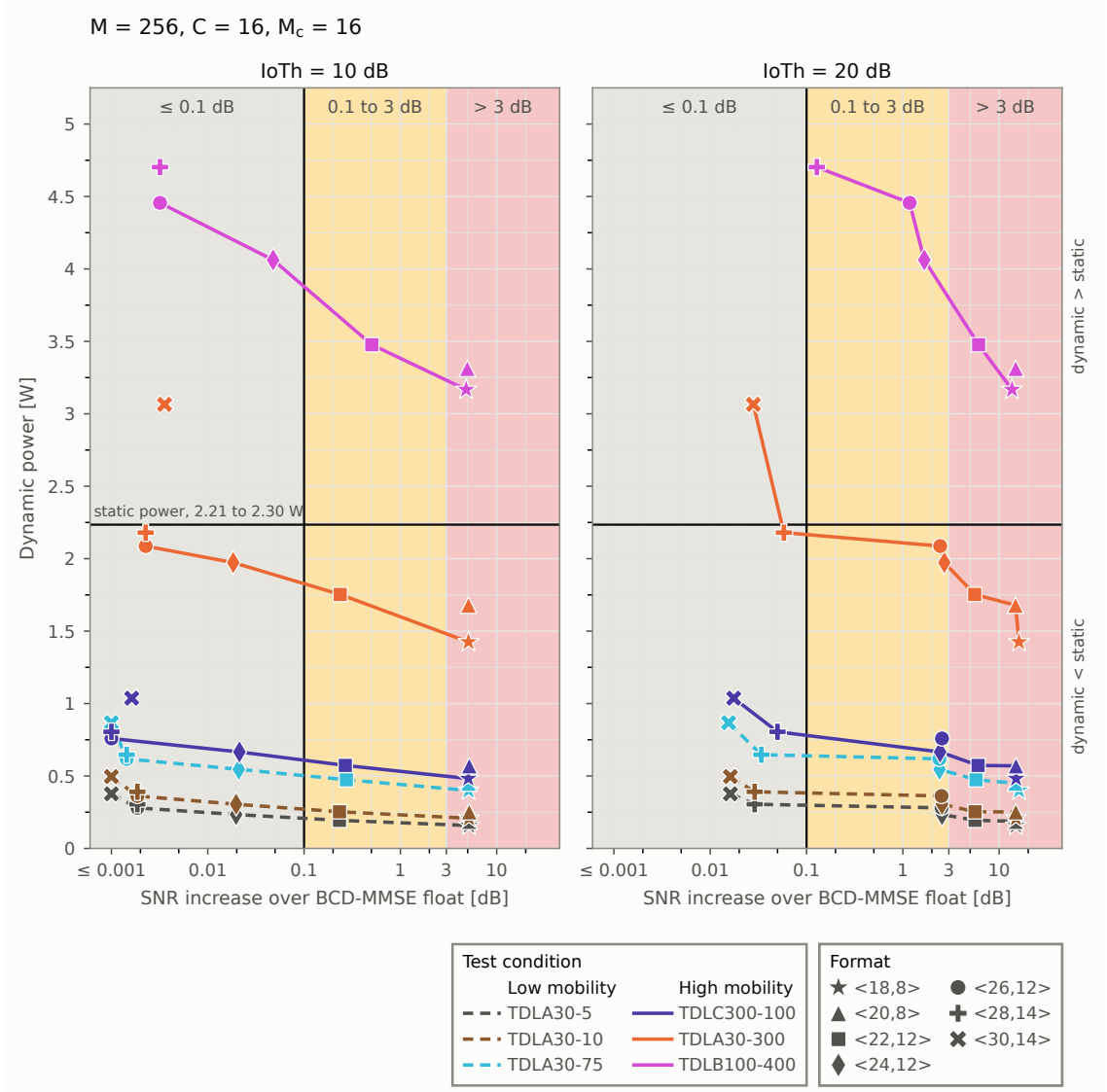


Fonte: a autora. Linha tracejada: baixa mobilidade; cheia: alta mobilidade. As linhas ligam os formatos que nenhum outro supera ao mesmo tempo em aumento de SNR e em potência.

condições. O de 0,1 dB fica no $\langle 28, 14\rangle$, com 10 a 30% mais potência dinâmica. Na TDLB100-400, nenhum formato que cabe no FPGA atende 0,1 dB. O $\langle 28, 14\rangle$ fica em 0,13 dB, e o $\langle 30, 14\rangle$ não cabe.

Com IoTh de 10 dB, os formatos ficam menores e só mudam entre as condições no grupo 1, para 0,1 dB: na TDLA30-5, na TDLA30-10 e na TDLA30-300, basta o $\langle 22, 12\rangle$.

Figure 5.13: Aumento de SNR e potência dinâmica de cada formato nas condições de teste, grupo 3.



Fonte: a autora. Linha tracejada: baixa mobilidade; cheia: alta mobilidade. As linhas ligam os formatos que nenhum outro supera ao mesmo tempo em aumento de SNR e em potência.

Table 5.7: Formato escolhido em cada condição de teste, com o pior aumento de SNR, em dB, e a potência dinâmica, em W.

	IoTh of 10 dB (A to D)						IoTh of 20 dB (E to H)					
	Up to 3 dB			Up to 0.1 dB			Up to 3 dB			Up to 0.1 dB		
Test condition	**Format**	**dB**	**W**	**Format**	**dB**	**W**	**Format**	**dB**	**W**	**Format**	**dB**	**W**
Group 1 (M = 128, C = 8)												
Low mobility												
TDLA30-5	$\langle 18, 8\rangle$	2.62	0.15	$\langle 22, 12\rangle$	0.09	0.19	$\langle 24, 12\rangle$	0.19	0.23	$\langle 26, 12\rangle$	0.10	0.28
TDLA30-10	$\langle 18, 8\rangle$	2.62	0.21	$\langle 22, 12\rangle$	0.09	0.25	$\langle 24, 12\rangle$	0.19	0.30	$\langle 26, 12\rangle$	0.10	0.36
TDLA30-75	$\langle 18, 8\rangle$	2.52	0.37	$\langle 24, 12\rangle$	0.01	0.55	$\langle 22, 12\rangle$	2.97	0.47	$\langle 26, 12\rangle$	0.02	0.62
High mobility												
TDLC300-100	$\langle 18, 8\rangle$	2.51	0.45	$\langle 24, 12\rangle$	0.01	0.67	$\langle 24, 12\rangle$	0.26	0.67	$\langle 26, 12\rangle$	0.10	0.76
TDLA30-300	$\langle 18, 8\rangle$	2.53	1.37	$\langle 22, 12\rangle$	0.10	1.75	$\langle 24, 12\rangle$	0.25	1.97	$\langle 26, 12\rangle$	0.08	2.09
TDLB100-400	$\langle 18, 8\rangle$	2.09	2.91	$\langle 24, 12\rangle$	0.01	4.06	$\langle 24, 12\rangle$	0.47	4.06	$\langle 26, 12\rangle$	0.02	4.46
Group 3 (M = 256, C = 16)												
Low mobility												
TDLA30-5	$\langle 22, 12\rangle$	0.23	0.19	$\langle 24, 12\rangle$	0.02	0.23	$\langle 24, 12\rangle$	2.55	0.23	$\langle 28, 14\rangle$	0.03	0.30
TDLA30-10	$\langle 22, 12\rangle$	0.23	0.25	$\langle 24, 12\rangle$	0.02	0.30	$\langle 24, 12\rangle$	2.55	0.30	$\langle 28, 14\rangle$	0.03	0.39
TDLA30-75	$\langle 22, 12\rangle$	0.28	0.47	$\langle 24, 12\rangle$	0.02	0.55	$\langle 24, 12\rangle$	2.40	0.55	$\langle 28, 14\rangle$	0.03	0.65
High mobility												
TDLC300-100	$\langle 22, 12\rangle$	0.27	0.57	$\langle 24, 12\rangle$	0.02	0.67	$\langle 24, 12\rangle$	2.44	0.67	$\langle 28, 14\rangle$	0.05	0.81
TDLA30-300	$\langle 22, 12\rangle$	0.24	1.75	$\langle 24, 12\rangle$	0.02	1.97	$\langle 24, 12\rangle$	2.70	1.97	$\langle 28, 14\rangle$	0.06	2.18
TDLB100-400	$\langle 22, 12\rangle$	0.50	3.48	$\langle 24, 12\rangle$	0.05	4.06	$\langle 24, 12\rangle$	1.67	4.06	–	–	–

Fonte: a autora. Menor formato que cabe na arquitetura da condição de teste e cujo pior aumento de SNR, entre as configurações com aquele IoTh, fica até o limite. Modo `AP_RND_ZERO`/`AP_WRAP`. –: nenhum formato que cabe no FPGA atende o limite.

6 CONCLUSIONS

Este trabalho estudou o BCD-MMSE no *uplink* de um sistema MIMO massivo com processamento descentralizado e ruído colorido. O foco foi a inversão da matriz $\mathbf{G}_c$ em cada DU, feita em ponto fixo em um FPGA. Um simulador em Python mediu o efeito do formato de ponto fixo nas curvas de SER. Depois, o módulo de inversão foi implementado em hardware, com uma arquitetura para cada condição de teste.

[completar] Formatos escolhidos para cada grupo de configurações e aumento de SNR de cada um, em relação ao ponto flutuante (Seção 5.2).

[completar] Recursos e potência das arquiteturas. Todas atendem o limite de vazão da sua condição de teste.

As arquiteturas foram dimensionadas pela vazão, e não pelo atraso. Nas condições de baixa mobilidade, elas distribuem as inversões ao longo do tempo de coerência e gastam menos área e potência. Em troca, o atraso até a última inversão de um bloco fica maior. Parte do atraso vem do próprio método de estimação, pois juntar as N amostras de ruído leva tempo.

6.1 Future work

Um trabalho futuro pode incluir um prazo de atraso no dimensionamento do hardware e medir quanto ele custa em área e potência. Também pode considerar um padrão TDD real, que muda o tempo de coleta e o número de amostras disponível. Outras extensões são levar ao hardware as outras inversões do BCD-MMSE, que hoje ficam em ponto flutuante, e avaliar mais de uma iteração do algoritmo.

Também fica para um trabalho futuro reescalar $\mathbf{G}_c$ antes da inversão. Hoje, a entrada do módulo não é reescalada, e os elementos de $\mathbf{G}_c$ crescem quando a SNR cai. Por isso, QPSK e IoTh de 20 dB pedem mais bits inteiros (Seção 5.4).

REFERENCES

3GPP. *NR; Base Station (BS) radio transmission and reception*. [S.l.], 2020.

3GPP. *NR; Physical channels and modulation*. [S.l.], 2025.

AMD. **Vitis High-Level Synthesis User Guide (UG1399)**. 2026.1. ed. [S.l.], 2026. Disponível em: docs.amd.com.

ITU-R. **IMT Vision – Framework and overall objectives of the future development of IMT for 2020 and beyond**. [S.l.], 2015.

ITU-R. **Guidelines for evaluation of radio interface technologies for IMT-2020**. [S.l.], 2017.

JAECKEL, S.; RASCHKOWSKI, L.; THIELE, L. **QuaDRiGa – Quasi Deterministic Radio Channel Generator: User Manual and Documentation**. Document revision v2.8.1. Berlin, 2023.

JEON, C. et al. Decentralized equalization with feedforward architectures for massive mu-mimo. **IEEE Transactions on Signal Processing**, v. 67, n. 17, p. 4418–4432, 2019.

KASTNER, R.; MATAI, J.; NEUENDORFFER, S. **Parallel Programming for FPGAs**. [S.l.]: arXiv, 2018. ArXiv:1805.03648.

KULKARNI, A.; OUAMEUR, M. A.; MASSICOTTE, D. Hardware topologies for decentralized large-scale mimo detection using newton method. **IEEE Transactions on Circuits and Systems I: Regular Papers**, v. 68, n. 9, p. 3732–3745, 2021.

LEDOIT, O.; WOLF, M. A well-conditioned estimator for large-dimensional covariance matrices. **Journal of Multivariate Analysis**, v. 88, n. 2, p. 365–411, 2004.

LI, K. et al. Decentralized coordinate-descent data detection and precoding for massive mu-mimo. In: **2019 IEEE International Symposium on Circuits and Systems (ISCAS)**. [S.l.: s.n.], 2019. p. 1–5.

LI, K. et al. Decentralized equalization for massive mu-mimo on fpga. In: **2017 51st Asilomar Conference on Signals, Systems, and Computers**. [S.l.: s.n.], 2017. p. 1532–1536.

LI, K. et al. Decentralized baseband processing for massive mu-mimo systems. **IEEE Journal on Emerging and Selected Topics in Circuits and Systems**, v. 7, n. 4, p. 491–507, 2017.

LIN, X.; LEE, N. (Ed.). **5G and Beyond: Fundamentals and Standards**. Cham: Springer, 2021.

OSINSKY, A. et al. Regularization for Cholesky decomposition in massive MIMO detection. **IEEE Wireless Communications Letters**, v. 12, n. 9, p. 1603–1607, 2023.

TOMASIN, S. et al. Estimation of interference correlation in mmwave cellular systems. **IEEE Transactions on Wireless Communications**, v. 23, n. 3, p. 1771–1782, 2024.

TSE, D.; VISWANATH, P. **Fundamentals of Wireless Communication**. Cambridge: Cambridge University Press, 2005.

WANG, H. et al. An efficient approximate expectation propagation detector with block-diagonal neumann-series. **IEEE Transactions on Circuits and Systems I: Regular Papers**, v. 70, n. 3, p. 1403–1416, 2023.

WANG, H. et al. An efficient detector for massive mimo based on improved matrix partition. **IEEE Transactions on Signal Processing**, v. 69, p. 2971–2986, 2021.

ZHAO, X. et al. Decentralized linear mmse equalizer under colored noise for massive mimo systems. In: **2021 IEEE Global Communications Conference (GLOBECOM)**. [S.l.: s.n.], 2021.

ZHAO, X. et al. Efficient lmmse equalization for massive mimo systems under decentralized baseband processing architecture. **IEEE Journal on Selected Areas in Communications**, v. 43, n. 3, p. 736–751, 2025.

ZHUANG, Z. et al. Decentralized mimo systems with imperfect csi using lmmse receivers. **IEEE Journal of Selected Topics in Signal Processing**, v. 19, n. 2, p. 331–348, 2025.

APPENDIX A — SIMULATION PARAMETERS

Table A.1: Channel parameters set in the MATLAB script that runs QuaDRiGa.

Source	Parameter	Name in the MATLAB script	Value
ZHAO et al., 2025	Number of cells and BSs	`tx_position` (one BS)	1
	Sector where the UEs are	`sector_deg`	120°
	Position of the interferers	`n_ue = K + r`	Same rule as the served UEs
	Channel generator	–	QuaDRiGa
	Channel constant within a PRB	–	Flat channel
QuaDRiGa default JAECKEL; RASCHKOWSKI; THIELE, 2023	Uses only the minimal version of the 3GPP model	`use_3GPP_baseline`	`false`
	Shape of the spatial correlation of the large-scale parameters	`autocorrelation_function`	`Comb300`
	Adds the direct-path delay to the delays	`use_absolute_delays`	`false`
	Each path starts with a random phase	`use_random_initial_phase`	`true`
	Samples per half wavelength along the track	`sample_density`	2.5
Defined in this work	QuaDRiGa version	`version`	2.8.1-0
	Propagation scenario	`scenario`	`3GPP_38.901_UMa_NLOS` (Table A.2)
	Bandwidth	`bandwidth`	20 MHz
	Flat-channel extraction	`fr(bandwidth, 1)`	Response at one subcarrier
	BS array	`tx_array`	ULA with $\lambda/2$ spacing
	Antenna elements (BS and UE)	`qd_arrayant('omni')`	Omnidirectional
	Sector orientation	`tx_array`, `ang`	Centered on the direction perpendicular to the array
	UE placement	`rx_track`	Uniform over the sector area
	Realizations	`rng(seed + rr)`	New positions in each realization

Source: ZHAO et al., 2025; JAECKEL; RASCHKOWSKI; THIELE, 2023 and the author.

Table A.2: Parameters of the propagation scenario `3GPP_38.901_UMa_NLOS`: valid range and values used in the simulation.

Set in	Parameter	Name	Valid range of the propagation scenario	Value used
MATLAB script	Carrier frequency	`center_freq`	0.5 to 100 GHz	3.5 GHz (FR1) and 28 GHz (FR2)
	BS–UE distance	`r_min_m`, `r_max_m`	10 to 5000 m	50 to 100 m
	BS height	`bs_height_m`	25 m	25 m
	UE height	`ue_height_m`	1.5 to 22.5 m	1.5 m
Scenario file (`.conf`)	Number of clusters	`NumClusters`	21	
	Paths per cluster	`NumSubPaths`	20	
	Shadowing standard deviation	`SF_sigma`	6 dB	

Source: JAECKEL; RASCHKOWSKI; THIELE, 2023; ZHAO et al., 2025 and the author.

Table A.3: Signal and noise parameters.

Source	Parameter	Name in the Python simulator	Value
ZHAO et al., 2025	Noise model	`M5_Noise.noise_model`	$\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H + \sigma^2\mathbf{I})$
	SNR definition	`M5_Noise.snr_definition`	$10\log_{10}(E_s/\sigma^2)$
	Mean symbol energy	`M4_Source.Es`	$E_s = 1$
	Thermal noise power	`M5_Noise.snr_definition`	$\sigma^2 = 10^{-\mathrm{SNR}/10}$
Defined in this work	Channel input	`M2_Channel.backend`	$\mathbf{H}$ and $\overline{\mathbf{H}}$ read from the `.npz` file of QuaDRiGa
	Channel normalization	`M2_Channel.normalization`	Same mean power for every UE
	Constellation energy	`M4_Source.unit_energy_constellation`	Unit mean
	Interferer power	`M5_Noise.beta_convention`	$\beta = \sigma^2(10^{\mathrm{IoTh}/10} - 1)/r$
	Noise of the data symbols	`M5_Noise.data_noise_independent`	Independent of the N samples

Source: ZHAO et al., 2025 and the author. IoTh and modulation are in Table 4.2.

Table A.4: Receiver parameters: partitioning, covariance estimation and equalization.

Source	Parameter	Name in the Python simulator	Value
ZHAO et al., 2025	Antennas per cluster	`M3_Clustering.equal_cluster_size`	$M_c = M/C$
	Noise samples to estimate $\hat{\mathbf{R}}$	`M5_Noise.n_noise_samples`	$N = 192$
	Covariance estimate	`M6_Covariance.estimator`	Sample mean of the N samples
	Channel knowledge at the BS	`M7_Equalizers.perfect_csi`	Perfect $\mathbf{H}$
	Compared equalizers	`M7_Equalizers.curves`	LMMSE, ZF and BCD-MMSE
	BCD-MMSE iterations	`M7_Equalizers.bcd_iterations`	1
	BCD update order	`M7_Equalizers.bcd_update_rule`	Sequential, $c = 1, \ldots, C$
	Computation of the equalizer $\mathbf{W}$	`M1_Driver.recompute_W_per_realization`	Once per realization and SNR point
Defined in this work	Antenna partitioning into clusters	`M3_Clustering.mapping`	Contiguous
	N in each test condition	`M1_Driver.test_conditions`	192, 168, 132 or 24 (Table 4.3)
	Noise sample draws	`M5_Noise.resample_noise_per_realization`	New samples at each realization and SNR point
	Conditioning of $\hat{\mathbf{R}}$	`M6_Covariance.shrinkage`	Ledoit–Wolf shrinkage
	ZF form	`M7_Equalizers.zf_form`	$(\mathbf{H}^H\mathbf{H})^{-1}\mathbf{H}^H$

Source: ZHAO et al., 2025 and the author. C and K are in Table 4.2. Names in the code: `dbp_sim/params.py`.

Table A.5: Evaluation parameters.

Source	Parameter	Name in the Python simulator	Value
ZHAO et al., 2025	Channel realizations	`M1_Driver.n_realizations`	100
	Symbols per realization	`M1_Driver.n_symbols`	480
	Detection	`M8_Metric.detection`	Nearest constellation point
	Metric	`M8_Metric.metric`	Symbol error rate (SER)
	SNR range	`M1_Driver.snr_db_grid`	-10 to 10 dB, 2 dB step
Defined in this work	Bias removal before detection	`M8_Metric.bias_removal`	No
	Random seed	`M1_Driver.rng_seed`	0

Source: ZHAO et al., 2025 and the author. Names in the code: `dbp_sim/params.py`.

APPENDIX B — HARDWARE RESOURCES

Table B.1: Recursos, caminho crítico e intervalo entre inversões da versão sem otimização, por formato.

Format	LUT	FF	DSP	BRAM	Critical path (ns)	Interval (cycles)
16 × 16						
⟨18, 8⟩ RND_ZERO/WRAP	4,095 (0.5)	2,195 (0.1)	87 (1.5)	9 (0.3)	7.80	26,702
⟨18, 8⟩ TRN/SAT	4,920 (0.6)	2,253 (0.1)	83 (1.4)	9 (0.3)	7.95	26,735
⟨20, 8⟩ RND_ZERO/WRAP	4,597 (0.5)	2,792 (0.2)	87 (1.5)	13 (0.5)	8.20	30,398
⟨22, 12⟩ RND_ZERO/WRAP	4,907 (0.6)	3,063 (0.2)	87 (1.5)	13 (0.5)	8.09	30,398
⟨24, 12⟩ RND_ZERO/WRAP	6,378 (0.7)	3,399 (0.2)	94 (1.6)	11 (0.4)	7.84	32,462
⟨26, 12⟩ RND_ZERO/WRAP	6,824 (0.8)	3,839 (0.2)	111 (1.9)	11 (0.4)	8.18	34,374
⟨28, 14⟩ RND_ZERO/WRAP	7,027 (0.8)	4,089 (0.2)	117 (2.0)	11 (0.4)	8.36	35,414
⟨30, 14⟩ TRN/SAT	9,086 (1.0)	4,297 (0.2)	184 (3.1)	13 (0.5)	9.15	38,567
32 × 32						
⟨18, 8⟩ RND_ZERO/WRAP	4,162 (0.5)	2,268 (0.1)	87 (1.5)	9 (0.3)	8.29	–
⟨18, 8⟩ TRN/SAT	4,984 (0.6)	2,328 (0.1)	83 (1.4)	9 (0.3)	8.32	–
⟨20, 8⟩ RND_ZERO/WRAP	4,667 (0.5)	2,865 (0.2)	87 (1.5)	18 (0.7)	8.03	–
⟨22, 12⟩ RND_ZERO/WRAP	4,971 (0.6)	3,136 (0.2)	87 (1.5)	18 (0.7)	7.72	159,086
⟨24, 12⟩ RND_ZERO/WRAP	6,351 (0.7)	3,474 (0.2)	94 (1.6)	18 (0.7)	8.03	167,310
⟨26, 12⟩ RND_ZERO/WRAP	6,855 (0.8)	3,916 (0.2)	111 (1.9)	18 (0.7)	8.58	174,974
⟨28, 14⟩ RND_ZERO/WRAP	7,058 (0.8)	4,166 (0.2)	117 (2.0)	18 (0.7)	8.43	179,102
⟨30, 14⟩ TRN/SAT	9,042 (1.0)	4,372 (0.3)	184 (3.1)	18 (0.7)	8.54	194,367

Fonte: a autora. Entre parênteses, a porcentagem do XCU50. BRAM em blocos de 18 Kb. –: sem medida.

Table B.2: Recursos, caminho crítico e intervalo entre inversões das arquiteturas otimizadas, por formato.

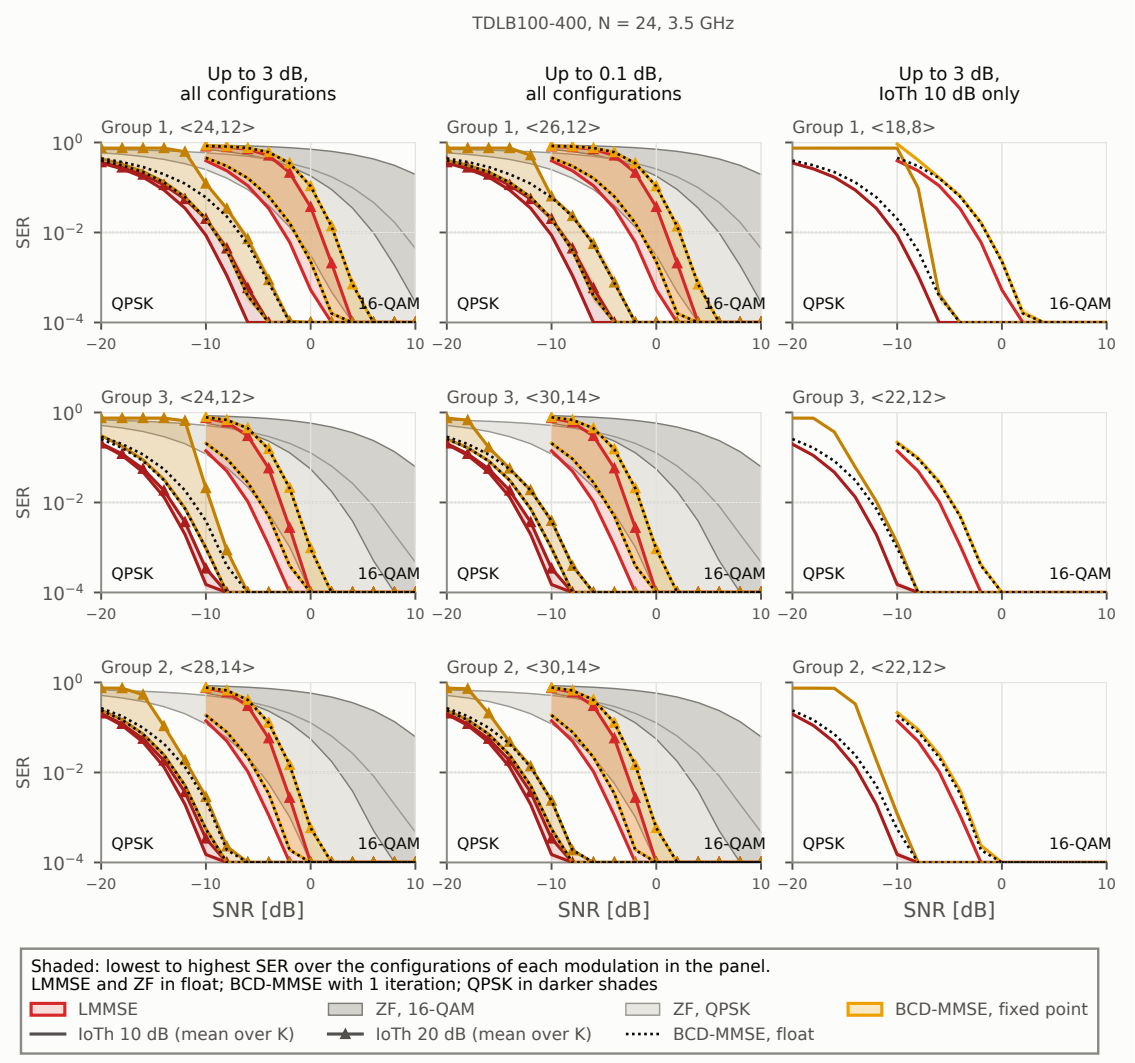
Format	LUT	FF	DSP	BRAM	Critical path (ns)	Interval (cycles)
TDLA30-5 (limit: 9,090 cycles)						
⟨18, 8⟩ RND_ZERO/WRAP	21,577 (2.5)	21,148 (1.2)	99 (1.7)	11 (0.4)	7.42	7,081
⟨18, 8⟩ TRN/SAT	22,731 (2.6)	21,425 (1.2)	99 (1.7)	11 (0.4)	8.06	7,342
⟨20, 8⟩ RND_ZERO/WRAP	27,571 (3.2)	27,126 (1.6)	99 (1.7)	15 (0.6)	7.24	7,346
⟨22, 12⟩ RND_ZERO/WRAP	28,296 (3.2)	28,748 (1.6)	99 (1.7)	15 (0.6)	7.62	7,345
⟨24, 12⟩ RND_ZERO/WRAP	35,604 (4.1)	35,600 (2.0)	106 (1.8)	15 (0.6)	7.45	7,594
⟨26, 12⟩ RND_ZERO/WRAP	42,897 (4.9)	42,872 (2.5)	126 (2.1)	15 (0.6)	7.76	7,842
⟨28, 14⟩ RND_ZERO/WRAP	47,449 (5.4)	47,927 (2.7)	128 (2.2)	15 (0.6)	7.76	8,087
⟨30, 14⟩ TRN/SAT	57,285 (6.6)	56,508 (3.2)	220 (3.7)	15 (0.6)	9.03	8,627
TDLA30-10 (limit: 4,545 cycles)						
⟨18, 8⟩ RND_ZERO/WRAP	21,779 (2.5)	21,306 (1.2)	99 (1.7)	14 (0.5)	7.54	2,780
⟨18, 8⟩ TRN/SAT	22,736 (2.6)	21,603 (1.2)	99 (1.7)	14 (0.5)	8.36	2,765
⟨20, 8⟩ RND_ZERO/WRAP	27,825 (3.2)	27,274 (1.6)	99 (1.7)	25 (0.9)	7.54	2,900
⟨22, 12⟩ RND_ZERO/WRAP	28,540 (3.3)	28,904 (1.7)	99 (1.7)	25 (0.9)	7.66	2,900
⟨24, 12⟩ RND_ZERO/WRAP	35,940 (4.1)	35,730 (2.0)	106 (1.8)	25 (0.9)	7.55	3,020
⟨26, 12⟩ RND_ZERO/WRAP	43,181 (5.0)	43,042 (2.5)	126 (2.1)	22 (0.8)	8.27	3,140
⟨28, 14⟩ RND_ZERO/WRAP	47,727 (5.5)	48,105 (2.8)	128 (2.2)	22 (0.8)	7.63	3,200
⟨30, 14⟩ TRN/SAT	56,852 (6.5)	56,671 (3.3)	220 (3.7)	25 (0.9)	8.11	3,320
TDLA30-75 (limit: 606 cycles)						
⟨18, 8⟩ RND_ZERO/WRAP	31,951 (3.7)	27,211 (1.6)	377 (6.3)	96 (3.6)	8.85	483.9
⟨18, 8⟩ TRN/SAT	34,717 (4.0)	27,279 (1.6)	377 (6.3)	96 (3.6)	8.98	483.9
⟨20, 8⟩ RND_ZERO/WRAP	39,080 (4.5)	33,904 (1.9)	377 (6.3)	180 (6.7)	8.84	493.9
⟨22, 12⟩ RND_ZERO/WRAP	40,337 (4.6)	35,826 (2.1)	377 (6.3)	180 (6.7)	8.81	493.9
⟨24, 12⟩ RND_ZERO/WRAP	51,087 (5.9)	43,450 (2.5)	379 (6.4)	180 (6.7)	8.40	503.9
⟨26, 12⟩ RND_ZERO/WRAP	59,133 (6.8)	52,228 (3.0)	459 (7.7)	180 (6.7)	8.53	513.9
⟨28, 14⟩ RND_ZERO/WRAP	63,997 (7.3)	56,982 (3.3)	471 (7.9)	180 (6.7)	7.87	518.9
⟨30, 14⟩ TRN/SAT	78,510 (9.0)	68,929 (4.0)	803 (13.5)	180 (6.7)	8.70	528.9
TDLC300-100 (limit: 454 cycles)						
⟨18, 8⟩ RND_ZERO/WRAP	34,515 (4.0)	27,337 (1.6)	456 (7.7)	128 (4.8)	8.21	387.3
⟨18, 8⟩ TRN/SAT	38,237 (4.4)	27,663 (1.6)	456 (7.7)	128 (4.8)	9.02	387.3
⟨20, 8⟩ RND_ZERO/WRAP	42,135 (4.8)	34,427 (2.0)	456 (7.7)	240 (8.9)	8.93	394.8
⟨22, 12⟩ RND_ZERO/WRAP	43,487 (5.0)	36,344 (2.1)	456 (7.7)	240 (8.9)	9.37	394.8
⟨24, 12⟩ RND_ZERO/WRAP	55,482 (6.4)	44,110 (2.5)	456 (7.7)	240 (8.9)	8.95	402.3
⟨26, 12⟩ RND_ZERO/WRAP	63,231 (7.3)	53,101 (3.0)	560 (9.4)	240 (8.9)	8.69	409.8
⟨28, 14⟩ RND_ZERO/WRAP	68,616 (7.9)	57,333 (3.3)	576 (9.7)	240 (8.9)	7.79	413.6
⟨30, 14⟩ TRN/SAT	84,536 (9.7)	69,741 (4.0)	1,004 (16.9)	240 (8.9)	9.26	421.1
TDLA30-300 (limit: 151 cycles)						
⟨18, 8⟩ RND_ZERO/WRAP	95,842 (11.0)	64,774 (3.7)	1,702 (28.6)	538 (20.0)	8.99	128
⟨18, 8⟩ TRN/SAT	112,100 (12.9)	66,164 (3.8)	1,702 (28.6)	538 (20.0)	9.66	128
⟨20, 8⟩ RND_ZERO/WRAP	116,952 (13.4)	75,921 (4.4)	1,714 (28.8)	918 (34.2)	8.97	128
⟨22, 12⟩ RND_ZERO/WRAP	120,871 (13.9)	80,557 (4.6)	1,706 (28.7)	922 (34.3)	9.54	128
⟨24, 12⟩ RND_ZERO/WRAP	154,016 (17.7)	103,627 (5.9)	1,700 (28.6)	926 (34.4)	8.82	128
⟨26, 12⟩ RND_ZERO/WRAP	175,047 (20.1)	110,964 (6.4)	2,092 (35.1)	932 (34.7)	9.21	128
⟨28, 14⟩ RND_ZERO/WRAP	184,816 (21.2)	121,813 (7.0)	2,156 (36.2)	936 (34.8)	8.73	128
⟨30, 14⟩ TRN/SAT	236,708 (27.2)	147,578 (8.5)	3,756 (63.1)	940 (35.0)	9.26	128
TDLB100-400 (limit: 113 cycles)						
⟨18, 8⟩ RND_ZERO/WRAP	240,836 (27.6)	150,465 (8.6)	3,404 (57.2)	272 (10.1)	9.44	64
⟨18, 8⟩ TRN/SAT	277,257 (31.8)	140,110 (8.0)	3,404 (57.2)	268 (10.0)	9.60	64
⟨20, 8⟩ RND_ZERO/WRAP	289,316 (33.2)	174,219 (10.0)	3,428 (57.6)	524 (19.5)	9.62	64
⟨22, 12⟩ RND_ZERO/WRAP	299,409 (34.3)	187,085 (10.7)	3,412 (57.3)	524 (19.5)	9.40	64
⟨24, 12⟩ RND_ZERO/WRAP	372,697 (42.8)	234,826 (13.5)	3,400 (57.1)	532 (19.8)	9.74	64
⟨26, 12⟩ RND_ZERO/WRAP	421,164 (48.3)	251,084 (14.4)	4,184 (70.3)	536 (19.9)	9.42	64
⟨28, 14⟩ RND_ZERO/WRAP	444,699 (51.0)	278,059 (15.9)	4,312 (72.4)	544 (20.2)	9.38	64
⟨30, 14⟩ TRN/SAT	–	–	–	–	–	–

Fonte: a autora. 16×16. Entre parênteses, porcentagem do XCU50. BRAM de 18 Kb. –: não cabe no FPGA.

APPENDIX C — SER CURVES

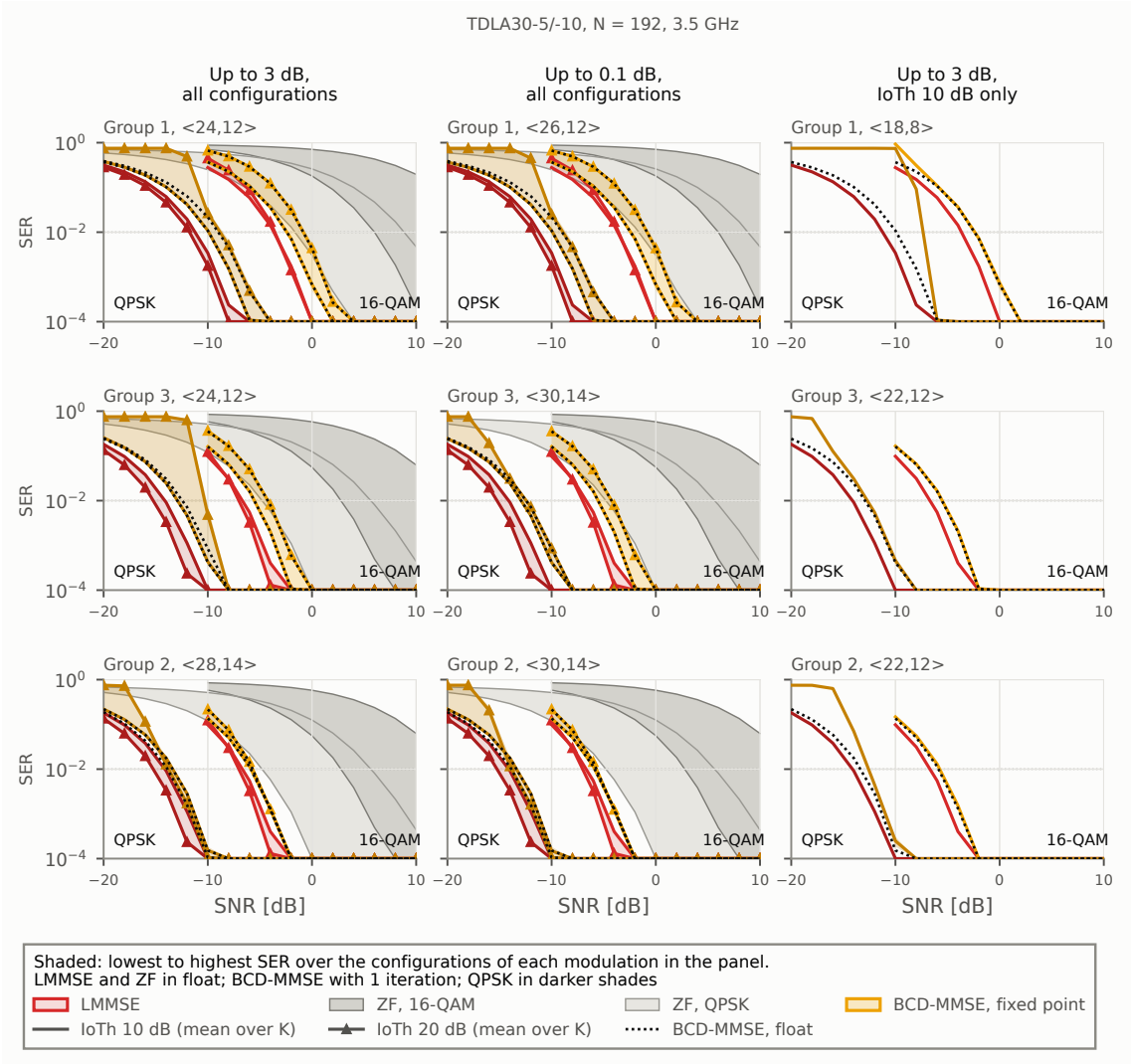
As Figuras C.1 e C.2 mostram as curvas de SER dos três grupos na TDLB100-400 e na TDLA30-5/TDLA30-10. Cada linha é um grupo. As duas primeiras colunas são os formatos escolhidos com todas as configurações, até 3 dB e até 0,1 dB (Tabela 5.1). A terceira coluna é o formato escolhido até 3 dB só com as configurações de IoTh de 10 dB, e, por isso, só tem essas configurações. Em cada painel, as curvas da esquerda são de QPSK, e as da direita, de 16-QAM. Cada faixa vai do menor ao maior SER entre as configurações da modulação. As linhas são a média entre os dois valores de K em cada IoTh.

Figure C.1: Curvas de SER dos três grupos na TDLB100-400, nos formatos escolhidos.



Fonte: a autora.

Figure C.2: Curvas de SER dos três grupos na TDLA30-5 e na TDLA30-10, nos formatos escolhidos.

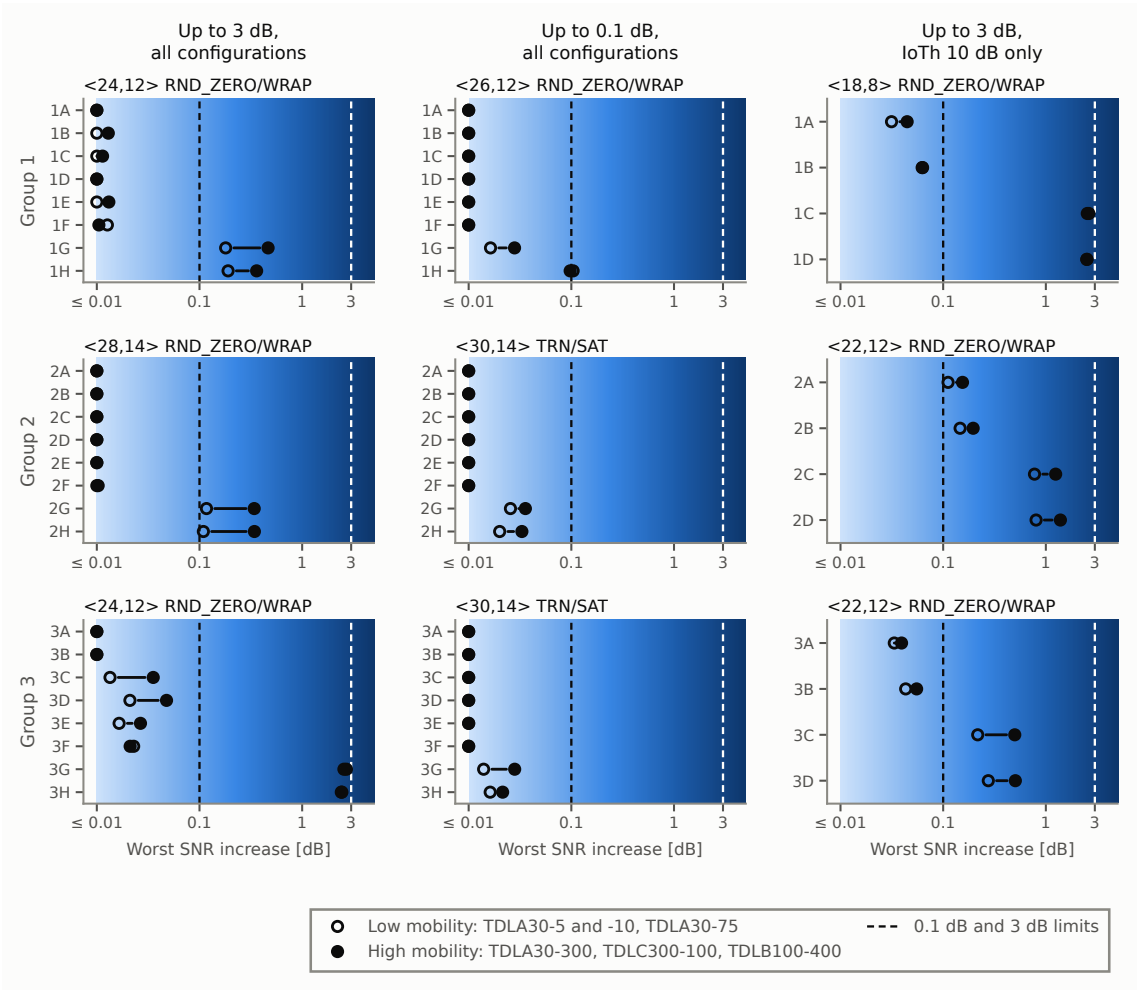


Fonte: a autora.

APPENDIX D — SNR INCREASE PER CONFIGURATION

Figure D.1 breaks down the largest SNR increase by mobility profile (Section 4.3.2). Each point represents the largest increase among the test conditions of a profile. Each row corresponds to a group, and the columns follow the structure of Table 5.1: the first two columns include all configurations, the first with the 3 dB format and the second with the 0.1 dB format, while the third column includes only the 10 dB IoTh configurations, with the 3 dB format selected specifically for that subset. When the difference between them exceeds 0.1 dB, the high-mobility profile shows the larger SNR increase (filled point to the right of the unfilled one). The largest observed difference is 0.58 dB, in configuration 2D with $\langle 22, 12 \rangle$: 0.80 dB for low mobility versus 1.39 dB for high mobility.

Figure D.1: Largest SNR increase, in dB, of each configuration in each mobility profile, in the three groups, with the selected formats.



Source: the author.